

Índice

Especificación de Requisitos de Software (SRS) — BIOPET	2
1. Introducción	3
1.1. Propósito	3
1.2. Alcance	3
1.3. Definiciones, acrónimos y abreviaturas	3
1.4. Referencias	5
1.5. Resumen del documento	5
1.6. Control documental e historial de revisiones	6
2. Descripción global	7
2.1. Perspectiva del producto	7
2.2. Cambio de plataforma tecnológica respecto a la Entrega 1A	7
2.3. Funciones del producto (resumen)	7
2.4. Características de los usuarios	8
2.5. Restricciones	8
2.6. Supuestos y dependencias	8
2.7. Interfaces externas	9
2.8. Matriz de permisos por rol y operación	10
2.9. Estados principales del dominio y transiciones	12
3. Requisitos específicos	14
3.1. Requisitos funcionales (REQ-F)	14
REQ-F-001 — Registro de usuario dueño de mascota	14
REQ-F-002 — Rechazo de registro con correo duplicado	15
REQ-F-003 — Autenticación mediante usuario y contraseña	15
REQ-F-004 — Renovación de sesión (refresh)	16
REQ-F-005 — Cierre de sesión con revocación de token	16
REQ-F-006 — Control de acceso por rol y por propietario del recurso (RBAC + aislamiento)	17
REQ-F-007 — Consulta del perfil propio	18
REQ-F-008 — Creación de mascota	18
REQ-F-009 — Listado paginado de mascotas activas, filtrado por propietario según rol	19
REQ-F-010 — Consulta de mascota por identificador	19
REQ-F-011 — Actualización de mascota con verificación de propiedad	20
REQ-F-012 — Baja lógica de una mascota	20
REQ-F-013 — Registro de atención médica y consulta del historial clínico	21
REQ-F-014 — Registro de medicamentos prescritos	22
REQ-F-015 — Gestión de citas veterinarias	23
REQ-F-016 — API de recepción de telemetría de dispositivos IoT	24
REQ-F-017 — Generación de recomendaciones clínicas informativas a partir del historial médico	25
REQ-F-018 — Comprobantes de pago digitales	26
REQ-F-019 — Reportes estadísticos exportables	26
REQ-F-020 — Auditoría de operaciones del sistema	27
REQ-F-021 — Resumen de mascotas activas agrupadas por especie	28
REQ-F-022 — Notificaciones al usuario por correo electrónico (RF-07 recuperado)	28
REQ-F-023 — Gestión administrativa de usuarios	29
REQ-F-024 — Gestión de vacunas	30
REQ-F-025 — Consulta de información externa de especies	31
REQ-F-026 — Recuperación de contraseña	31
3.2. Requisitos no funcionales (REQ-NF)	32
REQ-NF-001 — Rendimiento del listado de mascotas	32
REQ-NF-002 — Comunicación cifrada obligatoria	33
REQ-NF-003 — Expiración configurable de tokens JWT	33
REQ-NF-004 — Claims estándar del JWT	34
REQ-NF-005 — Interfaz responsiva	34

REQ-NF-006 — Compatibilidad de navegadores	35
REQ-NF-007 — Verificación de estado y medición de disponibilidad	36
REQ-NF-008 — Cifrado de contraseñas	37
REQ-NF-009 — Registro de eventos de autenticación (OWASP A09)	37
REQ-NF-010 — Limitación de intentos de login (OWASP A07)	38
REQ-NF-011 — Arquitectura en capas	39
REQ-NF-012 — Caché del listado de mascotas: TTL configurable y medición de aciertos	39
REQ-NF-013 — Estrategia híbrida de acceso a datos (ORM + procedimientos almacenados)	41
REQ-NF-014 — Comportamiento del sistema ante indisponibilidad de Redis	42
REQ-NF-015 — Aislamiento de datos por propietario (requisito transversal)	43
REQ-NF-016 — Política de contraseñas	44
REQ-NF-017 — Respaldo y recuperación de la base de datos	45
REQ-NF-018 — Accesibilidad de la interfaz	46
3.3. Criterios INVEST aplicados a las historias de usuario	47
3.4. Tabla de control de completitud de requisitos	49
4. Trazabilidad (resumen)	52
4.1. Trazabilidad histórica: identificadores originales → identificadores actuales	53
5. Modelo de datos (referencia)	55
6. Interfaces de usuario (referencia)	55
7. Observaciones e información pendiente	56
7.1. Cerrado en revisiones anteriores	56
7.2. Cerrado en la revisión 2026-09-11 (esta revisión)	56
7.3. Limitaciones declaradas que siguen abiertas	57
7.4. Acciones que requieren cambio de código (fuera del alcance de esta revisión)	58

Especificación de Requisitos de Software (SRS) — BIOPET

Control documental

Versión documental	v1.0.0
Revisión	2026-09-11
Fecha de emisión de esta revisión	11 de septiembre de 2026
Tag histórico de referencia	v1.0.0 (0d5cd525ce648cca7219da204e16fa622e671a87, 2026-08-18)
Commit de revisión/corrección actual	8130ee00b0083808fc60567018589e38302b375d
Repositorio	https://github.com/Grinjoww/ENTREGA-FINAL-BIOPET

Aviso de correspondencia con el tag (importante). El tag v1.0.0 es inmutable y **no se mueve, no se borra y no se recrea** en esta revisión. Esta copia del SRS **no** corresponde byte a byte al contenido del tag v1.0.0: el documento fue corregido después de ese tag (verificable con `git diff v1.0.0 -- docs/requisitos/`). Por eso se declaran por separado la *versión documental* (v1.0.0, que no cambia porque no se altera el alcance comprometido), el *tag histórico de referencia* y el *commit de revisión* real sobre el que se generó esta copia. Cualquier afirmación de evidencia reproducible de este documento debe reproducirse sobre el commit indicado como “commit de revisión”, salvo cuando el propio requisito diga explícitamente que la evidencia se generó sobre el commit del tag.

Conforme a: ISO/IEC/IEEE 29148:2018 (estructura de SRS), INCOSE Guide to Writing Requirements v4 (calidad de requisitos individuales y de conjunto), criterios INVEST de Cohn (historias de usuario, aplicados explícitamente en la sección 3.3), plantilla de Cockburn (casos de uso).

Estado de aprobación: este documento **no está firmado** por el docente-director al momento de esta revisión. El PDF `SRS-v1.0.0-para-firma.pdf` se envía para aprobación/firma tras cerrar esta revisión; el

estado de firma se actualizará aquí, con fecha real, únicamente cuando exista una firma legítima recibida del docente — nunca antes, y nunca simulada o copiada de otro documento.

Procedencia de este documento: este archivo se reconstruye a partir de dos fuentes: (1) el SRS de la Entrega 1A (PFC_Entrega1A_BMT.pdf), que especificaba el backend sobre ASP.NET Core 8/C#, y (2) el código realmente implementado en las Entregas 1B, Tercera Entrega y Unidad IV, sobre **Java 21 + Spring Boot 3.2 + Spring Security 6 + Spring Data JPA + PostgreSQL 16 + Redis 7**. Ningún requisito de este documento fue inventado sin sustento: cada uno proviene del SRS original, del código fuente verificado, o de la Guía oficial de la Tercera Entrega. El detalle de cada cambio respecto al documento original está en docs/requisitos/cambios/CAMBIOS-SRS.md y, fila por requisito, en docs/requisitos/CHANGELOG-REQ.md.

1. Introducción

1.1. Propósito

Este documento especifica los requisitos funcionales y no funcionales del sistema BIOPET en su estado **v1.0.0 (Entrega Final)**. Su propósito es servir como fuente única de verdad para la matriz de trazabilidad, las pruebas automatizadas y la evidencia empírica exigidas por el bloque A.3 de la Guía, incorporando además los módulos de la Unidad IV (citás, consultas, vacunas, gestión administrativa de usuarios y consulta externa de especies) que no existían en la revisión v0.9.0-rc de la Tercera Entrega.

1.2. Alcance

BIOPET es un sistema web para centralizar la información clínica y administrativa de clínicas veterinarias de pequeña y mediana escala. El alcance completo del producto (definido en la Entrega 1A) contempla gestión de dueños y mascotas, historial clínico, citas, telemetría IoT, recomendación clínica asistida, facturación digital y reportes.

Alcance implementado y verificado en v1.0.0: autenticación completa (registro, login, refresh, logout con revocación), control de acceso por rol (RBAC), CRUD completo de la entidad Mascota con verificación de propiedad, resumen agregado de mascotas por especie (vía procedimiento almacenado), gestión administrativa de usuarios, gestión de vacunas, gestión de citas a nivel de API y consulta externa de información de especies con caché.

Ampliado en v1.0.0 (Unidad IV): gestión administrativa de usuarios (REQ-F-023), CRUD de vacunas (REQ-F-024) y consulta externa de información de especies con caché (REQ-F-025), con interfaz de usuario real para vacunas (`frontend/src/app/features/vacunas.component.ts`) y solo a nivel de API para usuarios y especies externas (sin pantalla propia todavía). Adicionalmente, el backend incorporó CRUD completo de citas (`CitaController`) y de consultas médicas (`ConsultaController`), que cubren respectivamente la obligación A de REQ-F-015 y la de REQ-F-013 (detalle de alcance real, sin sobre-declarar, en la sección 3.1).

Alcance pendiente, heredado de la Entrega 1A: vista consolidada de historial clínico cronológico (obligación B de REQ-F-013) y calendario interactivo de citas (obligación B de REQ-F-015) — ambos con trabajo parcial de backend desde v1.0.0 —, más prescripción de medicamentos, telemetría IoT, recomendaciones asistidas, facturación digital, reportes exportables, notificaciones por correo y recuperación de contraseña. El modelo de datos conceptual de estos módulos ya existe (ver sección 5 y el DER de la Entrega 1A). Se documentan como requisitos con estado **implementado** o **pendiente** (según la taxonomía de la sección 1.3.1) para que la matriz de trazabilidad (bloque A.3.3 de la Guía) los declare correctamente, en vez de omitirlos.

1.3. Definiciones, acrónimos y abreviaturas

Término	Definición
RBAC	Role-Based Access Control: control de acceso basado en el rol del usuario autenticado.

Término	Definición
JWT	JSON Web Token (RFC 7519): token firmado que representa afirmaciones sobre un sujeto autenticado.
ORM	Object-Relational Mapping: mapeo objeto-relacional (Spring Data JPA / Hibernate en este proyecto).
SP	Stored Procedure / procedimiento almacenado en el motor de base de datos.
TTL	Time To Live: tiempo de vida de una entrada de caché.
MoSCoW deberá	Técnica de priorización: Must, Should, Could, Won't. Verbo modal normativo (equivalente al <i>shall</i> de ISO/IEC/IEEE 29148) que indica obligación contractual del requisito. Todos los enunciados de la sección 3 lo usan.
Aislamiento de datos	Regla por la cual un usuario con rol ROLE_DUEÑO solo puede acceder a los recursos asociados a su propia cuenta (ver REQ-NF-015).

1.3.1. Taxonomía de estados de requisito (valores permitidos) Todo requisito de este documento declara su campo **Estado** con **uno y solo uno** de los tres valores siguientes. No existe ningún cuarto valor: los matices (cobertura parcial de pruebas, defectos conocidos, evidencia faltante) se registran en el campo **Observaciones** del propio requisito, no inventando un estado nuevo.

Estado	Significado exacto
verificado	El requisito se da por cumplido. La conducta exigida está realizada en el repositorio y todos sus criterios de aceptación están respaldados por un artefacto reproducible y nombrado: prueba automatizada, captura HTTP o de log, corrida de medición archivada, o inspección de un fragmento de código/configuración citado explícitamente (método <i>Inspection</i> de ISO/IEC/IEEE 29148).
implementado	El requisito NO se da por cumplido. Existe código real en el repositorio que lo realiza, total o parcialmente, pero al menos uno de sus criterios de aceptación falla por un defecto conocido, o bien carece de artefacto que lo respalde. La causa exacta se declara siempre en Observaciones.
pendiente	El requisito NO se da por cumplido. No existe en el repositorio implementación específica de lo que el requisito exige. Se conserva especificado (con criterios de aceptación y método de verificación previstos) para que la matriz de trazabilidad lo declare sin ocultarlo.

Lectura obligada de implementado. **implementado nunca** significa “entregado” ni “cumplido parcialmente de forma aceptable”: significa que el requisito, tomado como un todo, **no está satisfecho hoy**. Se distingue de **pendiente** únicamente en que existe código real que lo aborda, dato que se conserva para no ocultar el trabajo hecho ni exagerar el que falta. Un requisito **implementado** cuenta como **no cumplido** en todos los recuentos de este documento (secciones 3.4.1 y 3.4.2).

1.3.2. Plantilla única de requisito Todos los requisitos de las secciones 3.1 y 3.2 usan exactamente esta plantilla, sin excepción:

- **Tipo** — Funcional, o la categoría no funcional (Rendimiento, Seguridad, Usabilidad, Mantenibilidad, Compatibilidad, Disponibilidad, Operación).
- **Prioridad** — MoSCoW: Must, Should o Could.
- **Enunciado** — patrón [condición] [sujeto] deberá [acción] [objeto] [restricción] de ISO/IEC/IEEE 29148.
- **Rationale** — origen y justificación del requisito.
- **Criterios de aceptación** — condiciones observables, en forma *Dado / cuando / entonces*, que permiten responder CUMPLE / NO CUMPLE sin juicio subjetivo.
- **Método de verificación** — cómo se comprueban esos criterios (prueba automatizada, inspección, demostración, análisis o medición), con el artefacto concreto cuando existe.
- **Trazabilidad** — rutas reales del repositorio: historia de usuario, caso de uso, clases, endpoints, pruebas, scripts, configuración, decisiones de arquitectura o evidencias, según corresponda al tipo de requisito.
- **Estado** — uno de los tres valores de la sección 1.3.1.
- **Observaciones** — solo cuando hay un matiz, defecto o limitación que declarar.

Nota sobre requisitos con más de una obligación. Tres requisitos heredados agrupan dos obligaciones distintas con distinto grado de avance: **REQ-F-013** (registrar consultas médicas / consultar el historial consolidado), **REQ-F-015** (gestionar citas por API / presentarlas en un calendario interactivo) y **REQ-NF-012** (TTL de caché configurable / medir la tasa de aciertos). En lugar de crear identificadores nuevos, **cada uno conserva su identificador único** y separa sus obligaciones dentro del propio bloque, bajo los epígrafes **Obligación A** y **Obligación B**, con criterios de aceptación numerados por obligación (A1, A2, ..., B1, B2, ...). El campo Estado es **uno solo** y describe el requisito completo: basta con que una de las dos obligaciones no esté cumplida para que el requisito no pueda declararse **verificado**.

Se descartó deliberadamente la alternativa de crear subrequisitos con sufijo de letra por dos motivos: rompería la correspondencia exacta de identificadores entre este SRS y `docs/trazabilidad/matriz.csv` —que el guion de CI `scripts/validate-traceability.sh` comprueba en ambos sentidos— y fragmentaría la trazabilidad histórica hacia las historias de usuario, los casos de uso y los identificadores **RF-NN** de la Entrega 1A, que están fijados sobre el identificador base. **Ningún requisito se renumera y no existe ningún identificador con sufijo de letra en este documento.**

1.4. Referencias

- ISO/IEC/IEEE 29148:2018 — Requirements Engineering.
- INCOSE Guide to Writing Requirements v4.
- RFC 7519 (JWT), RFC 7807 (ProblemDetails).
- Guía de la Tercera Entrega — PFC Aplicaciones Web 2026-2027, UTEQ.
- `PFC_Entrega1A_BMT.pdf` — SRS y diseño original (ASP.NET Core).
- `docs/requisitos/cambios/CAMBIOS-SRS.md` — registro de cambios respecto al documento original.
- `docs/requisitos/CHANGELOG-REQ.md` — registro formal de cambios fila por requisito.
- `docs/trazabilidad/matriz.csv` — matriz de trazabilidad (bloque A.3.3).
- `docs/basedatos/CATALOGO-SP.md` — catálogo de procedimientos almacenados.
- `docs/adr/` — decisiones de arquitectura (ADR-002 a ADR-007).

1.5. Resumen del documento

La sección 2 describe el producto de forma global (perspectiva, funciones, usuarios, restricciones, interfaces externas, matriz de permisos y estados del dominio). La sección 3 contiene el detalle de cada requisito funcional y no funcional con la plantilla única de la sección 1.3.2, e incluye en 3.4 la tabla de control de completitud de los campos obligatorios. La sección 4 resume la trazabilidad end-to-end. Las secciones 5 y 6 remiten a los artefactos de modelo de datos e interfaz que ya existen en el repositorio. La sección 7 documenta, sin inventar contenido, lo que aún falta por completar.

1.6. Control documental e historial de revisiones

Este documento tiene **un solo índice**: la tabla de contenidos generada automáticamente al inicio del PDF. La lista manual de secciones que duplicaba ese índice en revisiones anteriores fue eliminada en la revisión 2026-09-11.

Cada fila del historial está respaldada por un commit real del repositorio (`git log -- docs/requisitos/SRS.md`). No se declara ninguna revisión anterior que no pueda sustentarse con historial Git.

Revisión	Fecha	Commit	Resumen del cambio
v0.7.0	2026-06-14	—	Primera consolidación de requisitos tras la migración de pila (registro histórico en CHANGELOG-REQ.md).
v0.9.0-rc	2026-07-30	2115a35	Consolidación de historias de usuario y casos de uso en el SRS; creación de CHANGELOG-REQ.md .
v0.9.0-rc	2026-07-31	73c41b4	Cierre de observaciones oficiales de las Entregas 1A y 1B (OBS-02 a OBS-05).
v0.9.0-rc	2026-08-10	9804ed3	Reconciliación de SRS y matriz de trazabilidad con la Unidad IV (corrección del CI de trazabilidad).
v0.9.0-rc	2026-08-17	5b735e6	Enlace de evidencia para varios REQ-NF pendientes.
v0.9.0-rc	2026-08-18	91b72b5	Cierre de REQ-NF-005 con la re-corrida Lighthouse (móvil y escritorio) y cierre de PRISMA en la sección 7.
v1.0.0	2026-09-03	4f53376	Paso a v1.0.0: sección 3.3 (INVEST sobre las 24 historias), alineación de REQ-F-013 / REQ-F-015 / REQ-F-025 con el código de Unidad IV.
v1.0.0	2026-09-04	a9f0562	Cierre de trazabilidad y evidencia de requisitos.
v1.0.0	2026-09-07	70c3596, e9b1dc5	Sincronización SRS / matriz / código antes de la firma; portada institucional y URL del repositorio en la copia para firma.

Revisión	Fecha	Commit	Resumen del cambio
v1.0.0	2026-09-11	8130ee0 (base)	Esta revisión. Corrección integral tras la retroalimentación del docente-director: plantilla única con criterios de aceptación verificables en todos los requisitos, bloque individual para REQ-F-013 a REQ-F-020, taxonomía de tres estados, descomposición de requisitos compuestos, interfaces externas, matriz de permisos, estados del dominio, corrección de la sección 2.6 y separación explícita entre versión documental, tag histórico y commit de revisión.

2. Descripción global

2.1. Perspectiva del producto

BIOPET es un sistema nuevo, no una extensión de un producto previo existente en la clínica. Reemplaza procesos manuales (hojas de cálculo, mensajería) por una plataforma web centralizada, según el problema documentado en la Entrega 1A (entrevistas a tres veterinarios y dos auxiliares, encuestas a quince dueños de mascotas).

2.2. Cambio de plataforma tecnológica respecto a la Entrega 1A

El SRS original (Entrega 1A) especificaba ASP.NET Core 8/C# como backend (ADR-001 original) y Bootstrap 5 + HTML/CSS/JS como frontend. El equipo migró la implementación real a **Java 21 + Spring Boot 3.2** en el backend y **Angular 17+** en el frontend, decisión reflejada en ADR-002-pila-tecnologica.md del repositorio actual. Los requisitos funcionales de negocio (qué hace el sistema) no cambiaron; los requisitos no funcionales técnicos (cómo se implementa: framework, ORM, mecanismo JWT) se actualizaron para reflejar la pila real, evitando que el SRS describa un sistema que no es el que existe en el repositorio.

2.3. Funciones del producto (resumen)

- Gestión de usuarios y autenticación (roles: `ROLE_ADMIN`, `ROLE_VETERINARIO`, `ROLE_AUXILIAR`, `ROLE_DUEÑO`), incluida la administración de cuentas por un `ROLE_ADMIN` (Unidad IV).
- Gestión de mascotas (CRUD + resumen agregado por especie).
- Gestión de vacunas (CRUD, con interfaz de usuario) y de citas y consultas médicas (CRUD a nivel de API; sin calendario interactivo ni vista de historial clínico consolidado todavía — ver 3.1).
- Consulta de información externa de especies (taxonomía, hábitat, dieta), con caché Redis.
- (*Pendiente*) Vista consolidada de historial clínico cronológico, prescripción de medicamentos, telemetría IoT, recomendaciones asistidas, facturación, reportes exportables, notificaciones por correo y recuperación de contraseña.

2.4. Características de los usuarios

Rol	Descripción	Nivel técnico esperado
ROLE_ADMIN	Supervisa roles, operatividad general del sistema y tiene visibilidad global de datos. Único rol con acceso al CRUD administrativo de usuarios.	Medio-alto (personal administrativo de la clínica).
ROLE_VETERINARIO	Registra consultas médicas y vacunas; gestiona mascotas; solo puede modificar las citas en las que él es el veterinario asignado.	Medio (personal clínico).
ROLE_AUXILIAR	Apoya operaciones administrativas: agenda citas, gestiona mascotas, vacunas y consultas.	Medio.
ROLE_DUEÑO	Consulta únicamente la información asociada a sus propias mascotas. No crea, actualiza ni elimina recursos clínicos.	Básico (público general, sin capacitación previa).

2.5. Restricciones

- El proyecto debe completarse dentro de un semestre académico (PPA 2026-2027), lo que limita el alcance implementado al núcleo Must Have.
- La base de datos debe ser PostgreSQL 16 (decisión ya tomada, ver `ADR-004-postgresql.md`).
- La autenticación debe usar JWT en cookies `HttpOnly + Secure + SameSite=Strict` (bloque A.1 de la Guía), no `localStorage` ni `sessionStorage`.
- Toda operación de base de datos que no sea CRUD elemental debe implementarse como función/procedimiento almacenado (bloque A.2 de la Guía), no como JPQL/HQL con joins o agregaciones.
- **Restricción operacional (compromiso académico, no requisito verificable).** El equipo se compromete a mantener el sistema desplegable y operativo durante las semanas de evaluación establecidas por la asignatura. Esta condición se registra aquí como restricción operacional y **no** como requisito con estado de cumplimiento, porque el proyecto no dispone de monitoreo continuo que permita medir disponibilidad histórica. La parte que sí es técnicamente verificable (existencia y respuesta del endpoint de verificación de estado) se especifica en REQ-NF-007.

2.6. Supuestos y dependencias

- Se asume disponibilidad de un motor PostgreSQL 16 y Redis 7 accesibles desde el backend (vía Docker Compose en desarrollo; servicios gestionados en el despliegue de Render, ver `docs/despliegue/DEPLOYMENT.md`).
- Se asume que el operador del despliegue ejecuta el procedimiento de respaldo descrito en `docs/despliegue/BACKUP.md` (ver REQ-NF-017).

Servicios externos realmente integrados en v1.0.0 (corrección de esta revisión). Una versión anterior de este documento afirmaba que “ninguno está implementado todavía”, lo cual contradecía a REQ-F-025. La situación real, verificada contra el código, es la siguiente:

Servicio externo	Estado real	Dónde está en el código	Requisito
API Ninjas — Animals API (https://api.api-ninjas.com/v1/animals)	Integrado y en uso. Consulta de taxonomía, hábitat y dieta por nombre de especie, con caché <i>cache-aside</i> en Redis. Autenticación por cabecera X-API-Key (<code>APP_EXTERNAL_API_KEY</code>).	Backend/src/main/java/com/biopet/integration/ExternalApiClient.java, ExternalApiService.java, RestTemplateConfig.java	REQ-F-025
Servicio de correo electrónico	No integrado. Sin cliente, sin configuración y sin proveedor elegido.	—	REQ-F-022 (pendiente)
Servicio de IA para recomendaciones clínicas	No integrado. Sin cliente y sin decisión de arquitectura formal sobre el proveedor.	—	REQ-F-017 (pendiente)
Plataforma de telemetría IoT	No integrado. Sin API de ingesta.	—	REQ-F-016 (pendiente)

Las tres integraciones no implementadas se mantienen documentadas como requisitos con estado **pendiente**; **no** se declaran como implementadas ni se les asigna configuración, endpoint o evidencia que no exista.

2.7. Interfaces externas

Esta sección describe las interfaces del sistema con su entorno, conforme a ISO/IEC/IEEE 29148:2018. Solo se describen interfaces que existen realmente en el repositorio.

2.7.1. Interfaces de usuario La interfaz es una aplicación web de página única (Angular 17+) servida como estático. Las pantallas realmente implementadas están en `frontend/src/app/features/: login.component.ts, mascotas.component.ts` y `vacunas.component.ts`. El detalle está en la sección 6.

2.7.2. Interfaces de software

Interfaz	Protocolo / mecanismo	Dirección	Artefacto en el repositorio
API REST de BIOPET	HTTP/JSON sobre TLS; errores en formato <code>ProblemDetail</code> (RFC 7807); autenticación JWT en cookies <code>HttpOnly + Secure + SameSite=Strict</code> o cabecera <code>Authorization: Bearer</code>	Entrante (frontend y clientes)	<code>com.biopet.controller.*</code> , <code>com.biopet.exception.ProblemDetailFactory</code>
Contrato OpenAPI	OpenAPI 3 (springdoc), expuesto en <code>/api/openapi</code> y navegable en <code>/api/docs</code>	Saliente (documentación)	<code>com.biopet.config.OpenApiConfig</code> , <code>application.yml</code> (springdoc.*)

Interfaz	Protocolo / mecanismo	Dirección	Artefacto en el repositorio
PostgreSQL 16	JDBC (org. postgresql.Driver); esquema versionado con Flyway; rutinas almacenadas invocadas con @Procedure	Saliente	application.yml (spring.datasource.*, spring.flyway.*), Backend/src/main/ resources/db/ migration/, db/procs/ application.yml
Redis 7	Protocolo RESP vía Spring Data Redis (StringRedisTemplate) y caché de Spring (spring.cache.type: redis)	Saliente	(spring.data.redis.*, spring.cache. redis.*), Token BlacklistService, ExternalApiService ExternalApiClient, RestTemplateConfig
API Ninjas — Animals API	HTTPS/JSON con Rest Template; clave en cabecera X-API-Key	Saliente	
Endpoints de operación	HTTP/JSON; Spring Boot Actuator, exposición limitada a health, info y metrics	Entrante (operador / plataforma)	application.yml (management. endpoints.web. exposure.include)

2.7.3. Interfaces de hardware BIOPET no interactúa directamente con hardware. La API de ingesta de telemetría de dispositivos IoT prevista en la Entrega 1A corresponde a REQ-F-016 y está en estado **pendiente**: no existe endpoint, cliente ni protocolo definido en el repositorio.

2.7.4. Interfaces de comunicación

Aspecto	Valor real configurado	Artefacto
Transporte	HTTPS/TLS 1.3 (conector dual HTTP/HTTPS en desarrollo)	TomcatDualConnectorConfig, application-tls.yml, docker- compose.tls.yml
CORS	Orígenes permitidos por variable CORS_ALLOWED_ORIGINS (por defecto http://localhost:4200)	SecurityConfig.cors ConfigurationSource()
Cabeceras de seguridad	HSTS (max-age=31536000, includeSubDomains, preload), CSP default-src 'self'; frame-ancestors 'none'; object-src 'none', X-Content- Type-Options, X-Frame- Options: DENY, Referrer- Policy: no-referrer	SecurityConfig, prueba SecurityHeadersTest
Sesión	Sin estado de sesión en servidor (SessionCreationPolicy. STATELESS)	SecurityConfig

2.8. Matriz de permisos por rol y operación

Matriz construida **exclusivamente** a partir del código de este repositorio: las anotaciones @PreAuthorize de com.biopet.controller.* (control por rol) y las reglas de datos implementadas en com.biopet.service.* (control por propiedad del recurso). No se toma ninguna regla de otro proyecto.

Leyenda de las columnas de rol:

- **Sí** — el rol puede ejecutar la operación sobre cualquier registro.
- **Solo propias** — el rol puede ejecutar la operación, pero el resultado debe restringirse a los recursos asociados a su propia cuenta.
- **No** — la solicitud se rechaza con 403 Forbidden y cuerpo ProblemDetail.
- **Público** — no requiere autenticación (SecurityConfig, lista permitAll).

2.8.1. Autenticación y perfil

Operación (endpoint)	ADMIN	VETERINARIO	AUXILIAR	DUENO
POST /api/auth/registro	Público	Público	Público	Público
POST /api/auth/login	Público	Público	Público	Público
POST /api/auth/refresh	Público	Público	Público	Público
POST /api/auth/logout	Público	Público	Público	Público
GET /api/usuarios/me	Sí	Sí	Sí	Sí

2.8.2. Usuarios (administración)

Operación (endpoint)	ADMIN	VETERINARIO	AUXILIAR	DUENO
GET /api/usuarios	Sí	No	No	No
GET /api/usuarios/{id}	Sí	No	No	No
POST /api/usuarios	Sí	No	No	No
PUT /api/usuarios/{id}	Sí	No	No	No
DELETE /api/usuarios/{id}	Sí	No	No	No

Regla adicional de datos: un ROLE_ADMIN **no** puede modificar el rol de su propia cuenta (UsuarioService, prueba adminNoPuedeEscalarSuPropioRolDevuelve403).

2.8.3. Mascotas

Operación (endpoint)	ADMIN	VETERINARIO	AUXILIAR	DUENO
GET /api/mascotas	Sí	Sí	Sí	Solo propias
GET /api/mascotas/{id}	Sí	Sí	Sí	Solo propias
POST /api/mascotas	Sí	Sí	Sí	No
PUT /api/mascotas/{id}	Sí	Sí	Sí	No
DELETE /api/mascotas/{id}	Sí	Sí	Sí	No

El resumen agregado GET /api/mascotas/resumen-especies está autorizado para los cuatro roles y se rige por una regla de datos propia: el rol ROLE_ADMIN obtiene el resumen global, o el de un dueño concreto si envía el parámetro dueñoId; para cualquier otro rol el parámetro se ignora y el resumen se calcula siempre sobre el propio usuario autenticado (MascotaService.resumenPorEspecie). La consulta de especies externas y los endpoints de operación se detallan en 2.8.7.

2.8.4. Citas

Operación (endpoint)	ADMIN	VETERINARIO	AUXILIAR	DUENO
GET /api/citas	Sí	Sí	Sí	Solo propias
GET /api/citas/{id}	Sí	Sí	Sí	Solo propias

Operación (endpoint)	ADMIN	VETERINARIO	AUXILIAR	DUENO
POST /api/citas	Sí	No	Sí	No
PUT /api/citas/{id}	Sí	Solo las asignadas a él	Sí	No
DELETE /api/citas/{id}	Sí	No	No	No

2.8.5. Consultas médicas

Operación (endpoint)	ADMIN	VETERINARIO	AUXILIAR	DUENO
GET /api/consultas	Sí	Sí	Sí	Solo propias
GET /api/consultas/{id}	Sí	Sí	Sí	Solo propias
POST /api/consultas	Sí	Sí	Sí	No
PUT /api/consultas/{id}	Sí	Sí	Sí	No
DELETE /api/consultas/{id}	Sí	Sí	Sí	No

Defecto abierto declarado. La celda “Solo propias” de GET /api/consultas describe la regla especificada, no la implementada: `ConsultaService.listar()` devuelve `consultaRepository.findAllByActivoTrue(pageable)` también para `ROLE_DUENO`, sin filtrar por propietario. Ver REQ-F-006, REQ-F-013, REQ-NF-015 y la sección 7. No se corrige código en esta revisión documental.

2.8.6. Vacunas

Operación (endpoint)	ADMIN	VETERINARIO	AUXILIAR	DUENO
GET /api/vacunas	Sí	Sí	Sí	Solo propias
GET /api/vacunas/mascota/{mascotaId}	Sí	Sí	Sí	Solo propias
GET /api/vacunas/{id}	Sí	Sí	Sí	Solo propias
POST /api/vacunas	Sí	Sí	Sí	No
PUT /api/vacunas/{id}	Sí	Sí	Sí	No
DELETE /api/vacunas/{id}	Sí	Sí	Sí	No

2.8.7. Información externa de especies y operación

Operación (endpoint)	ADMIN	VETERINARIO	AUXILIAR	DUENO
GET /api/externa/especies	Sí	Sí	Sí	Sí
GET /actuator/health	Público	Público	Público	Público
GET /api/docs	Público	Público	Público	Público
GET /api/openapi	Público	Público	Público	Público

2.9. Estados principales del dominio y transiciones

Estados tomados directamente de las entidades y migraciones del repositorio. No se documentan estados que el código no tenga.

2.9.1. Cita (`com.biopet.entity.EstadoCita`) Valores permitidos: PROGRAMADA, CANCELADA, COMPLETADA (restricción CHECK replicada en `sp_actualizar_estado_citas_masivas`).

Desde	Hacia	Disparador real
<i>(inexistente)</i>	PROGRAMADA	POST /api/citas. CitaService. crear fuerza EstadoCita. PROGRAMADA e ignora cualquier estado enviado por el cliente (prueba crearCitaForzando EstadoDistintoIgnoraElEstado Enviado).
PROGRAMADA	COMPLETADA	PUT /api/citas/{id} con estado=COMPLETADA, o CALL sp_ actualizar_estado_citas_ masivas(...) para cierre masivo por veterinario y fecha límite.
PROGRAMADA	CANCELADA	PUT /api/citas/{id} con estado=CANCELADA, o el mismo procedimiento masivo.
cualquiera	<i>(baja lógica)</i>	DELETE /api/citas/{id} marca activo = false; el valor de estado no cambia.

El procedimiento masivo valida ambos estados contra la lista permitida y lanza RAISE EXCEPTION si alguno es inválido (prueba actualizarEstadoCitasMasivas_estadoInvalido_lanzaExcepcion).

2.9.2. Ciclo de vida común de los recursos de negocio Usuario, Mascota, Cita, Consulta y Vacuna comparten un único atributo de ciclo de vida: **activo** (booleano). No existe borrado físico en ninguna de las cinco entidades.

Desde	Hacia	Disparador real
<i>(inexistente)</i>	activo = true	Operación POST del recurso correspondiente.
activo = true	activo = false	Operación DELETE del recurso (baja lógica).
activo = false	—	No hay operación de reactivación implementada en la API.

Las consultas de lectura filtran siempre por **activoTrue**, de modo que un registro dado de baja deja de aparecer en listados y búsquedas por identificador (responde 404 con ProblemDetail). El trigger **set_actualizado_en** mantiene **actualizado_en** en cada UPDATE.

2.9.3. Sesión y token JWT

Desde	Hacia	Disparador real
<i>(sin sesión)</i>	access token vigente + refresh token vigente	POST /api/auth/login.
access token vigente	access token renovado	POST /api/auth/refresh con un refresh token válido y no revocado.

Desde	Hacia	Disparador real
access token vigente	revocado	POST /api/auth/logout: Token BlacklistService.revoke escribe <code>jwt:blacklist:<jti></code> en Redis con TTL igual al tiempo restante del token.
access token vigente	expirado	Transcurre <code>JWT_EXPIRATION_MS</code> (1 h por defecto).
revocado o expirado	rechazo	Cualquier solicitud a recurso protegido responde 401 con <code>ProblemDetail</code> .

3. Requisitos específicos

Cada requisito sigue el patrón [condición] [sujeto] deberá [acción] [objeto] [restricción] de ISO/IEC/IEEE 29148, con identificador persistente, rationale, prioridad MoSCoW, criterios de aceptación verificables y método de verificación, cumpliendo las características INCOSE C1–C9 (Necessary, Appropriate, Unambiguous, Complete, Singular, Feasible, Verifiable, Correct, Conforming). La plantilla de campos es la de la sección 1.3.2 y los valores permitidos del campo Estado son exclusivamente los tres de la sección 1.3.1.

3.1. Requisitos funcionales (REQ-F)

Nota de consistencia de numeración. La numeración de esta sección es la que ya está fijada en `docs/requisitos/historias/HistoriasUsuario.md` (HU-001 a HU-024) y `docs/requisitos/casos-de-uso/CasosDeUso.md` (CU-01 a CU-24). Esta versión del SRS adopta esa numeración como única fuente de verdad, para que los tres documentos no se contradigan entre sí. **No existe ningún identificador con sufijo de letra:** los tres requisitos que agrupan dos obligaciones (REQ-F-013, REQ-F-015 y REQ-NF-012) las separan dentro de su propio bloque como *Obligación A* y *Obligación B*, conservando un identificador único cada uno (ver sección 1.3.2).

REQ-F-001 — Registro de usuario dueño de mascota

- **Tipo:** Funcional
- **Prioridad:** Must
- **Enunciado:** El sistema deberá permitir que cualquier visitante no autenticado se registre proporcionando nombre, correo electrónico y contraseña, asignándole automáticamente el rol `ROLE_DUEÑO` e ignorando cualquier rol enviado por el cliente.
- **Rationale:** heredado de RF-01 de la Entrega 1A; el registro público solo aplica al rol dueño, los demás roles se crean administrativamente (REQ-F-023).
- **Criterios de aceptación:**
 1. Dado un visitante no autenticado, cuando envía POST /api/auth/registro con nombre, correo no registrado y contraseña válidos, entonces la respuesta es 201 Created y el cuerpo contiene `id`, `nombre`, `email` y `rol`, sin ningún campo de contraseña ni hash.
 2. Dado un cuerpo de registro que incluye `"rol": "ROLE_ADMIN"` (o cualquier otro rol), cuando se procesa el registro, entonces el usuario creado queda con `rol = ROLE_DUEÑO` y `activo = true`.
 3. Dado un correo con mayúsculas, cuando se registra, entonces el correo se persiste en minúsculas.
 4. Dado un cuerpo con nombre vacío, correo con formato inválido o contraseña de menos de 8 caracteres, cuando se envía la solicitud, entonces la respuesta es 422 Unprocessable Entity con `ProblemDetail` de tipo `urn:biopet:error:validation` y el objeto `errors` enumera los campos inválidos.
- **Método de verificación:** demostración con captura HTTP real — `docs/mediciones/sec/raw/A01-access-control.txt` (líneas 9–11) registra dos registros reales con `status_registro=201` y el rol

forzado a `ROLE_DUEÑO` pese al valor enviado, generados por `scripts/security-evidence.sh`. Para el criterio 4, prueba automatizada `AuthControllerTest.registroConCamposInvalidos`. Para los criterios 2 y 3, inspección de `AuthService.registrar(rol(Rol.ROLE_DUEÑO) y email(request.email().toLowerCase()))`.

- **Trazabilidad:** HU-001 → CU-01 → `AuthController.registro` → `AuthService.registrar` → POST `/api/auth/registro`; DTO `RegistroRequest/UsuarioResponse`; evidencia `docs/mediciones/sec/A01-access-control.md` y `docs/mediciones/sec/raw/A01-access-control.txt`.
- **Estado:** verificado
- **Observaciones:** no existe prueba JUnit dedicada al camino feliz del registro; la evidencia del criterio 1 es la captura HTTP real citada, no una prueba automatizada. Una versión anterior de este documento citaba un test “`AuthControllerTest (registro exitoso)`” que no existe en `Backend/src/test/java/com/biopet/AuthControllerTest.java`; la referencia se corrige aquí.

REQ-F-002 — Rechazo de registro con correo duplicado

- **Tipo:** Funcional
- **Prioridad:** Must
- **Enunciado:** Al recibir una solicitud de registro con un correo electrónico ya existente, el sistema deberá rechazarla con código 409 `Conflict` y un cuerpo `ProblemDetail`, sin crear ningún registro nuevo.
- **Rationale:** funcionalidad presente en el código (`EmailDuplicadoException`) no documentada como requisito independiente en el SRS original de la Entrega 1A; se agrega para cerrar el hueco (bloque A.3 de la Guía).
- **Criterios de aceptación:**
 1. Dado un correo previamente registrado, cuando se intenta registrar de nuevo ese mismo correo, entonces la respuesta es 409 `Conflict`.
 2. El cuerpo de esa respuesta tiene `Content-Type: application/problem+json` y los campos `type = urn:biopet:error:conflict`, `title = "Conflicto de datos"`, `status = 409`, `detail` no vacío e `instance = /api/auth/registro`.
 3. La cantidad de usuarios con ese correo permanece igual: no se crea un segundo usuario.
- **Método de verificación:** prueba automatizada `AuthControllerTest.registroEmailDuplicado` (criterios 1 y 2, con aserciones sobre los cinco campos del `ProblemDetail`). Criterio 3: inspección de `AuthService.registrar`, donde la comprobación `usuarioRepository.existsByEmail(...)` lanza `EmailDuplicadoException` antes de cualquier invocación a `usuarioRepository.save(...)`.
- **Trazabilidad:** HU-001 → CU-01 → `AuthService.registrar` → `EmailDuplicadoException` → `GlobalExceptionHandler.emailDuplicado` → `ProblemType.CONFLICT` → POST `/api/auth/registro`.
- **Estado:** verificado

REQ-F-003 — Autenticación mediante usuario y contraseña

- **Tipo:** Funcional
- **Prioridad:** Must
- **Enunciado:** El sistema deberá permitir que un usuario registrado y activo inicie sesión mediante correo electrónico y contraseña, emitiendo un access token con vigencia de 1 hora y un refresh token, ambos con los siete claims JWT estándar del RFC 7519.
- **Rationale:** heredado de RF-16/RF-WEB-01 de la Entrega 1A.
- **Criterios de aceptación:**
 1. Dadas credenciales válidas, cuando se envía POST `/api/auth/login`, entonces la respuesta es 200 OK y establece las cookies `access_token` y `refresh_token` con atributos `HttpOnly`, `Secure` y `SameSite=Strict`.
 2. Cada token emitido contiene exactamente los siete claims `iss`, `sub`, `aud`, `exp`, `nbf`, `iat`, `jti`, con `nbf == iat` y `jti` distinto entre dos tokens emitidos consecutivamente.
 3. Dadas credenciales inválidas, cuando se envía la solicitud, entonces la respuesta es 401 `Unauthorized` con `ProblemDetail` de tipo `urn:biopet:error:unauthorized` y no se emite ningún token.
 4. Un token con `iss` o `aud` distintos de los configurados es rechazado.

- **Método de verificación:** pruebas automatizadas `AuthControllerTest.loginExitoso` y `AuthControllerTest.loginClaveIncorrecta` (criterios 1 y 3); `JwtServiceTest.accessTokenContieneLosSieteClaims`, `refreshTokenContieneLosSieteClaims`, `nbfEsIgualAlat`, `jtiEsUnicoEntreTokens` (criterio 2); `JwtServiceTest.issuerIncorrectoEsRechazado` y `audienceIncorrectaEsRechazada` (criterio 4).
- **Trazabilidad:** HU-002 → CU-02 → `AuthController.login` → `AuthService.login` → `JwtService.buildToken` → `JwtCookieService` → POST `/api/auth/login`; configuración `security.jwt.*` en `Backend/src/main/resources/application.yml`.
- **Estado:** verificado

REQ-F-004 — Renovación de sesión (refresh)

- **Tipo:** Funcional
- **Prioridad:** Must
- **Enunciado:** El sistema deberá permitir renovar el access token utilizando un refresh token válido y no revocado, sin exigir nuevamente las credenciales del usuario.
- **Rationale:** heredado de RNF-03/RNF-WEB-03 de la Entrega 1A (expiración configurable de JWT), llevado a requisito funcional explícito.
- **Criterios de aceptación:**
 1. Dada una cookie `refresh_token` válida y no revocada, cuando se envía POST `/api/auth/refresh`, entonces la respuesta es 200 OK y se emite una nueva cookie `access_token`.
 2. Dada la ausencia de cookie de refresh, entonces la respuesta es 401 con `ProblemDetail`.
 3. Dada una cookie de refresh inválida o manipulada, entonces la respuesta es 401 con `ProblemDetail`.
 4. Un access token presentado como refresh token no sirve para renovar: la respuesta es 401.
 5. Un refresh token previamente revocado por logout devuelve 401.
- **Método de verificación:** pruebas automatizadas `AuthControllerTest.refreshCookieValidaEmiteNuevaAccessCookie`, `refreshSinCookieDevuelve401ProblemDetail`, `refreshCookieInvalidaDevuelve401ProblemDetail`, `accessTokenNoSirveComoRefreshCookie` y `refreshCookieRevocadaDevuelve401` (un criterio por prueba, en ese orden).
- **Trazabilidad:** HU-003 → CU-03 → `AuthController.refresh` → `AuthService.refresh` → `JwtService.isAccessToken` → `TokenBlacklistService.isRevoked` → POST `/api/auth/refresh`.
- **Estado:** verificado

REQ-F-005 — Cierre de sesión con revocación de token

- **Tipo:** Funcional
- **Prioridad:** Must
- **Enunciado:** El sistema deberá permitir cerrar sesión registrando el `jti` de cada token vigente en una lista negra en Redis, con TTL igual a su tiempo restante de expiración, de modo que una solicitud posterior con ese token a un recurso protegido responda 401.
- **Rationale:** heredado de RF-17/RF-WEB-04 de la Entrega 1A; mecanismo de revocación definido en `docs/adr/ADR-003-jwt-redis.md`.
- **Criterios de aceptación:**
 1. Dado un usuario autenticado con ambas cookies, cuando envía POST `/api/auth/logout`, entonces la respuesta es 204 No Content, se escriben las claves `jwt:blacklist:<jti>` correspondientes y ambas cookies se eliminan del cliente.
 2. Dado un token cuyo tiempo restante de expiración es positivo, cuando se revoca, entonces la clave se escribe con TTL igual a ese tiempo restante; si el tiempo restante es cero o negativo, no se escribe nada en Redis.
 3. Dado un refresh token revocado, cuando se intenta usar en POST `/api/auth/refresh`, entonces la respuesta es 401.
 4. El logout es idempotente: sin cookies, o con cookies inválidas, la respuesta sigue siendo 204.
- **Método de verificación:** pruebas automatizadas `AuthControllerTest.logoutConAmbasCookiesRevocaTokensYLasElimina`, `logoutSoloConAccessCookieRevocaAccessYEliminaAmbas`, `logout`

SoloConRefreshCookieRevocaRefreshYEliminaAmbas, logoutSinCookiesEsIdempotente, logoutConCookiesInvalidasSigueSiendo204, refreshCookieRevocadaDevuelve401; y TokenBlacklistServiceTest.ttlPositivoEscribeEnRedisConTtlRestante, ttlNegativoNoEscribeEnRedis, ttlCeroNoEscribeEnRedis (criterio 2).

- **Trazabilidad:** HU-004 → CU-04 → AuthController.logout → TokenBlacklistService.revoke → clave Redis jwt:blacklist:<jti> → POST /api/auth/logout; docs/adr/ADR-003-jwt-redis.md.
- **Estado:** verificado
- **Observaciones:** el comportamiento del sistema cuando Redis no está disponible no forma parte de este requisito y **no está definido hoy**; se especifica por separado en REQ-NF-014, con estado pendiente.

REQ-F-006 — Control de acceso por rol y por propietario del recurso (RBAC + aislamiento)

- **Tipo:** Funcional
- **Prioridad:** Must
- **Enunciado:** El sistema deberá restringir el acceso a cada endpoint protegido según el rol del usuario autenticado, rechazando con 403 Forbidden y cuerpo ProblemDetail toda solicitud de un rol no autorizado para ese recurso, y deberá además restringir el contenido devuelto a un usuario con rol ROLE_DUEÑO a los recursos asociados a su propia cuenta, conforme a la matriz de permisos de la sección 2.8.
- **Rationale:** heredado de RF-13/RF-WEB-02 de la Entrega 1A. En esta revisión el enunciado se **amplía explícitamente** para incluir el aislamiento de datos por propietario: la retroalimentación del docente-director señaló que un control que solo comprueba el rol, sin comprobar la propiedad del dato, no satisface el propósito del requisito. La regla transversal completa se especifica en REQ-NF-015.
- **Criterios de aceptación:**
 1. Dado un usuario autenticado cuyo rol no está autorizado para una operación según la sección 2.8, cuando la invoca, entonces la respuesta es 403 Forbidden con Content-Type: application/problem+json, type = urn:biopet:error:forbidden e instance igual a la ruta solicitada.
 2. Dada una solicitud sin token, entonces la respuesta es 401 con ProblemDetail de tipo urn:biopet:error:unauthorized.
 3. Dado un usuario con rol ROLE_DUEÑO, cuando invoca **cualquier** operación de listado marcada “Solo propias” en la sección 2.8 (GET /api/mascotas, GET /api/citas, GET /api/consultas, GET /api/vacunas), entonces **ningún** elemento devuelto pertenece a otro dueño: el dueño nunca puede listar datos pertenecientes a otro dueño.
 4. Dado un usuario con rol ROLE_DUEÑO, cuando solicita por identificador un recurso de otro dueño, entonces la respuesta es 403.
- **Método de verificación:** pruebas automatizadas MascotaControllerTest.crearMascotaConRolInsuficienteDevuelveProblemDetail, duenoIntentaActualizarMascotaPropiaSigueRecibiendo403PorRol, duenoIntentaEliminarMascotaPropiaSigueRecibiendo403PorRol, duenoSoloVeSusPropiasMascotasEnListado, duenoConsultaMascotaDeOtroDuenioDevuelve403; CitaControllerTest.duenoSoloVeSusPropiasCitasEnListado, duenoNoPuedeConsultarCitaDeMascotaAjena; VacunaControllerTest.duenoSoloVeVacunasDeSusPropiasMascotas, duenoConsultaVacunaDeMascotaAjenaDevuelve403; UsuarioControllerTest.accesoConRolNoAdminDevuelve403, accesoSinAutenticacionDevuelve401; y captura HTTP real end-to-end en docs/mediciones/sec/A01-access-control.md con docs/mediciones/sec/raw/A01-access-control.txt (IDOR entre dos dueños reales sobre TLS).
- **Trazabilidad:** HU-005 → CU-05 → anotaciones @PreAuthorize de MascotaController, CitaController, ConsultaController, VacunaController, UsuarioController, ExternalApiController → SecurityConfig (anyRequest().authenticated(), ProblemAccessDeniedHandler, ProblemAuthenticationEntryPoint) → reglas de propiedad en MascotaService, CitaService, VacunaService, ConsultaService → matriz de permisos de la sección 2.8 → evidencia docs/mediciones/sec/A01-access-control.md.
- **Estado:** implementado
- **Observaciones:** el criterio 3 **no se cumple** hoy para GET /api/consultas. ConsultaService.listar() (Backend/src/main/java/com/biopet/service/ConsultaService.java) evalúa if (usuario.getRol() == Rol.ROLE_DUEÑO) pero ambas ramas devuelven consultaRepository.find

`AllByActivoTrue(pageable)`: no hay filtro por propietario, y `ConsultaRepository` no declara ningún método equivalente a `findAllByMascota_Duenio_IdAndActivoTrue` (que sí existe en `CitaRepository` y `VacunaRepository`). En consecuencia, un usuario con rol `ROLE_DUENO` autenticado puede listar consultas médicas de mascotas ajenas. Este documento **no modifica código**: el defecto se declara aquí, en REQ-F-013, en REQ-NF-015, en la sección 2.8.5 y en la sección 7, y su corrección queda como acción de seguimiento para el propietario del módulo. Los criterios 1, 2 y 4 sí están verificados por las pruebas citadas.

REQ-F-007 — Consulta del perfil propio

- **Tipo:** Funcional
- **Prioridad:** Should
- **Enunciado:** El sistema deberá permitir que un usuario autenticado, sea cual sea su rol, consulte sus propios datos de perfil (identificador, nombre, correo y rol), sin exponerle el listado completo de usuarios.
- **Rationale:** funcionalidad presente en el código (`GET /api/usuarios/me`), base del `authGuard` de Angular; no documentada como requisito independiente en el SRS original.
- **Criterios de aceptación:**
 1. Dado un usuario autenticado con cualquier rol, cuando invoca `GET /api/usuarios/me`, entonces la respuesta es 200 OK con `id`, `nombre`, `email`, `rol` y `activo` del propio usuario.
 2. La respuesta no incluye ningún campo de contraseña ni hash.
 3. Dado un usuario cuyo rol no es `ROLE_ADMIN`, cuando invoca `GET /api/usuarios` (listado completo), entonces la respuesta es 403.
 4. Dada una solicitud sin autenticación a `GET /api/usuarios/me`, entonces la respuesta es 401 con `ProblemDetail`.
- **Método de verificación:** pruebas automatizadas `UsuarioControllerTest.meSigueFuncionando` (criterio 1), `accesoConRolNoAdminDevuelve403` (criterio 3), `AuthControllerTest.accesoSinToken` (criterio 4). Criterio 2: inspección del record `UsuarioResponse`, que declara únicamente `id`, `nombre`, `email`, `rol` y `activo`.
- **Trazabilidad:** HU-006 → CU-06 → `UsuarioController.me` → `AuthService.perfil` → `UsuarioResponse` → `GET /api/usuarios/me`.
- **Estado:** verificado

REQ-F-008 — Creación de mascota

- **Tipo:** Funcional
- **Prioridad:** Must
- **Enunciado:** El sistema deberá permitir que un usuario con rol `ROLE_ADMIN`, `ROLE_VETERINARIO` o `ROLE_AUXILIAR` registre una nueva mascota (nombre, especie, raza y fecha de nacimiento) asociada a un usuario existente, activo y con rol `ROLE_DUENO`.
- **Rationale:** heredado de RF-01/RF-02 de la Entrega 1A (registro y asociación con propietario), acotado a la entidad `Mascota` implementada en esta fase.
- **Criterios de aceptación:**
 1. Dado un usuario `ROLE_ADMIN` y un dueño activo con rol `ROLE_DUENO`, cuando envía `POST /api/mascotas` con datos válidos, entonces la respuesta es 201 Created con la mascota creada y `activo = true`.
 2. Dado un `duenioId` que corresponde a un usuario cuyo rol **no** es `ROLE_DUENO` (`ADMIN`, `VETERINARIO` o `AUXILIAR`), entonces la solicitud es rechazada con 400 Bad Request y `ProblemDetail`.
 3. Dado un `duenioId` inexistente o correspondiente a un usuario inactivo, entonces la respuesta es 404 con `ProblemDetail`.
 4. Dado un usuario con rol `ROLE_DUENO`, cuando intenta crear una mascota, entonces la respuesta es 403.
 5. Dado un cuerpo con campos obligatorios ausentes o inválidos, entonces la respuesta es 422 con `ProblemDetail` de validación.
- **Método de verificación:** pruebas automatizadas `MascotaControllerTest.adminCreaMascotaCon`

DuenioActivoRolDuenoExitosa (criterio 1); adminIntentaCrearMascotaAsignadaAUsuarioAdminEsRechazado, adminIntentaCrearMascotaAsignadaAVeterinarioEsRechazado, adminIntentaCrearMascotaAsignadaAAuxiliarEsRechazado (criterio 2); adminIntentaCrearMascotaConDuenioIdInexistenteDevuelve404, adminIntentaCrearMascotaConUsuarioInactivoDevuelve404 (criterio 3); crearMascotaConRolInsuficienteDevuelveProblemDetail (criterio 4); crearMascotaConCamposInvalidosDevuelve422 (criterio 5).

- **Trazabilidad:** HU-007 → CU-07 → MascotaController.crear → MascotaService.crear → MascotaService.resolverDuenio → POST /api/mascotas; DTO MascotaRequest/MascotaResponse.
- **Estado:** verificado

REQ-F-009 — Listado paginado de mascotas activas, filtrado por propietario según rol

- **Tipo:** Funcional
- **Prioridad:** Must
- **Enunciado:** El sistema deberá permitir consultar el listado paginado de mascotas activas; si el solicitante tiene rol ROLE_DUEÑO, el listado deberá restringirse únicamente a sus propias mascotas, y para los demás roles autorizados deberá incluir todas las mascotas activas.
- **Rationale:** heredado de RF-02 de la Entrega 1A, con la regla de aislamiento de datos entre dueños explícita en MascotaService.listar.
- **Criterios de aceptación:**
 1. Dado un usuario ROLE_DUEÑO con mascotas propias y existiendo mascotas de otro dueño, cuando invoca GET /api/mascotas, entonces la respuesta es 200 OK y contiene exclusivamente sus mascotas: ninguna mascota de otro dueño aparece en la página.
 2. Dado un usuario ROLE_ADMIN o ROLE_AUXILIAR, cuando invoca el mismo endpoint, entonces el listado incluye las mascotas activas de todos los dueños.
 3. Dadas dos cuentas ROLE_DUEÑO distintas que solicitan la misma página con los mismos parámetros de paginación, entonces cada una recibe únicamente sus propios datos: la caché no comparte resultados entre usuarios.
 4. Las mascotas con activo = false no aparecen en el listado.
- **Método de verificación:** pruebas automatizadas MascotaControllerTest.duenoSoloVeSusPropiasMascotasEnListado (criterio 1), adminConservaListadoGlobalDeMascotas y auxiliarConservaAccesoGlobalAlListado (criterio 2), dosDuenosConMismaPaginaNoComparenResultadosDeCache (criterio 3), adminEliminaMascotaExitosamente (criterio 4, que comprueba que la mascota dada de baja desaparece del listado activo).
- **Trazabilidad:** HU-008 → CU-08 → MascotaController.listar → MascotaService.listar (@Cacheable(value = "mascotas", key = "#email + ...")) → MascotaRepository.findAllByDuenioIdAndActivoTrue / findAllByActivoTrue → GET /api/mascotas; evidencia docs/mediciones/sec/A01-access-control.md.
- **Estado:** verificado

REQ-F-010 — Consulta de mascota por identificador

- **Tipo:** Funcional
- **Prioridad:** Must
- **Enunciado:** El sistema deberá permitir consultar el detalle completo de una mascota activa a partir de su identificador, respondiendo 404 con ProblemDetail si no existe y 403 si el solicitante es un ROLE_DUEÑO que no es su propietario.
- **Rationale:** complementa REQ-F-009 para el caso de consulta puntual, necesario antes de una edición o para mostrar el detalle en la interfaz.
- **Criterios de aceptación:**
 1. Dado un ROLE_DUEÑO y una mascota de su propiedad, cuando invoca GET /api/mascotas/{id}, entonces la respuesta es 200 OK con el detalle de esa mascota.
 2. Dado un ROLE_DUEÑO y una mascota de otro dueño, entonces la respuesta es 403 con ProblemDetail de tipo urn:biopet:error:forbidden.

3. Dado un `ROLE_ADMIN` o un `ROLE_VETERINARIO`, entonces puede consultar la mascota de cualquier dueño con 200 OK.
 4. Dado un identificador inexistente o de una mascota inactiva, entonces la respuesta es 404 con `ProblemDetail` de tipo `urn:biopet:error:not-found`.
- **Método de verificación:** pruebas automatizadas `MascotaControllerTest.duenoConsultaSuPropia MascotaPorId` (criterio 1), `duenoConsultaMascotaDeOtroDuenioDevuelve403` (criterio 2), `admin ConsultaMascotaDeCualquierDuenio` y `veterinarioConsultaMascotaDeCualquierDuenio` (criterio 3), `buscarMascotaInexistenteDevuelveProblemDetail` (criterio 4). Evidencia HTTP adicional del criterio 2 en `docs/mediciones/sec/raw/A01-access-control.txt`.
 - **Trazabilidad:** `HU-009` → `CU-09` → `MascotaController.buscar` → `MascotaService.buscar` → `MascotaService.verificarPropiedad` → `RecursoNoEncontradoException` → `GET /api/mascotas/{id}`.
 - **Estado:** verificado

REQ-F-011 — Actualización de mascota con verificación de propiedad

- **Tipo:** Funcional
- **Prioridad:** Must
- **Enunciado:** El sistema deberá permitir que un usuario con rol `ROLE_ADMIN`, `ROLE_VETERINARIO` o `ROLE_AUXILIAR` actualice los datos de una mascota activa, registrando automáticamente la fecha de actualización (`actualizado_en`) y manteniendo la restricción de que el propietario asignado tenga rol `ROLE_DUENO`.
- **Rationale:** heredado de RF-02 de la Entrega 1A.
- **Criterios de aceptación:**
 1. Dado un `ROLE_ADMIN` y una mascota activa, cuando envía `PUT /api/mascotas/{id}` con datos válidos, entonces la respuesta es 200 OK con los datos actualizados.
 2. Dado un nuevo `duenioId` cuyo rol no es `ROLE_DUENO`, entonces la solicitud es rechazada con 400 y `ProblemDetail`.
 3. Dado un usuario con rol `ROLE_DUENO`, cuando intenta actualizar incluso su propia mascota, entonces la respuesta es 403 (la restricción es por rol, no por propiedad).
 4. Tras una actualización, el valor de `actualizado_en` de la fila es posterior al que tenía antes de la operación.
- **Método de verificación:** pruebas automatizadas `MascotaControllerTest.adminActualizaMascota AsignandolaA0troUsuarioActivoDuenoExitosa` (criterio 1), `adminIntentaActualizarMascotaCon RolDeDuenioIncorrectoEsRechazado` (criterio 2), `duenoIntentaActualizarMascotaPropiaSigue Recibiendo403PorRol` (criterio 3), y prueba de integración `TriggerActualizadoEnIntegrationTest` sobre el trigger `set_actualizado_en` en PostgreSQL real (criterio 4).
- **Trazabilidad:** `HU-010` → `CU-10` → `MascotaController.actualizar` → `MascotaService.actualizar` → `trigger set_actualizado_en` (`Backend/src/main/resources/db/migration/V1__ schema_inicial.sql`) → `PUT /api/mascotas/{id}`.
- **Estado:** verificado

REQ-F-012 — Baja lógica de una mascota

- **Tipo:** Funcional
- **Prioridad:** Must
- **Enunciado:** El sistema deberá permitir dar de baja lógicamente una mascota existente (cambio del atributo `activo` a `false`), sin eliminar físicamente el registro de la base de datos.
- **Rationale:** preservación de historial e integridad referencial con consultas, citas y vacunas, que referencian la mascota.
- **Criterios de aceptación:**
 1. Dado un `ROLE_ADMIN` y una mascota activa, cuando envía `DELETE /api/mascotas/{id}`, entonces la respuesta es 204 No Content.
 2. Tras la baja, la fila sigue existiendo en la tabla `mascotas` con `activo = false`; no se ejecuta ningún `DELETE` físico.

3. Tras la baja, la mascota deja de aparecer en GET /api/mascotas y GET /api/mascotas/{id} responde 404.
 4. Dado un usuario con rol `ROLE_DUENO`, cuando intenta dar de baja incluso su propia mascota, entonces la respuesta es 403.
- **Método de verificación:** prueba automatizada `MascotaControllerTest.adminEliminaMascota Exitosamente` (criterios 1 a 3, incluida la comprobación de que la mascota desaparece del listado activo) y `duenoIntentaEliminarMascotaPropiaSigueRecibiendo403PorRol` (criterio 4). Criterio 2: inspección de `MascotaService.eliminar (mascota.setActivo(false); mascotaRepository.save(mascota);`, sin invocación a `delete`).
 - **Trazabilidad:** HU-011 → CU-11 → `MascotaController.eliminar` → `MascotaService.eliminar` → DELETE /api/mascotas/{id}; sección 2.9.2 (ciclo de vida común).
 - **Estado:** verificado

REQ-F-013 — Registro de atención médica y consulta del historial clínico

Requisito con dos obligaciones. Este requisito, heredado de RF-03 y RF-04 de la Entrega 1A, agrupa dos obligaciones distintas con distinto grado de avance: **(A)** registrar y gestionar las consultas médicas, y **(B)** consultar el historial clínico consolidado de una mascota. Conserva su identificador único, su historia HU-012 y su caso de uso CU-12; las obligaciones se separan más abajo con criterios numerados A1...A7 y B1...B5, y el Estado describe el requisito completo.

- **Tipo:** Funcional
- **Prioridad:** Should
- **Enunciado:** El sistema deberá permitir **(A)** registrar, consultar por identificador, actualizar y dar de baja lógica las consultas médicas (motivo, diagnóstico, tratamiento y observaciones) asociadas a una mascota activa y a un usuario con rol `ROLE_VETERINARIO`, restringiendo el acceso de un usuario con rol `ROLE_DUENO` a las consultas de sus propias mascotas; y **(B)** consultar, a través de un endpoint propio, el historial clínico consolidado de una mascota activa —datos de la mascota y de su dueño, número de consultas y de citas, última consulta, última vacuna y próxima vacuna—, ordenando cronológicamente los eventos clínicos que lo componen.
- **Rationale:** heredado de RF-03 (registro de atención médica) y RF-04 (consulta del historial clínico “de forma cronológica”) de la Entrega 1A. La Unidad IV entregó el módulo real `ConsultaController/ConsultaService/ConsultaRepository`, que cubre la obligación A, y la capa de datos de la obligación B (`fn_historial_clinico_mascota`), por lo que el requisito deja de estar sin implementación.
- **Criterios de aceptación:**
 1. **(A1)** Dado un `ROLE_ADMIN`, `ROLE_VETERINARIO` o `ROLE_AUXILIAR`, una mascota activa y un usuario con rol `ROLE_VETERINARIO`, cuando envía POST /api/consultas con datos válidos, entonces la respuesta es 201 Created con la consulta creada y `activo = true`.
 2. **(A2)** Dado un `veterinarioId` cuyo rol no es `ROLE_VETERINARIO`, entonces la solicitud es rechazada con 400 y `ProblemDetail`.
 3. **(A3)** Dado un `ROLE_DUENO`, cuando intenta crear, actualizar o eliminar una consulta, entonces la respuesta es 403.
 4. **(A4)** Dado un `ROLE_DUENO` y una consulta de una mascota ajena, cuando invoca GET /api/consultas/{id}, entonces la respuesta es 403.
 5. **(A5)** Dado un identificador inexistente o de una consulta inactiva, entonces GET /api/consultas/{id} responde 404 con `ProblemDetail`.
 6. **(A6)** Dado un `ROLE_DUENO`, cuando invoca GET /api/consultas, entonces **ningún** elemento de la página devuelta corresponde a una mascota de otro dueño.
 7. **(A7)** Dado un DELETE /api/consultas/{id}, entonces la fila permanece en la tabla con `activo = false` y deja de aparecer en listados y búsquedas.
 8. **(B1)** Dada una mascota activa con consultas, citas y vacunas registradas, cuando se solicita su historial clínico consolidado, entonces la respuesta incluye los datos de la mascota, los de su dueño, el conteo de consultas y de citas, la última consulta, la última vacuna y la próxima vacuna.
 9. **(B2)** Dada una mascota sin eventos clínicos, entonces la respuesta es satisfactoria con los conteos

en cero, no un error.

10. **(B3)** Dado un identificador de mascota inexistente, entonces el resultado es vacío (a nivel de rutina) o 404 con `ProblemDetail` (a nivel de API).
 11. **(B4)** Dado un usuario con rol `ROLE_DUEÑO`, cuando solicita el historial de una mascota ajena, entonces la respuesta es 403.
 12. **(B5)** La agregación se implementa como procedimiento almacenado versionado en `db/procs/`, no como JPQL con joins (ver REQ-NF-013).
- **Método de verificación:** obligación A — pruebas automatizadas `ConsultaControllerTest.adminCreaConsultaExitosamente` (A1), `crearConsultaConCamposInvalidosDevuelve422` e inspección de `ConsultaService.resolverVeterinario` (A2), `duenoNoPuedeCrearConsultaDevuelve403` (A3), `duenoDeOtraMascotaNoPuedeVerConsultaAjena` (A4), `buscarConsultaInexistenteDevuelve404` (A5), `adminEliminaConsultaExitosamente` (A7); **A6 sin artefacto que lo demuestre y, además, incumplido — ver Observaciones.** Obligación B — pruebas de integración `ProcedimientosBiopetIntegrationTest.historialClinico_mascotaConDatos_consolidaHistorial` y `historialClinico_mascotaInexistente_devuelveListaVacía` sobre PostgreSQL real con Testcontainers (B1 a B3 a nivel de rutina) y `docs/basedatos/CATALOGO-SP.md` (B5); **B4 y la exposición de B1 como endpoint: verificación prevista** — prueba de controlador equivalente a `MascotaControllerTest.duenoConsultaMascotaDeOtroDuenioDevuelve403`, a escribir cuando el endpoint exista.
 - **Trazabilidad:** HU-012 → CU-12 → obligación A: `ConsultaController.{listar, buscar, crear, actualizar, eliminar}` → `ConsultaService` → `ConsultaRepository` → GET `/api/consultas`, GET `/api/consultas/{id}`, POST `/api/consultas`, PUT `/api/consultas/{id}`, DELETE `/api/consultas/{id}`; prueba `Backend/src/test/java/com/biopet/ConsultaControllerTest.java`; procedimiento de validación cruzada `db/procs/sp_registrar_consulta_validada.sql` con su prueba `ProcedimientosBiopetIntegrationTest.registrarConsultaValidada_casoFeliz_retornaIdYPersiste`. Obligación B: `db/procs/fn_historial_clinico_mascota.sql` → `ProcedimientoBiopetRepository.historialClinicoMascota` → proyección `com.biopet.repository.HistorialClinico` → `@NamedStoredProcedureQuery` declarado en `com.biopet.entity.Mascota` → `Backend/src/test/java/com/biopet/repository/ProcedimientosBiopetIntegrationTest.java`; migración `Backend/src/main/resources/db/migration/V6__formalizar_procedimientos_jpa.sql`; catálogo `docs/basedatos/CATALOGO-SP.md`; matriz de permisos, sección 2.8.5.
 - **Estado:** implementado
 - **Observaciones:** el requisito **no está cumplido**. Tres limitaciones reales, verificadas contra el código actual y **no corregidas** en esta revisión documental:
 - (a) **El criterio A6 no se cumple.** `ConsultaService.listar()` devuelve `consultaRepository.findAllByActivoTrue(pageable)` tanto para `ROLE_DUEÑO` como para el resto de roles, y `ConsultaRepository` no declara ningún método de filtrado por propietario (a diferencia de `CitaRepository` y `VacunaRepository`, que sí declaran `findAllByMascota_Duenio_IdAndActivoTrue`). Ver REQ-F-006 y REQ-NF-015.
 - (b) **La obligación B no está expuesta.** Ningún controlador de `com.biopet.controller` invoca `ProcedimientoBiopetRepository.historialClinicoMascota`: no hay endpoint de historial consolidado ni pantalla asociada, por lo que B4 no puede comprobarse. Además la rutina devuelve un **resumen consolidado** (conteos, última consulta, última y próxima vacuna), no la lista cronológica completa de eventos que pide el enunciado original.
 - (c) GET `/api/consultas` (listado) y PUT `/api/consultas/{id}` no tienen prueba automatizada dedicada en `ConsultaControllerTest` (6 pruebas frente a las 25 de `CitaControllerTest`, que cubre el módulo equivalente).

REQ-F-014 — Registro de medicamentos prescritos

- **Tipo:** Funcional
- **Prioridad:** Could
- **Enunciado:** El sistema deberá permitir registrar los medicamentos prescritos durante una consulta médica (nombre del medicamento, dosis, frecuencia y duración), asociándolos a la consulta que los origina.

- **Rationale:** heredado de RF-05 de la Entrega 1A. Depende de la obligación A de REQ-F-013 (la consulta médica debe existir antes de poder prescribir sobre ella).
- **Criterios de aceptación:**
 1. Dada una consulta médica existente y activa, cuando se registra una prescripción con medicamento, dosis, frecuencia y duración, entonces la prescripción queda persistida y asociada a esa consulta.
 2. Dada una consulta inexistente, entonces la operación responde 404 con `ProblemDetail`.
 3. Dado un usuario con rol `ROLE_DUENO`, cuando intenta registrar una prescripción, entonces la respuesta es 403.
 4. Dado un `ROLE_DUENO` propietario de la mascota, cuando consulta las prescripciones de una consulta de esa mascota, entonces puede leerlas; si la mascota es ajena, la respuesta es 403.
 5. Dado un cuerpo con campos obligatorios ausentes, entonces la respuesta es 422 con `ProblemDetail` de validación.
- **Método de verificación:** previsto — prueba de controlador con `MockMvc` siguiendo el patrón de `ConsultaControllerTest` (casos 201, 404, 403 y 422), más la entidad y migración `Flyway` correspondientes. No se puede ejecutar hoy porque no existe implementación.
- **Trazabilidad:** HU-013 → CU-13 → depende de REQ-F-013; entidad conceptual del DER de la Entrega 1A (`PFC_Entrega1A_BMT.pdf`, sección 6). Sin controlador, servicio, repositorio, entidad ni migración en el repositorio actual.
- **Estado:** pendiente

REQ-F-015 — Gestión de citas veterinarias

Requisito con dos obligaciones. Heredado de RF-06 de la Entrega 1A (“Registrar, modificar y consultar citas veterinarias mediante calendario interactivo”), agrupa **(A)** la gestión de las citas a través de la API y **(B)** su presentación en un calendario interactivo. Conserva su identificador único, HU-014 y CU-14; las obligaciones se separan con criterios A1...A6 y B1...B5, y el Estado describe el requisito completo.

- **Tipo:** Funcional
- **Prioridad:** Should
- **Enunciado:** El sistema deberá permitir **(A)** listar, consultar por identificador, crear, actualizar y dar de baja lógica citas veterinarias (mascota, veterinario asignado, fecha y hora, estado y motivo), restringiendo la lectura de un usuario con rol `ROLE_DUENO` a las citas de sus propias mascotas y permitiendo a un `ROLE_VETERINARIO` modificar únicamente las citas en las que él es el veterinario asignado; y **(B)** presentar esas citas en una vista de calendario que permita consultarlas por periodo, crear una cita sobre una fecha seleccionada y abrir el detalle de una cita existente, respetando la visibilidad por rol de la sección 2.8.4.
- **Rationale:** heredado de RF-06 de la Entrega 1A, que pide explícitamente “mediante calendario interactivo”. La Unidad IV entregó el módulo real `CitaController/CitaService/CitaRepository` con CRUD completo y reglas de acceso por dato, que cubre la obligación A; la obligación B sigue sin iniciarse.
- **Criterios de aceptación:**
 1. **(A1)** Dado un `ROLE_ADMIN` o `ROLE_AUXILIAR`, una mascota activa y un usuario con rol `ROLE_VETERINARIO`, cuando envía `POST /api/citas` con datos válidos, entonces la respuesta es 201 `Created` y la cita queda con `estado = PROGRAMADA`, aunque el cliente haya enviado otro estado.
 2. **(A2)** Dado un `ROLE_DUENO` con citas propias y existiendo citas de otro dueño, cuando invoca `GET /api/citas`, entonces ningún elemento devuelto corresponde a otro dueño; y `GET /api/citas/{id}` de una cita ajena responde 403.
 3. **(A3)** Dado un `ROLE_VETERINARIO`, cuando intenta actualizar una cita asignada a otro veterinario, entonces la respuesta es 403; cuando actualiza una cita propia, la respuesta es 200 `OK`.
 4. **(A4)** Dado un `ROLE_DUENO` o un `ROLE_VETERINARIO`, cuando intenta `POST /api/citas`, entonces la respuesta es 403; sólo `ROLE_ADMIN` puede ejecutar `DELETE /api/citas/{id}`.
 5. **(A5)** Dada una mascota o un veterinario inexistentes, entonces la respuesta es 404; dado un `veterinarioId` cuyo rol no es `ROLE_VETERINARIO`, entonces 400; dados campos inválidos, 422.
 6. **(A6)** Dado un `DELETE /api/citas/{id}`, entonces la fila permanece con `activo = false` (baja

- lógica) y el valor de **estado** no cambia.
7. **(B1)** Dado un usuario autenticado, cuando abre la vista de citas, entonces se muestra una rejilla de calendario con las citas del periodo visible.
 8. **(B2)** Dado un **ROLE_DUENO**, entonces el calendario muestra únicamente citas de sus propias mascotas.
 9. **(B3)** Dado un **ROLE_ADMIN** o **ROLE_AUXILIAR**, cuando selecciona una fecha libre, entonces puede crear una cita para esa fecha desde el calendario.
 10. **(B4)** Dada una cita mostrada en el calendario, cuando se selecciona, entonces se abre su detalle con mascota, veterinario, fecha, estado y motivo.
 11. **(B5)** La vista cumple el mismo umbral de accesibilidad exigido por REQ-NF-018.
- **Método de verificación:** obligación A — pruebas automatizadas de `CitaControllerTest` (25 pruebas): `crearCitaValidaDevuelve201`, `crearCitaForzandoEstadoDistintoIgnoraElEstadoEnviado`, `auxiliarPuedeCrearCita` (A1); `duenoSoloVeSusPropiasCitasEnListado`, `duenoConsultaCitaDeSuPropiaMascota`, `duenoNoPuedeConsultarCitaDeMascotaAjena` (A2); `veterinarioActualizaSuPropiaCitaDevuelve200`, `veterinarioNoPuedeActualizarCitaDeOtroVeterinarioDevuelve403` (A3); `duenoNoPuedeCrearCitaDevuelve403`, `veterinarioNoPuedeCrearCitaDevuelve403`, `auxiliarNoPuedeEliminarCitaDevuelve403`, `duenoNoPuedeActualizarCitaDevuelve403` (A4); `crearCitaConMascotaInexistenteDevuelve404`, `crearCitaConVeterinarioInexistenteDevuelve404`, `crearCitaConUsuarioNoVeterinarioAsignadoDevuelve400`, `crearCitaConCamposInvalidosDevuelve422` (A5); `adminEliminaCitaDevuelve204YBajaLogica` (A6). Obligación B — **verificación prevista**, no ejecutable hoy: componente Angular en `frontend/src/app/features/` consumiendo `GET /api/citas`, con corrida Lighthouse sobre la nueva ruta (B5) e inspección manual por rol (B1 a B4).
 - **Trazabilidad:** HU-014 → CU-14 → obligación A: `CitaController.{listar, buscar, crear, actualizar, eliminar}` → `CitaService` (`verificarAccesoLectura`, `verificarPermisoEscritura`) → `CitaRepository.findAllByMascota_Duenio_IdAndActivoTrue` → `GET /api/citas`, `GET /api/citas/{id}`, `POST /api/citas`, `PUT /api/citas/{id}`, `DELETE /api/citas/{id}`; procedimiento almacenado de actualización masiva `db/procs/sp_actualizar_estado_citas_masivas.sql` con sus pruebas `ProcedimientosBiopetIntegrationTest.actualizarEstadoCitasMasivas_soloActualizaLasQueCumplenFiltro` y `actualizarEstadoCitasMasivas_estadoInvalido_lanzaExcepcion`; estados y transiciones en la sección 2.9.1; matriz de permisos, sección 2.8.4. Obligación B: sin componente en `frontend/src/app/features/` (sólo existen `login.component.ts`, `mascotas.component.ts` y `vacunas.component.ts`); wireframe conceptual de la Entrega 1A (`PFC_Entrega1A-BMT.pdf`, sección 7).
 - **Estado:** implementado
 - **Observaciones:** el requisito **no está cumplido**. La obligación A está íntegramente realizada y evidenciada: los seis criterios A1 a A6 están cubiertos por `CitaControllerTest`, que es el módulo con mayor cobertura de pruebas del proyecto. La obligación B **no está iniciada**: no existe ningún componente de calendario en `frontend/src/app/features/`, por lo que los criterios B1 a B5 no tienen ni implementación ni artefacto. El requisito no puede declararse **verificado** mientras el calendario interactivo que exige RF-06 siga sin construirse.

REQ-F-016 — API de recepción de telemetría de dispositivos IoT

- **Tipo:** Funcional
- **Prioridad:** Could
- **Enunciado:** El sistema deberá exponer una API autenticada que reciba y almacene lecturas de telemetría (identificador de dispositivo, marca de tiempo y valores medidos) enviadas por dispositivos IoT de rastreo asociados a una mascota registrada.
- **Rationale:** heredado de RF-08 y RF-09 de la Entrega 1A. Requiere una decisión de arquitectura previa sobre el protocolo de ingesta y el mecanismo de autenticación de dispositivos, que no se ha tomado.
- **Criterios de aceptación:**
 1. Dado un dispositivo autenticado y asociado a una mascota activa, cuando envía una lectura válida, entonces la respuesta es 201 **Created** y la lectura queda persistida con su marca de tiempo.
 2. Dada una lectura de un dispositivo no registrado o no asociado a ninguna mascota, entonces la

- respuesta es 403 o 404 con `ProblemDetail`, y no se persiste nada.
3. Dado un cuerpo con valores fuera del rango declarado para el tipo de medida, entonces la respuesta es 422 con `ProblemDetail`.
 4. Dado un `ROLE_DUEÑO`, cuando consulta la telemetría de sus propias mascotas, entonces la obtiene; la de mascotas ajenas responde 403.
- **Método de verificación:** previsto — prueba de integración del endpoint de ingesta con un cliente simulado de dispositivo, más un ADR que fije protocolo y autenticación de dispositivos antes de implementar. No ejecutable hoy.
 - **Trazabilidad:** HU-015 → CU-15 → entidad conceptual `Dispositivo_IoT` del DER de la Entrega 1A; sistema externo identificado en `docs/diagrams/c4-contexto/C4-L1-contexto.md`. Sin controlador, servicio, entidad ni migración en el repositorio actual (ver sección 2.7.3).
 - **Estado:** pendiente

REQ-F-017 — Generación de recomendaciones clínicas informativas a partir del historial médico

- **Tipo:** Funcional
- **Prioridad:** Could
- **Enunciado:** Al recibir una solicitud de recomendaciones para una mascota con historial clínico registrado, el sistema deberá generar, mediante un servicio de IA externo, una lista de recomendaciones de cuidado en texto a partir de los datos del historial clínico de esa mascota, y deberá devolver cada recomendación acompañada de la advertencia explícita “informativa, no sustituye diagnóstico veterinario”, enviando al servicio externo exclusivamente los campos clínicos mínimos enumerados en el criterio 5.
- **Rationale:** heredado de RF-10 de la Entrega 1A; reformulado en la Tercera Entrega para cerrar OBS-04 (ambigüedad leve señalada por el docente en “recomendaciones informativas”), sin ampliar el alcance original: sigue dependiendo de REQ-F-013 (historial clínico) y del mismo servicio de IA externo ya previsto desde la Entrega 1A. En esta revisión se añade además la restricción explícita sobre los datos que pueden salir del sistema (criterio 5), siguiendo el principio técnico de minimización de datos ya aplicado en REQ-F-025.
- **Criterios de aceptación:**
 1. Dado un historial clínico existente para la mascota solicitada, cuando se invoca el endpoint de recomendaciones, entonces la respuesta incluye el campo `recomendaciones: string[]` y, por cada elemento, el campo `advertencia: "informativa, no sustituye diagnóstico veterinario"`.
 2. Dada una mascota sin historial clínico registrado, entonces la respuesta es 200 OK con `recomendaciones: []`, no un error.
 3. Dado un `ROLE_DUEÑO`, cuando solicita recomendaciones para una mascota ajena, entonces la respuesta es 403.
 4. Dado que el servicio de IA externo no responde o devuelve error, entonces la respuesta es 502 Bad Gateway con `ProblemDetail`, siguiendo el mismo patrón que `ExternalApiException`, sin exponer detalles internos del proveedor.
 5. **Restricción de datos enviados al servicio externo.** La carga útil enviada al proveedor de IA contiene exclusivamente: especie, raza, edad o fecha de nacimiento de la mascota, y los campos `motivo`, `diagnóstico`, `tratamiento` y `observaciones` de sus consultas. Queda explícitamente prohibido enviar: el nombre, el correo electrónico o el identificador del dueño; el nombre o el identificador del veterinario; el nombre o el identificador de la mascota; cualquier token JWT, cookie, clave de API propia o credencial del sistema; y cualquier campo no enumerado en la primera frase de este criterio. La clave del proveedor viaja únicamente en la cabecera de autenticación de la llamada saliente, nunca en el cuerpo ni en la URL.
 6. La comprobación del criterio 5 es observable: la carga útil enviada se puede capturar en una prueba con doble de prueba (mock) del cliente externo y comparar campo por campo contra la lista permitida.
- **Método de verificación:** previsto — prueba de integración con un doble de prueba (mock) del servicio de IA, siguiendo el patrón real ya usado en `ExternalApiServiceTest`, que valide: (a) el contrato de entrada/salida (criterios 1 y 2), (b) el control de acceso por propietario (criterio 3), (c) la traducción de fallos del proveedor a 502 (criterio 4) y (d) la carga útil saliente campo por campo

contra la lista permitida (criterios 5 y 6). No ejecutable hoy: no hay implementación.

- **Trazabilidad:** HU-016 → CU-16 → depende de REQ-F-013 (ambas obligaciones); patrón de integración externa de referencia ya implementado en `com.biopet.integration.ExternalApiClient/ExternalApiService` y en `com.biopet.exception.ExternalApiException → GlobalExceptionHandler.errorApiExterna → ProblemType.BAD_GATEWAY`; sistema externo identificado en `docs/diagrams/c4-contexto/C4-L1-contexto.md`; sección 2.6 (servicio de IA declarado como **no integrado**).
- **Estado:** pendiente
- **Observaciones:** el proveedor concreto del servicio de IA sigue siendo una decisión de arquitectura no tomada; no existe ADR que la fije. El criterio 5 define el principio técnico mínimo de minimización de datos que deberá cumplir la implementación futura; **no** declara conformidad con ninguna norma legal de protección de datos, porque el proyecto no ha realizado ese análisis.

REQ-F-018 — Comprobantes de pago digitales

- **Tipo:** Funcional
- **Prioridad:** Should
- **Enunciado:** El sistema deberá generar un comprobante de pago digital en formato PDF por cada servicio veterinario facturado, registrando el método de pago empleado y asociando el comprobante al dueño y a la mascota atendidos.
- **Rationale:** heredado de RF-11 y RF-12 de la Entrega 1A.
- **Criterios de aceptación:**
 1. Dado un servicio veterinario registrado y un método de pago válido, cuando se emite el comprobante, entonces se genera un archivo PDF descargable que incluye identificador del comprobante, fecha, dueño, mascota, detalle del servicio, importe y método de pago.
 2. El identificador del comprobante es único y correlativo.
 3. Dado un `ROLE_DUEÑO`, cuando solicita sus propios comprobantes, entonces los obtiene; los de otro dueño responden 403.
 4. Dado un método de pago no admitido, entonces la respuesta es 422 con `ProblemDetail`.
- **Método de verificación:** previsto — prueba de integración que emita un comprobante y valide la estructura del PDF generado y la unicidad del correlativo, más prueba de control de acceso por propietario. No ejecutable hoy.
- **Trazabilidad:** HU-017 → CU-17 → entidades conceptuales `Factura` y `Detalle_Factura` del DER de la Entrega 1A. Para el criterio 2 existe ya en la base de datos el generador de códigos correlativos `db/procs/fn_siguiete_numero_ficha.sql` (`ProcedimientoBiopetRepository.siguieteNumeroFicha`, secuencia `seq_ficha_biopet`), reutilizable como numerador. Sin controlador, servicio, entidad ni migración de facturación en el repositorio actual.
- **Estado:** pendiente

REQ-F-019 — Reportes estadísticos exportables

- **Tipo:** Funcional
- **Prioridad:** Could
- **Enunciado:** El sistema deberá permitir generar reportes estadísticos de actividad de la clínica para un rango de fechas dado y exportarlos en formato PDF y Excel.
- **Rationale:** heredado de RF-14 de la Entrega 1A.
- **Criterios de aceptación:**
 1. Dado un rango de fechas válido, cuando se solicita el reporte, entonces se obtienen los indicadores del periodo: mascotas activas, citas programadas, consultas y vacunas dentro del rango, y mascotas sin consulta.
 2. Dado un rango invertido (fecha inicial posterior a la final), entonces los indicadores del periodo resultan en cero, sin error.
 3. Dado un reporte generado, entonces puede descargarse como PDF y como Excel, con los mismos valores en ambos formatos.
 4. Dado un usuario cuyo rol no es `ROLE_ADMIN`, cuando solicita el reporte global de la clínica, entonces la respuesta es 403.

5. La agregación multi-tabla se implementa como procedimiento almacenado (ver REQ-NF-013), no como JPQL con joins.
- **Método de verificación:** pruebas de integración `ProcedimientosBiopetIntegrationTest.reporteDashboard_conDatos_calculaIndicadores` y `reporteDashboard_rangoInvertido_noCuentaDentroDelRango` sobre PostgreSQL real (criterios 1, 2 y 5). Criterios 3 y 4: **verificación prevista** — prueba de controlador sobre el endpoint de reportes y verificación del contenido de los archivos exportados, a escribir cuando existan.
 - **Trazabilidad:** HU-018 → CU-18 → procedimiento almacenado `db/procs/fn_reporte_dashboard.sql` → `ProcedimientoBiopetRepository.reporteDashboard` → proyección `com.biopet.repository.ReporteDashboard` → `Backend/src/test/java/com/biopet/repository/ProcedimientosBiopetIntegrationTest.java`; catálogo `docs/basedatos/CATALOGO-SP.md` (categoría “3) Reporte”). Sin controlador, servicio de exportación ni endpoint en el repositorio actual.
 - **Estado:** pendiente
 - **Observaciones:** la capa de datos del reporte existe y está probada (procedimiento almacenado, repositorio, proyección y prueba de integración), pero **ninguna** de las obligaciones visibles del requisito —endpoint, exportación a PDF y a Excel, restricción por rol— está implementada, por lo que el estado sigue siendo **pendiente** y no **implementado**. Esa base de datos se registra aquí en Trazabilidad para que el trabajo ya hecho no quede oculto.

REQ-F-020 — Auditoría de operaciones del sistema

- **Tipo:** Funcional
- **Prioridad:** Should
- **Enunciado:** El sistema deberá registrar, para cada operación relevante —los eventos de autenticación y las operaciones de creación, actualización y baja sobre mascotas, citas, consultas, vacunas y usuarios—, el sujeto que la ejecuta, la marca de tiempo, la dirección IP de origen y el resultado de la operación.
- **Rationale:** heredado de RF-15 de la Entrega 1A, reforzado por el control OWASP A09 exigido por el bloque C.2 de la Guía.
- **Criterios de aceptación:**
 1. Dado un evento de autenticación (login exitoso, login fallido, bloqueo por rate limit, refresh exitoso, refresh fallido, logout, uso de token revocado), cuando ocurre, entonces se emite una línea de log estructurada con prefijo `AUTH_AUDIT`, el tipo de evento, la IP, la marca de tiempo UTC y el sujeto.
 2. Los valores nulos se normalizan a **unknown** y los caracteres de control se eliminan del mensaje, de modo que un sujeto manipulado no pueda inyectar líneas de log falsas (*log forging*).
 3. La línea de log no contiene contraseñas, hashes ni tokens.
 4. Dada una operación de creación, actualización o baja sobre mascotas, citas, consultas, vacunas o usuarios, cuando se ejecuta, entonces se registra un evento de auditoría con sujeto, marca de tiempo, IP, recurso afectado y resultado.
- **Método de verificación:** pruebas automatizadas `AuthenticationAuditServiceTest` (10 pruebas): `loginExitosoRegistraEventoEstructurado`, `loginFallidoRegistraEventoWarn`, `loginBloqueadoRegistraEventoWarn`, `refreshExitosoRegistraEventoInfo`, `refreshFallidoRegistraEventoWarn`, `logoutExitosoRegistraEventoInfo`, `tokenRevocadoRegistraEventoWarn` (criterio 1); `valoresNulosSeNormalizanComoUnknown` y `eliminaCaracteresDeControlParaEvitarLogForging` (criterio 2); `noRegistraDatosSensibles` (criterio 3); más la captura de log real en `docs/mediciones/sec/A09-logging.md` y `docs/mediciones/sec/raw/A09-audit-logs.txt`. Criterio 4: **sin artefacto** — ver Observaciones.
- **Trazabilidad:** HU-019 → CU-19 → `com.biopet.security.AuthenticationAuditService` (métodos `loginExitoso`, `loginFallido`, `loginBloqueado`, `refreshExitoso`, `refreshFallido`, `logoutExitoso`, `tokenRevocado`) → invocado desde `AuthService` y `JwtAuthenticationFilter` → prueba `Backend/src/test/java/com/biopet/security/AuthenticationAuditServiceTest.java` → evidencia `docs/mediciones/sec/A09-logging.md`; requisito no funcional hermano REQ-NF-009.
- **Estado:** implementado
- **Observaciones:** el criterio 4 **no se cumple**: sólo existe auditoría de eventos de *autenticación* (cubierta además por REQ-NF-009 y operativa en producción). No hay ningún interceptor, aspecto ni servicio

que registre las operaciones CRUD de negocio sobre mascotas, citas, consultas, vacunas ni usuarios. Se marca **implementado** y no **pendiente** para no ocultar el trabajo de auditoría de autenticación ya entregado y verificado.

REQ-F-021 — Resumen de mascotas activas agrupadas por especie

- **Tipo:** Funcional
- **Prioridad:** Should
- **Enunciado:** El sistema deberá permitir consultar el total de mascotas activas agrupadas por especie; para roles distintos de `ROLE_ADMIN` el resultado deberá restringirse siempre a las mascotas del propio usuario autenticado, y para el rol `ROLE_ADMIN` deberá poder filtrarse por un dueño específico o consultarse sin filtro para obtener el total global.
- **Rationale:** requisito de la Tercera Entrega, exigido por el bloque A.2.2 de la Guía como ejemplo de operación agregada (`GROUP BY`) que debe implementarse obligatoriamente vía función/procedimiento almacenado, no vía JPQL. Se numera 021 (no 013) porque los identificadores 013 a 020 ya estaban ocupados por los requisitos heredados de la Entrega 1A, fijados en `HistoriasUsuario.md/CasosDeUso.md` antes de que este requisito existiera.
- **Criterios de aceptación:**
 1. Dadas dos mascotas activas de un mismo dueño con especies distintas (“Perro” y “Gato”), cuando se solicita el resumen filtrado por ese dueño, entonces se obtienen exactamente dos filas, una por especie, cada una con total 1.
 2. Dado un identificador de dueño inexistente, entonces el resultado es una lista vacía, no un error.
 3. Dado un usuario cuyo rol no es `ROLE_ADMIN`, cuando invoca `GET /api/mascotas/resumen-especies` con o sin parámetro `duenioId`, entonces el resultado se calcula siempre sobre su propio identificador: el parámetro enviado se ignora.
 4. Dado un usuario `ROLE_ADMIN`, cuando invoca el endpoint sin parámetro `duenioId`, entonces el resumen es global (todas las mascotas activas); cuando lo invoca con `duenioId`, el resumen se limita a ese dueño.
 5. La agregación se ejecuta mediante un procedimiento almacenado versionado, no mediante JPQL con `GROUP BY`.
- **Método de verificación:** prueba de integración `ResumenEspeciesIntegrationTest.resumenAgruparPorEspecieYFiltrarPorDuenio` sobre PostgreSQL real con `Testcontainers` (criterio 1) y `ProcedimientosBiopetIntegrationTest.resumenPorEspecie_duenioInexistente_devuelveListaVacía` (criterio 2). Criterios 3 y 4: inspección de `MascotaService.resumenPorEspecie`, cuya única regla es `Long duenioIdEfectivo = (usuarioAutenticado.getRol() == Rol.ROLE_ADMIN) ? duenioIdSolicitado : usuarioAutenticado.getId();`. Criterio 5: docs/basedatos/CATALOGO-SP.md y scripts/audit-sql-dynamic.sh (ver REQ-NF-013).
- **Trazabilidad:** HU-020 → CU-20 → `MascotaController.resumenPorEspecies` → `MascotaService.resumenPorEspecie` → `ProcedimientoBiopetRepository.resumenPorEspecie` → procedimiento db/procs/fn_resumen_mascotas_por_especie.sql → `GET /api/mascotas/resumen-especies`; proyección `com.biopet.repository.ResumenEspecie`; DTO `ResumenEspecieResponse`; pruebas `ResumenEspeciesIntegrationTest` y `ProcedimientosBiopetIntegrationTest`; catálogo docs/basedatos/CATALOGO-SP.md (categoría “2) Agregado”).
- **Estado:** verificado
- **Observaciones:** los criterios 3 y 4 se verifican por inspección de las cuatro líneas citadas de `MascotaService.resumenPorEspecie`; no existe prueba automatizada de controlador que ejercite el endpoint por rol. Queda registrado como mejora de cobertura, no como defecto funcional.

REQ-F-022 — Notificaciones al usuario por correo electrónico (RF-07 recuperado)

- **Tipo:** Funcional
- **Prioridad:** Could
- **Enunciado:** Al ocurrir un evento relevante para la cuenta de un usuario (por ejemplo, un registro exitoso), el sistema deberá enviar una notificación por correo electrónico al usuario afectado a través de un servicio de correo externo, de forma asíncrona, sin bloquear ni condicionar la respuesta HTTP de la

operación que originó el evento.

- **Rationale:** corresponde al RF-07 original de la Entrega 1A (“Servicio de Correos”), documentado como sistema externo en `docs/diagrams/c4-contexto/C4-L1-contexto.md` pero sin requisito REQ-F formal hasta la Tercera Entrega. Se incorporó para cerrar OBS-02 sin duplicar REQ-F-007 (funcionalidad distinta ya implementada). Por eso se numera 022, el siguiente identificador libre después de REQ-F-021.
- **Criterios de aceptación:**
 1. Dado un registro de usuario exitoso y un servicio de correo disponible, cuando se completa el registro, entonces se envía un correo de bienvenida a la dirección registrada.
 2. Dado que el servicio de correo externo no está disponible, cuando ocurre un evento notificable, entonces la operación de dominio que lo originó (por ejemplo, el registro) completa exitosamente con su código HTTP normal, y el fallo de envío queda registrado en el log del sistema sin propagarse como error al cliente.
 3. El envío es asíncrono: el tiempo de respuesta de la operación de dominio no depende del tiempo de respuesta del servicio de correo.
 4. El cuerpo del correo no incluye contraseñas, hashes ni tokens.
- **Método de verificación:** previsto — prueba de integración con un servidor SMTP de pruebas (por ejemplo, Mailhog o GreenMail) que confirme el envío en el caso exitoso (criterio 1), y prueba con doble de prueba que fuerce el fallo del proveedor y verifique que la operación de dominio sigue devolviendo su código normal (criterio 2), siguiendo el patrón real ya usado en `ExternalApiServiceTest.fallaAlEscribirEnRedisNoRompeLaRespuesta`. No ejecutable hoy: no hay implementación.
- **Trazabilidad:** HU-021 → CU-21 → sistema externo “Servicio de Correos” identificado en `docs/diagrams/c4-contexto/C4-L1-contexto.md`, sección “Trazabilidad”; sección 2.6 de este documento (declarado **no integrado**). Sin controlador, servicio, configuración SMTP ni dependencia de correo en `Backend/pom.xml`.
- **Estado:** pendiente
- **Observaciones:** el proveedor de correo concreto (SMTP propio, SES, SendGrid u otro) es una decisión de arquitectura no tomada; no existe ADR que la fije.

REQ-F-023 — Gestión administrativa de usuarios

- **Tipo:** Funcional
- **Prioridad:** Should
- **Enunciado:** El sistema deberá permitir que un usuario con rol `ROLE_ADMIN` liste, consulte por identificador, cree, actualice y dé de baja lógica cuentas de usuario de cualquier rol, sin permitir que un administrador modifique el rol de su propia cuenta.
- **Rationale:** requisito de la actividad de Unidad IV; formaliza el CRUD administrativo de `Usuario` `Controller/UsuarioService`, funcionalidad distinta de REQ-F-007 (consulta del perfil propio, `/api/usuarios/me`), que no se modifica ni se duplica. Se numera 023, el siguiente identificador libre después de REQ-F-022.
- **Criterios de aceptación:**
 1. Dado un `ROLE_ADMIN`, cuando invoca `GET /api/usuarios`, entonces obtiene la página de usuarios; cuando invoca `GET /api/usuarios/{id}` con un identificador existente, obtiene ese usuario; con uno inexistente, 404 con `ProblemDetail`.
 2. Dado un `ROLE_ADMIN` y datos válidos, cuando invoca `POST /api/usuarios`, entonces la respuesta es 201 `Created` con el usuario creado; con un correo ya registrado, 409 `Conflict`; sin contraseña, 400; con campos inválidos, 422.
 3. Dado un `ROLE_ADMIN`, cuando intenta cambiar el rol de su **propia** cuenta mediante `PUT /api/usuarios/{id}`, entonces la respuesta es 403.
 4. Dado un `ROLE_ADMIN`, cuando invoca `DELETE /api/usuarios/{id}`, entonces la respuesta es 204 y el usuario queda con `activo = false` (baja lógica, sin borrado físico).
 5. Dado un usuario cuyo rol no es `ROLE_ADMIN` (incluido `ROLE_VETERINARIO`), cuando invoca cualquiera de estas cinco operaciones, entonces la respuesta es 403; sin autenticación, 401.
- **Método de verificación:** pruebas automatizadas de `UsuarioControllerTest` (16 pruebas): `listarUsuariosComoAdmin`, `buscarUsuarioPorId`, `buscarUsuarioInexistenteDevuelve404` (crite-

rio 1); crearUsuarioDevuelve201, crearUsuarioConEmailDuplicadoDevuelve409, crearUsuarioSinPasswordDevuelve400, crearUsuarioConCamposInvalidosDevuelve422, actualizarUsuarioDevuelve200, actualizarUsuarioInexistenteDevuelve404 (criterio 2); adminNoPuedeEscalarSuPropioRolDevuelve403 (criterio 3); eliminarUsuarioDevuelve204YBajaLogica, eliminarUsuarioInexistenteDevuelve404 (criterio 4); accesoConRolNoAdminDevuelve403, veterinarioNoPuedeCrearUsuariosDevuelve403, accesoSinAutenticacionDevuelve401 (criterio 5).

- **Trazabilidad:** HU-022 → CU-22 → UsuarioController.{listar,buscar,crear,actualizar,eliminar} → UsuarioService → UsuarioRepository → GET /api/usuarios, GET /api/usuarios/{id}, POST /api/usuarios, PUT /api/usuarios/{id}, DELETE /api/usuarios/{id}; DTO UsuarioRequest/UsuarioResponse; prueba Backend/src/test/java/com/biopet/UsuarioControllerTest.java; matriz de permisos, sección 2.8.2.
- **Estado:** verificado

REQ-F-024 — Gestión de vacunas

- **Tipo:** Funcional
- **Prioridad:** Should
- **Enunciado:** El sistema deberá permitir registrar, consultar (de forma global, filtrada por mascota, o por identificador), actualizar y dar de baja lógica las vacunas aplicadas a una mascota, restringiendo la consulta de un usuario con rol ROLE_DUEÑO a las vacunas de sus propias mascotas.
- **Rationale:** requisito de la actividad de Unidad IV; formaliza el módulo VacunaController/VacunaService, sin requisito formal previo en el SRS.
- **Criterios de aceptación:**
 1. Dado un ROLE_ADMIN, ROLE_VETERINARIO o ROLE_AUXILIAR y una mascota activa, cuando envía POST /api/vacunas con datos válidos, entonces la respuesta es 201 Created; con una mascota inexistente, 404; con campos inválidos, 422.
 2. Dado un ROLE_DUEÑO, cuando intenta crear una vacuna, entonces la respuesta es 403.
 3. Dado un ROLE_DUEÑO con vacunas propias y existiendo vacunas de mascotas ajenas, cuando invoca GET /api/vacunas, entonces ningún elemento devuelto corresponde a una mascota de otro dueño.
 4. Dado un ROLE_DUEÑO y una vacuna de una mascota ajena, cuando invoca GET /api/vacunas/{id}, entonces la respuesta es 403; con un identificador inexistente, 404.
 5. Dado un ROLE_DUEÑO y un mascotaId ajeno, cuando invoca GET /api/vacunas/mascota/{mascotaId}, entonces la respuesta es 403; con una mascota propia, 200 OK con las vacunas de esa mascota.
 6. Dado un DELETE /api/vacunas/{id} autorizado, entonces la vacuna queda con activo = false y deja de aparecer en los listados.
- **Método de verificación:** pruebas automatizadas de VacunaControllerTest (9 pruebas): adminCreaVacunaExitosamente, crearVacunaConMascotaInexistenteDevuelve404, crearVacunaConCamposInvalidosDevuelve422 (criterio 1); duenoIntentaCrearVacunaEsRechazadoPorRol (criterio 2); duenoSoloVeVacunasDeSusPropiasMascotas (criterio 3); duenoConsultaVacunaDeMascotaAjenaDevuelve403, buscarVacunaInexistenteDevuelve404 (criterio 4); adminActualizaVacunaExitosamente, adminEliminaVacunaExitosamente (criterio 6). Criterio 5: inspección de VacunaService.listarPorMascota, que invoca verificarAcceso(usuario, mascota) antes de consultar el repositorio, aplicando la misma regla que los criterios 3 y 4; y colección Postman docs/postman/BIOPET-Vacunas.postman_collection.json (12 requests, ejecutable con Newman).
- **Trazabilidad:** HU-023 → CU-23 → VacunaController.{listar,listarPorMascota,buscar,crear,actualizar,eliminar} → VacunaService (verificarAcceso) → VacunaRepository.findAllByMascota_Duenio_IdAndActivoTrue → GET /api/vacunas, GET /api/vacunas/mascota/{mascotaId}, GET /api/vacunas/{id}, POST /api/vacunas, PUT /api/vacunas/{id}, DELETE /api/vacunas/{id}; interfaz de usuario frontend/src/app/features/vacunas.component.ts; matriz de permisos, sección 2.8.6.
- **Estado:** verificado
- **Observaciones:** la operación GET /api/vacunas/mascota/{mascotaId} no tiene prueba automatizada dedicada en VacunaControllerTest; su criterio 5 se sostiene por inspección de código y por la colección

Postman, no por una prueba JUnit. La corrida Newman archivada (`docs/mediciones/postman/newman-report.json`, 2026-07-31) es **anterior** a la colección de vacunas y no puede citarse como evidencia de ejecución de esta última. Queda registrado como mejora de cobertura pendiente.

REQ-F-025 — Consulta de información externa de especies

- **Tipo:** Funcional
- **Prioridad:** Could
- **Enunciado:** El sistema deberá permitir a un usuario autenticado consultar información de una especie (nombre científico, hábitat y dieta) desde un servicio externo, utilizando una caché Redis para evitar llamadas repetidas al servicio externo dentro de una ventana de tiempo configurable, y enviando al servicio externo únicamente el nombre de la especie consultada.
- **Rationale:** requisito de la actividad de Unidad IV; integración con un proveedor externo (API Ninjas, *Animals API*) vía `ExternalApiClient`, con patrón *cache-aside* en `ExternalApiService` (TTL configurable mediante `app.external-api.cache-ttl-seconds`, valor por defecto 600 s).
- **Criterios de aceptación:**
 1. Dada una especie ya consultada dentro de la ventana de TTL, cuando se vuelve a consultar, entonces la respuesta se sirve desde Redis con **fuelle** = "cache" y **no** se invoca al servicio externo.
 2. Dada una especie no cacheada, cuando se consulta, entonces se invoca al servicio externo, la respuesta se devuelve con **fuelle** = "api-ninjas" y se almacena en Redis bajo la clave `external-api:animal:<especie normalizada>` con el TTL configurado.
 3. Dado que el servicio externo no devuelve resultados para la especie, entonces la respuesta es 502 Bad Gateway con `ProblemDetail` de tipo `urn:biopet:error:external-service`.
 4. Dado que la escritura en la caché falla, entonces la respuesta al cliente se entrega igualmente: el fallo de caché no rompe la operación.
 5. Dado un valor cacheado corrupto (JSON no deserializable), entonces se señala el error de forma controlada en lugar de devolver datos inválidos.
 6. **Restricción de datos enviados al servicio externo.** La llamada saliente contiene únicamente el nombre de la especie normalizado (`especie.trim().toLowerCase()`) como parámetro `name`; no se envía ningún dato de usuario, mascota, dueño ni veterinario. La clave del proveedor viaja solo en la cabecera `X-API-Key`, nunca en la URL ni en el cuerpo.
- **Método de verificación:** pruebas automatizadas de `ExternalApiServiceTest` (6 pruebas, `Backend/src/test/java/com/biopet/integration/`): `cacheHitDevuelveDatosDesdeRedisSinLlamarApiExterna` (criterio 1), `cacheMissConTaxonomiaYCaracteristicasCompletasGuardaEnCache` y `cacheMissSinTaxonomiaNiCaracteristicasDevuelveCamposNulos` (criterio 2), `cacheMissConResultadosVaciosLanzaExcepcion` (criterio 3), `fallaAlEscribirEnRedisNoRompeLaRespuesta` (criterio 4), `cacheConJsonCorruptoLanzaExcepcion` (criterio 5). Criterio 6: inspección de `ExternalApiClient.buscarPorEspecie`, cuya URL se compone únicamente con `queryParams("name", especie)` y cuya cabecera es `X-API-Key`. Medición empírica adicional en `docs/u4/evidencias/fred/redis-cache-comparacion.md` (comparación real de tiempos de respuesta con caché fría vs. caché caliente).
- **Trazabilidad:** HU-024 → CU-24 → `ExternalApiController.infoEspecie` → `ExternalApiService.obtenerInfoEspecie` → `ExternalApiClient.buscarPorEspecie` → GET `/api/externa/especies?especie=`; DTO `ExternalApiResponse`; excepción `ExternalApiException` → `GlobalExceptionHandler.errorApiExterna` → `ProblemType.BAD_GATEWAY`; configuración `app.external-api.base-url`, `app.external-api.key` (`APP_EXTERNAL_API_KEY` en `.env.example`), `app.external-api.cache-ttl-seconds`; interfaz externa declarada en la sección 2.6 y en la sección 2.7.2.
- **Estado:** verificado

REQ-F-026 — Recuperación de contraseña

- **Tipo:** Funcional
- **Prioridad:** Should
- **Enunciado:** El sistema deberá permitir que un usuario registrado que ha olvidado su contraseña solicite su restablecimiento indicando su correo electrónico, recibiendo un enlace con un token de un solo uso y vigencia limitada, mediante el cual deberá poder establecer una contraseña nueva sin intervención de un

administrador.

- **Rationale:** requisito nuevo incorporado en esta revisión. La retroalimentación del docente-director señaló su ausencia: hoy un usuario que olvida su contraseña **no tiene ninguna vía de recuperación** — la única forma de restablecerla es que un `ROLE_ADMIN` la sobrescriba mediante `PUT /api/usuarios/{id}` (REQ-F-023), lo que obliga a que un tercero conozca la contraseña nueva. Se documenta como requisito planificado, no como funcionalidad entregada.
- **Criterios de aceptación:**
 1. Dado un correo correspondiente a una cuenta activa, cuando se solicita el restablecimiento, entonces se genera un token de un solo uso con vigencia limitada y se envía al correo registrado.
 2. Dado un correo **no** registrado, entonces la respuesta al cliente es idéntica a la del caso anterior (mismo código y mismo cuerpo), para no revelar qué correos existen en el sistema (enumeración de cuentas).
 3. Dado un token válido y no usado, cuando se envía junto con una contraseña nueva que cumple REQ-NF-016, entonces la contraseña se actualiza, se almacena cifrada con BCrypt y el token queda invalidado.
 4. Dado un token ya usado, expirado o inexistente, entonces la respuesta es 400 con `ProblemDetail`, y la contraseña no se modifica.
 5. Tras un restablecimiento exitoso, las sesiones vigentes de ese usuario quedan revocadas: los access y refresh tokens emitidos antes del cambio responden 401.
 6. Los endpoints de solicitud y de confirmación están sujetos a la misma limitación de intentos por IP que el login (REQ-NF-010).
- **Método de verificación:** previsto — pruebas de controlador con `MockMvc` siguiendo el patrón de `AuthControllerTest`, que cubran: respuesta uniforme para correo existente e inexistente (criterio 2), restablecimiento exitoso y revocación de sesiones previas (criterios 3 y 5), token usado/expirado (criterio 4) y bloqueo por rate limit (criterio 6); más la prueba de envío de correo descrita en REQ-F-022. No ejecutable hoy: no hay implementación.
- **Trazabilidad:** depende de REQ-F-022 (envío de correo, **pendiente**) y de REQ-NF-016 (política de contraseñas). Reutilizará `TokenBlacklistService.revoke` para el criterio 5 y `LoginRateLimiterService` para el criterio 6, ambos ya implementados. Sin historia de usuario ni caso de uso propios todavía: se deberán crear la historia y el caso de uso correspondientes, con el siguiente identificador libre de cada serie, al planificar su implementación. Sin endpoint, servicio, entidad de token ni migración en el repositorio actual: la búsqueda de `recuperar|reset|forgot|olvid` sobre `Backend/src` y `frontend/src` no devuelve ninguna coincidencia.
- **Estado:** pendiente
- **Observaciones:** este requisito **no está implementado** y no debe presentarse como entregado. Se incorpora al SRS para que la ausencia quede declarada explícitamente en lugar de omitida. Su fila en `docs/trazabilidad/matriz.csv` no referencia HU ni CU porque ninguna existe; el respaldo declarado es el método de verificación previsto.

3.2. Requisitos no funcionales (REQ-NF)

Los requisitos no funcionales usan la misma plantilla de la sección 1.3.2. En el campo **Tipo** se indica la categoría de calidad. En el campo **Trazabilidad** no se declaran historias de usuario salvo que existan realmente: un requisito no funcional traza legítimamente hacia decisiones de arquitectura, configuración, clases, pruebas, scripts, mediciones, evidencias, normas y documentación técnica.

REQ-NF-001 — Rendimiento del listado de mascotas

- **Tipo:** Rendimiento
- **Prioridad:** Must
- **Enunciado:** El tiempo de respuesta del listado de mascotas (`GET /api/mascotas`) no deberá superar 200 ms en el percentil 95 con caché caliente, ni 500 ms en el percentil 95 con caché fría.
- **Rationale:** heredado de RNF-01/RNF-WEB-04 de la Entrega 1A, con umbrales cuantitativos añadidos por el bloque C.1 de la Guía.
- **Criterios de aceptación:**

1. En una corrida de carga con caché caliente sobre GET /api/mascotas, el p95 medido es ≤ 200 ms.
 2. En una corrida de carga con caché fría sobre el mismo endpoint, el p95 medido es ≤ 500 ms.
 3. La tasa de error HTTP en ambas corridas es 0,0 %.
 4. Las mediciones son reproducibles: los archivos crudos de cada corrida quedan archivados con fecha y con el identificador de la versión medida.
- **Método de verificación:** medición con k6 — 10 corridas reales (5 en caliente y 5 en frío) ejecutadas el 2026-09-03 sobre el código del tag v1.0.0. Resultado registrado en docs/mediciones/perf/REPORT.md: p95 entre 6,04 ms y 18,56 ms según la corrida (máximo 15,06 ms en caché caliente y 18,56 ms en caché fría), muy por debajo de los umbrales de 200 ms y 500 ms, con 0,0 % de error en todas las corridas.
 - **Trazabilidad:** archivos crudos docs/mediciones/perf/k6-20260903T*-local-tls-v1.0.0-*.json (10 archivos) y reporte docs/mediciones/perf/REPORT.md; guiones de carga en k6/; endpoint medido GET /api/mascotas (MascotaController.listar); caché @Cacheable(value = "mascotas") en MascotaService.listar con TTL de REQ-NF-012; procedencia de los datos en docs/mediciones/DATA-PROVENANCE.md.
 - **Estado:** verificado

REQ-NF-002 — Comunicación cifrada obligatoria

- **Tipo:** Seguridad
- **Prioridad:** Must
- **Enunciado:** Toda comunicación entre cliente y servidor deberá realizarse mediante HTTPS/TLS, y las respuestas servidas sobre HTTPS deberán incluir la cabecera Strict-Transport-Security.
- **Rationale:** heredado de RNF-02/RNF-WEB-02 de la Entrega 1A; reforzado por el control OWASP A02 del bloque C.2 de la Guía.
- **Criterios de aceptación:**
 1. Una conexión al puerto TLS negocia TLS 1.3 con una suite AEAD; la captura curl -v lo muestra explícitamente.
 2. Las respuestas servidas sobre HTTPS incluyen Strict-Transport-Security: max-age=31536000; includeSubDomains; preload.
 3. Las respuestas servidas sobre HTTP plano **no** incluyen la cabecera HSTS (evitando anunciarla sobre un canal no cifrado).
 4. Las respuestas incluyen además X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: no-referrer y la política Content-Security-Policy configurada.
- **Método de verificación:** captura real curl -v archivada en docs/mediciones/sec/raw/A02-tls.txt (TLSv1.3, TLS_AES_256_GCM_SHA384) y documentada en docs/mediciones/sec/A02-cryptography-tls.md (criterio 1); pruebas automatizadas SecurityHeadersTest.respuestaProtegidaIncluyeCabecerasDeSeguridad (criterios 2 y 4), SecurityHeadersTest.peticionHttpNoIncluyeHsts (criterio 3) y respuestasPublicasTambienIncluyenCabecerasBasicas; capturas de cabeceras en docs/mediciones/sec/raw/https-8443-headers.txt y http-8080-headers.txt.
- **Trazabilidad:** SecurityConfig (bloque .headers(...)), TomcatDualConnectorConfig (conector dual HTTP/HTTPS), Backend/src/main/resources/application-tls.yml, docker-compose.tls.yml, scripts/generate-dev-keystore.sh, docs/despliegue/Caddyfile y nginx-render.conf; validación documentada en docs/despliegue/VALIDACION-HEADERS-Z4.md; prueba Backend/src/test/java/com/biopet/SecurityHeadersTest.java.
- **Estado:** verificado
- **Observaciones:** la evidencia TLS se generó sobre el entorno de TLS de desarrollo (docker-compose.tls.yml con keystore generado por scripts/generate-dev-keystore.sh), no sobre el certificado del despliegue en Render. La configuración y las cabeceras verificadas son las mismas; el certificado y su cadena, no.

REQ-NF-003 — Expiración configurable de tokens JWT

- **Tipo:** Seguridad
- **Prioridad:** Must
- **Enunciado:** El sistema deberá emitir access tokens con expiración de 1 hora y refresh tokens con

expiración de 7 días, ambos configurables mediante variables de entorno (JWT_EXPIRATION_MS, JWT_REFRESH_EXPIRATION_MS), sin valores fijos en el código.

- **Rationale:** heredado de RNF-03/RNF-WEB-03 de la Entrega 1A.
- **Criterios de aceptación:**
 1. Los valores por defecto configurados son 3 600 000 ms (1 hora) para el access token y 604 800 000 ms (7 días) para el refresh token.
 2. Ambos valores se leen de variables de entorno y pueden sobrescribirse sin recompilar.
 3. Un token cuyo `exp` ya pasó es rechazado por el servicio de validación.
 4. El código fuente no contiene ninguno de esos dos valores literalmente escrito en una clase Java.
- **Método de verificación:** inspección de `Backend/src/main/resources/application.yml`, bloque `security.jwt: expiration-ms: ${JWT_EXPIRATION_MS:3600000}` y `refresh-expiration-ms: ${JWT_REFRESH_EXPIRATION_MS:604800000}` (criterios 1, 2 y 4); prueba automatizada `JwtServiceTest.tokenExpiradoEsRechazado` (criterio 3).
- **Trazabilidad:** `com.biopet.security.JwtService` (campos `@Value` de `security.jwt.expiration-ms` y `security.jwt.refresh-expiration-ms`), `Backend/src/main/resources/application.yml`, `.env.example` (JWT_EXPIRATION_MS, JWT_REFRESH_EXPIRATION_MS), prueba `Backend/src/test/java/com/biopet/security/JwtServiceTest.java`, decisión `docs/adr/ADR-003-jwt-redis.md`; requisito funcional relacionado REQ-F-003.
- **Estado:** verificado

REQ-NF-004 — Claims estándar del JWT

- **Tipo:** Seguridad
- **Prioridad:** Must
- **Enunciado:** Cada JWT emitido por el sistema deberá incluir los siete claims estándar `iss`, `sub`, `aud`, `exp`, `nbf`, `iat` y `jti` conforme al RFC 7519, con `iss` y `aud` configurables por variable de entorno.
- **Rationale:** refinamiento de RNF-03 exigido por el bloque A.1 de la Guía.
- **Criterios de aceptación:**
 1. Un access token emitido contiene los siete claims enumerados.
 2. Un refresh token emitido contiene los siete claims enumerados.
 3. El valor de `nbf` es igual al de `iat`.
 4. El valor de `jti` es distinto entre dos tokens emitidos consecutivamente.
 5. Un token cuyo `iss` no coincide con el configurado es rechazado; lo mismo para `aud`.
 6. Access token y refresh token se distinguen entre sí de forma inequívoca.
- **Método de verificación:** pruebas automatizadas de `JwtServiceTest: accessTokenContieneLosSieteClaims` (criterio 1), `refreshTokenContieneLosSieteClaims` (criterio 2), `nbfEsIgualAIat` (criterio 3), `jtiEsUnicoEntreTokens` (criterio 4), `issuerIncorrectoEsRechazado` y `audienceIncorrectaEsRechazada` (criterio 5), `accessYRefreshSeDistinguenCorrectamente` (criterio 6).
- **Trazabilidad:** `com.biopet.security.JwtService` (método `buildToken`), configuración `security.jwt.issuer: ${JWT_ISSUER:biopet-api}` y `security.jwt.audience: ${JWT_AUDIENCE:biopet-frontend}` en `application.yml`; prueba `Backend/src/test/java/com/biopet/security/JwtServiceTest.java`; norma RFC 7519; decisión `docs/adr/ADR-003-jwt-redis.md`.
- **Estado:** verificado

REQ-NF-005 — Interfaz responsiva

- **Tipo:** Usabilidad
- **Prioridad:** Must
- **Enunciado:** La interfaz deberá adaptarse correctamente a resoluciones comprendidas entre 320 px y 1440 px de ancho (móvil, tablet y escritorio), sin desbordamiento horizontal del contenido.
- **Rationale:** heredado de RNF-04/RNF-WEB-01 de la Entrega 1A.
- **Criterios de aceptación:**
 1. El documento servido declara `<meta name="viewport" content="width=device-width, ...">`.
 2. La hoja de estilos publicada contiene reglas `@media` que adaptan el diseño a anchos móviles.

3. En corridas Lighthouse con perfil móvil y con perfil de escritorio sobre las rutas `/login` y `/mascotas`, la puntuación de Performance es ≥ 90 en todos los casos.
 4. La hoja de estilos publicada define al menos un punto de corte para anchos móviles y el diseño de las rejillas de contenido usa unidades relativas, de modo que a 320 px el contenido refluye en una sola columna en lugar de desbordarse.
- **Método de verificación:** pruebas automatizadas de `FrontendResponsivenessTest` (7 pruebas): `indexHtmlContieneViewportMetaTag` (criterio 1), `stylesCssContieneMediaQueriesResponsive`, `buildIncluyeArchivoStylesCss`, `indexHtmlReferenciaStylesCss` (criterio 2), más `frontendBuildExisteIndexHtml`, `indexHtmlContieneDoctypeYHtmlValido` e `indexHtmlIncluyeAngularBootstrap`. Criterio 3: 12 corridas Lighthouse del 2026-08-18 (`docs/mediciones/lighthouse/lhci-20260818-0538-*.json`: perfiles móvil y desktop, rutas `/login` y `/mascotas`, 3 corridas cada combinación) con Performance 100 en desktop y 90–94 en móvil, con el perfil móvil simulando Slow 4G + CPU 4x (`lighthouseerc.js`). Criterio 4: inspección de `frontend/src/styles.css`, que declara `@media (max-width: 600px)` (línea 45), y de la regla `.grid-mascotas` de `frontend/src/app/features/mascotas.component.ts`.
 - **Trazabilidad:** `frontend/src/index.html` (meta viewport), `frontend/src/styles.css` y `frontend/src/app/features/mascotas.component.ts` (`.grid-mascotas` con `@media (max-width: 600px)`); prueba `Backend/src/test/java/com/biopet/FrontendResponsivenessTest.java`; configuración de medición `lighthouseerc.js` y `lighthouseerc.desktop.js`, guion `scripts/run-lighthouse.sh`; evidencias `docs/mediciones/lighthouse/` con sus sumas `SHA256SUMS.txt`.
 - **Estado:** verificado

REQ-NF-006 — Compatibilidad de navegadores

- **Tipo:** Compatibilidad
- **Prioridad:** Should
- **Enunciado:** La aplicación deberá ejecutar correctamente los flujos críticos (inicio de sesión, listado y gestión de mascotas, gestión de vacunas) en las versiones estables actuales de Google Chrome, Mozilla Firefox y Microsoft Edge.
- **Rationale:** heredado de RNF-05/RNF-WEB-05 de la Entrega 1A.
- **Criterios de aceptación:**
 1. En cada uno de los tres navegadores, el inicio de sesión completa correctamente y la sesión persiste mediante las cookies `HttpOnly`.
 2. En cada uno de los tres navegadores, el listado, la creación, la edición y la baja de una mascota se completan sin errores de consola.
 3. En cada uno de los tres navegadores, la pantalla de vacunas carga y opera sin errores de consola.
 4. La ejecución queda archivada con navegador, versión, fecha y resultado por flujo, de modo que un tercero pueda repetirla y comparar.
- **Método de verificación:** previsto — ejecución manual exploratoria de los tres flujos críticos en Chrome, Firefox y Edge, con registro en `docs/mediciones/` del navegador, la versión, la fecha y el resultado por flujo (criterios 1 a 4). **No ejecutada todavía.**
- **Trazabilidad:** aplicación Angular en `frontend/` (build de producción reproducible con `npm ci && npm run build`, objetivo `frontend` del `Makefile`); flujos críticos en `frontend/src/app/features/login.component.ts`, `mascotas.component.ts` y `vacunas.component.ts`; el proyecto no declara un archivo `browserslist` propio, por lo que aplica el objetivo por defecto de Angular 17.
- **Estado:** pendiente
- **Observaciones:** **ninguno** de los cuatro criterios de aceptación tiene artefacto: no existen capturas, ni matriz de resultados por navegador, ni corrida automatizada, ni registro de versiones probadas. El repositorio tampoco contiene ninguna decisión ni configuración específica de compatibilidad entre navegadores: no hay archivo `browserslist` propio en `frontend/`, por lo que aplica el objetivo por defecto de Angular 17. Que la aplicación compile y se despliegue no constituye implementación de **este** requisito, cuyo objeto es la ejecución comprobada de los flujos críticos en Chrome, Firefox y Edge. Se conserva el estado **pendiente** que ya declaraba `docs/trazabilidad/matriz.csv` antes de esta revisión; una versión intermedia de esta misma revisión lo había subido a **implementado**, lo cual no estaba justificado y se corrige aquí. Sigue como limitación declarada en la sección 7.3.

REQ-NF-007 — Verificación de estado y medición de disponibilidad

- **Tipo:** Disponibilidad
- **Prioridad:** Must
- **Enunciado:** El sistema deberá exponer un endpoint de verificación de estado en `/actuator/health`, accesible sin autenticación, que responda 200 OK con `status: "UP"` cuando la aplicación esté operativa; y la disponibilidad del sistema durante una ventana de evaluación declarada deberá medirse como el porcentaje de sondeos exitosos a ese endpoint, con un objetivo de al menos 99 % y registro fechado de cada sondeo.
- **Rationale:** heredado de RNF-06 de la Entrega 1A. El enunciado anterior —“el sistema deberá estar operativo durante las semanas de evaluación establecidas por la asignatura”— **no era objetivamente verificable**: no define qué se mide, sobre qué ventana, con qué umbral ni con qué artefacto. En esta revisión se reformula en dos partes medibles (endpoint observable y porcentaje de sondeos exitosos), y el compromiso académico de mantener el sistema operativo durante las semanas de evaluación se traslada a la sección 2.5 como **restricción operacional**, donde no se le exige un estado de cumplimiento. No se declara ninguna disponibilidad histórica, porque nunca se midió.
- **Criterios de aceptación:**
 1. GET `/actuator/health` responde 200 OK sin necesidad de autenticación.
 2. El cuerpo de la respuesta es JSON con `Content-Type: application/json` (o el tipo específico de Actuator) e incluye el campo `status` con valor `"UP"`.
 3. La respuesta del endpoint de salud incluye las mismas cabeceras de seguridad que el resto de respuestas públicas (REQ-NF-002).
 4. Para una ventana de evaluación declarada (fecha de inicio y fecha de fin), existe un registro fechado de sondeos periódicos a `/actuator/health` y el porcentaje de sondeos con respuesta 200 OK y `status: "UP"` es $\geq 99\%$.
- **Método de verificación:** pruebas automatizadas de `HealthCheckTest` (5 pruebas): `healthEndpointRespondeUp`, `healthEndpointRespuestaValidaSinAutenticacion` (criterio 1), `healthEndpointEstructuraValida`, `healthEndpointContentTypeCorrecto` (criterio 2), `healthEndpointIncluyeCabecerasSeguridad` (criterio 3). Criterio 4: **verificación prevista** — registro de sondeos periódicos (monitoreo externo o guion programado) archivado en `docs/mediciones/`, con fecha de inicio y fin de la ventana y el cómputo del porcentaje. **No existe hoy.**
- **Trazabilidad:** configuración `management.endpoints.web.exposure.include: health,info,metrics` en `Backend/src/main/resources/application.yml`; lista `permitAll` de `SecurityConfig` (entrada `/actuator/health`); prueba `Backend/src/test/java/com/biopet/HealthCheckTest.java`; `healthcheck` del despliegue definido en `render.yaml` y documentado en `docs/despliegue/DEPLOYMENT.md` y `docs/despliegue/RUNBOOK.md`; objetivo `up` del `Makefile` y `docker-compose.yml`; restricción operacional correspondiente en la sección 2.5.
- **Estado:** implementado
- **Observaciones:** el requisito **no está cumplido**. **Qué cambió respecto del enunciado anterior y por qué.** El enunciado que traía este requisito era “*El sistema deberá estar operativo durante las semanas de evaluación establecidas por la asignatura*”, con estado “*pendiente de confirmación explícita*” en el SRS e “*implementado*” en `docs/trazabilidad/matriz.csv`. Ese enunciado no es verificable: no dice qué se mide, sobre qué ventana, con qué umbral ni con qué artefacto, de modo que ningún resultado podría contradecirlo. La reformulación **no rebaja la intención original**: conserva la disponibilidad como obligación medible en el criterio 4 —que es precisamente la parte que **no** está cumplida— y añade delante la parte observable del mecanismo (criterios 1 a 3). El compromiso académico de mantener el sistema operativo durante las semanas de evaluación se conserva íntegro en la sección 2.5 como restricción operacional, donde no se le atribuye un estado de cumplimiento que nadie ha medido. **Evidencia por criterio.** Criterios 1 a 3: verificados por `HealthCheckTest` (5 pruebas) más la configuración `management.endpoints.web.exposure.include: health,info,metrics` y la entrada `/actuator/health` en la lista `permitAll` de `SecurityConfig`, y el `healthCheckPath: /actuator/health` de `render.yaml`. Criterio 4: **sin ningún artefacto** — no existe monitoreo continuo, ni log de uptime, ni captura fechada de sondeos sostenidos durante una ventana de evaluación. Por eso el requisito queda **implementado** y **no verificado**, y por eso no se afirma ninguna cifra de disponibilidad histórica. Es uno de los requisitos **Must** que no alcanzan **verificado** (ver sección 3.4.2).

REQ-NF-008 — Cifrado de contraseñas

- **Tipo:** Seguridad
- **Prioridad:** Must
- **Enunciado:** Las contraseñas de los usuarios deberán almacenarse siempre cifradas mediante el algoritmo de hash adaptativo BCrypt con factor de coste 12, nunca en texto plano, y no deberán devolverse en ninguna respuesta de la API.
- **Rationale:** heredado de RNF-07 de la Entrega 1A; reforzado por el control OWASP A02 del bloque C.2 de la Guía.
- **Criterios de aceptación:**
 1. El `PasswordEncoder` configurado es `BCryptPasswordEncoder` con factor de coste 12.
 2. Toda contraseña persistida se almacena como hash BCrypt (prefijo `$2b$12$`), incluida la del usuario administrador inicial.
 3. Un inicio de sesión con la contraseña correcta tiene éxito y con una incorrecta falla, lo que demuestra que el hash almacenado se verifica y no se compara texto plano.
 4. Ningún DTO de respuesta de la API expone la contraseña ni su hash.
- **Método de verificación:** inspección de `SecurityConfig.passwordEncoder()` (new `BCryptPasswordEncoder(12)`, criterio 1); inspección de `db/seed.sql`, donde el hash del usuario admin tiene prefijo `$2b$12$`, y de `AuthService.registrar / UsuarioService.crear`, que invocan `passwordEncoder.encode(...)` (criterio 2); pruebas automatizadas `AuthControllerTest.loginExitoso` y `loginClaveIncorrecta` (criterio 3); inspección de los records `UsuarioResponse`, `AuthResponse` y `AuthSessionResponse`, ninguno de los cuales declara un campo de contraseña (criterio 4).
- **Trazabilidad:** `com.biopet.config.SecurityConfig` (bean `passwordEncoder`), `com.biopet.service.AuthService`, `com.biopet.service.UsuarioService`, entidad `com.biopet.entity.Usuario` (campo `passwordHash`), `db/seed.sql`, `Backend/src/main/resources/db/migration/V1_ _schema_inicial.sql`; evidencia `docs/mediciones/sec/A02-cryptography-tls.md`; política de contraseñas asociada en REQ-NF-016.
- **Estado:** verificado

REQ-NF-009 — Registro de eventos de autenticación (OWASP A09)

- **Tipo:** Seguridad
- **Prioridad:** Must
- **Enunciado:** El sistema deberá registrar cada evento de autenticación (login exitoso, login fallido, bloqueo por límite de intentos, refresh exitoso, refresh fallido, logout y uso de token revocado) con dirección IP, marca de tiempo y sujeto, sin exceder 200 caracteres por campo registrado y sin incluir credenciales ni tokens.
- **Rationale:** requisito derivado del control OWASP A09, exigido por el bloque C.2 de la Guía; ausente en el SRS original de la Entrega 1A.
- **Criterios de aceptación:**
 1. Cada uno de los siete eventos enumerados produce una línea de log con prefijo `AUTH_AUDIT`, el tipo de evento, la IP, la marca de tiempo UTC y el sujeto.
 2. Los eventos de fallo o bloqueo se registran con nivel `WARN` y los de éxito con nivel `INFO`.
 3. Un valor nulo de IP o de sujeto se normaliza a `unknown`, nunca produce una línea incompleta.
 4. Los caracteres de control se eliminan del sujeto antes de registrarlo, de modo que no se puedan inyectar saltos de línea ni líneas falsas (*log forging*).
 5. Ninguna línea contiene contraseñas, hashes ni tokens.
 6. Ningún campo registrado supera 200 caracteres.
- **Método de verificación:** pruebas automatizadas de `AuthenticationAuditServiceTest` (10 pruebas): `loginExitosoRegistraEventoEstructurado`, `loginFallidoRegistraEventoWarn`, `loginBloqueadoRegistraEventoWarn`, `refreshExitosoRegistraEventoInfo`, `refreshFallidoRegistraEventoWarn`, `logoutExitosoRegistraEventoInfo`, `tokenRevocadoRegistraEventoWarn` (criterios 1 y 2); valores `NulosSeNormalizanComoUnknown` (criterio 3); `eliminaCaracteresDeControlParaEvitarLogForging` (criterio 4); `noRegistraDatosSensibles` (criterio 5); inspección de `AuthenticationAuditService`, que declara `LONGITUD_MAXIMA = 200` y trunca cada campo con `substring(0, LONGITUD_MAXIMA)` (criterio

6). Evidencia de log real en docs/mediciones/sec/A09-logging.md y docs/mediciones/sec/raw/A09-audit-logs.txt, con líneas AUTH_AUDIT event=LOGIN_SUCCESS, LOGIN_FAILURE y LOGIN_RATE_LIMITED con IP, timestamp UTC y sujeto reales.

- **Trazabilidad:** com.biopet.security.AuthenticationAuditService (invocado desde AuthService y JwtAuthenticationFilter); prueba Backend/src/test/java/com/biopet/security/AuthenticationAuditServiceTest.java; evidencia docs/mediciones/sec/A09-logging.md con docs/mediciones/sec/raw/A09-audit-logs.txt; control OWASP A09; requisito funcional hermano REQ-F-020.
- **Estado:** verificado
- **Observaciones:** no existe integración con un SIEM centralizado: los eventos quedan en el log de la aplicación. La auditoría de operaciones CRUD de negocio (distinta de la de autenticación) corresponde a REQ-F-020, que está implementado precisamente por esa carencia.

REQ-NF-010 — Limitación de intentos de login (OWASP A07)

- **Tipo:** Seguridad
- **Prioridad:** Must
- **Enunciado:** El sistema deberá bloquear temporalmente los intentos de inicio de sesión provenientes de una misma dirección IP tras 6 intentos fallidos consecutivos dentro de una ventana de 15 minutos, respondiendo 429 Too Many Requests con cabecera Retry-After durante el bloqueo, con los tres parámetros configurables mediante propiedades externas.
- **Rationale:** requisito derivado del control OWASP A07, exigido por el bloque C.2 de la Guía; ausente en el SRS original de la Entrega 1A.
- **Criterios de aceptación:**
 1. Los cinco primeros intentos fallidos desde una misma IP responden 401.
 2. El sexto intento fallido responde 429 con ProblemDetail de tipo urn:biopet:error:rate-limited y cabecera Retry-After: 900.
 3. Durante el bloqueo, incluso un intento con credenciales **correctas** desde esa IP es rechazado con 429.
 4. Los contadores son independientes por IP: los fallos desde IP distintas no se acumulan entre sí.
 5. Un inicio de sesión exitoso reinicia el contador de fallos de esa IP.
 6. Al expirar la ventana sin alcanzar el umbral, el contador se reinicia; al expirar el bloqueo, se admite un nuevo intento.
 7. Los parámetros de umbral, ventana y duración de bloqueo se leen de propiedades externas (security.rate-limit.login.max-attempts, .window, .block-duration), no de literales en el código.
- **Método de verificación:** pruebas automatizadas de LoginRateLimiterServiceTest (7 pruebas): cincoFallosNoBloquean (criterio 1), sextoFalloBloqueaYLanzaExcepcion (criterio 2), ipBloqueadaSigueRechazadaConTiempoRestante (criterio 3), ipsDiferentesMantienenContadoresSeparados (criterio 4), reiniciarEliminaFallosYBloqueo (criterio 5), ventanaExpiradaReiniciaElContador y bloqueoExpiradoPermiteNuevoIntento (criterio 6); y de AuthControllerTest: quintoIntentoFallidoSigueRespondiendo401, sextoIntentoFallidoDevuelve429ProblemDetail, ipBloqueadaRechazaInclusoCredencialesCorrectas, intentosDesdeIpsDistintasNoSeAcumulan, loginExitosoReiniciaContadorDeLaIp. Criterio 7: inspección de las anotaciones @Value del constructor de LoginRateLimiterService. Evidencia HTTP real (curl) en docs/mediciones/sec/A07-authentication.md con docs/mediciones/sec/raw/A07-auth-rate-limit.txt: cinco 401, un 429 con type=urn:biopet:error:rate-limited y Retry-After: 900.
- **Trazabilidad:** com.biopet.security.LoginRateLimiterService (invocado desde AuthService.login), excepción RateLimitExcedidoException → GlobalExceptionHandler.demasiadosIntentos → ProblemType.RATE_LIMITED; pruebas LoginRateLimiterServiceTest y AuthControllerTest; evidencia docs/mediciones/sec/A07-authentication.md; control OWASP A07.
- **Estado:** verificado
- **Observaciones:** el contador de intentos se mantiene en memoria del proceso (ConcurrentHashMap en LoginRateLimiterService), no en Redis. En un despliegue con más de una instancia el límite se

aplicaría por instancia, no de forma global; el despliegue actual es de instancia única. Como efecto secundario positivo, este mecanismo **no** depende de Redis y por tanto no se ve afectado por el escenario descrito en REQ-NF-014.

REQ-NF-011 — Arquitectura en capas

- **Tipo:** Mantenibilidad
- **Prioridad:** Must
- **Enunciado:** La arquitectura del backend deberá organizarse en capas separadas de presentación (**controller**), lógica de negocio (**service**) y acceso a datos (**repository/entity**), sin que la capa de presentación acceda directamente a los repositorios.
- **Rationale:** heredado de RNF-08 de la Entrega 1A; decisión de acceso a datos registrada en docs/adr/ADR-007-acceso-datos.md.
- **Criterios de aceptación:**
 1. Existen los paquetes `com.biopet.controller`, `com.biopet.service`, `com.biopet.repository`, `com.biopet.entity` y `com.biopet.dto`, cada uno con las clases que le corresponden.
 2. Ninguna clase de `com.biopet.controller` declara una dependencia de un tipo del paquete `com.biopet.repository`: los seis controladores inyectan únicamente servicios (`AuthService`, `UsuarioService`, `MascotaService`, `CitaService`, `ConsultaService`, `VacunaService`, `ExternalApiService`).
 3. Las entidades JPA no se exponen en las firmas públicas de los controladores: las respuestas se construyen con los records de `com.biopet.dto`.
 4. La estructura de capas está reflejada en el diagrama de componentes C4 de nivel 3 del repositorio.
- **Método de verificación:** inspección de la estructura de paquetes de `Backend/src/main/java/com/biopet/` (criterio 1); inspección de los constructores de los seis controladores de `com.biopet.controller` (criterio 2); inspección de las firmas de los métodos anotados con `@GetMapping/@PostMapping/@PutMapping/@DeleteMapping`, que devuelven `MascotaResponse`, `CitaResponse`, `ConsultaResponse`, `VacunaResponse`, `UsuarioResponse`, `ExternalApiResponse`, `ResumenEspecieResponse` o `AuthSessionResponse` (criterio 3); diagrama docs/diagrams/c4-componentes-backend/C4-L3-backend.md y su render `c4-componentes-backend.png` (criterio 4).
- **Trazabilidad:** paquetes `com.biopet.{controller,service,repository,entity,dto}` en `Backend/src/main/java/com/biopet/`; diagramas C4 en docs/diagrams/c4-componentes-backend/ y docs/diagrams/c4-contenedores/; decisiones docs/adr/ADR-002-pila-tecnologica.md y docs/adr/ADR-007-acceso-datos.md; diagrama de clases docs/diagrams/diagrama-clases/diagrama-clases.png.
- **Estado:** verificado
- **Observaciones:** no existe un control automatizado de dependencias entre capas (por ejemplo, una regla de ArchUnit) que impida introducir en el futuro una dependencia de controlador a repositorio; hoy la separación se comprueba por inspección. Registrado como mejora, no como defecto.

REQ-NF-012 — Caché del listado de mascotas: TTL configurable y medición de aciertos

Requisito con dos obligaciones. Agrupa (**A**) que el listado se almacene en caché con un TTL configurable por variable de entorno y (**B**) que la tasa de aciertos de esa caché se mida y se reporte empíricamente. La primera está cumplida y la segunda no, por lo que en revisiones anteriores el requisito arrastraba el valor inválido “verificado parcialmente”. Conserva su identificador único; las obligaciones se separan con criterios A1...A4 y B1...B4, y el Estado describe el requisito completo.

- **Tipo:** Rendimiento
- **Prioridad:** Should
- **Enunciado:** El listado paginado de mascotas deberá (**A**) almacenarse en caché Redis con un tiempo de vida (TTL) configurable mediante variable de entorno (`CACHE_TTL_MS`, valor por defecto 300 000 ms), sin valores fijos en el código y con una entrada de caché distinta por usuario y por parámetros de paginación; y (**B**) su tasa de aciertos (*hit ratio*), definida como `keyspace_hits / (keyspace_hits +`

`keyspace_misses`), deberá medirse y reportarse empíricamente sobre una corrida de carga declarada, con un objetivo de al menos 80 % en régimen de caché caliente.

- **Rationale:** heredado de la estrategia de caché definida en `docs/adr/ADR-003-jwt-redis.md`; llevado a requisito no funcional explícito por el bloque A.1 de la Guía, que pide evidencia empírica de la estrategia de caché y no sólo de su configuración. La obligación A es la condición que hace alcanzables los umbrales de REQ-NF-001 con caché caliente.
- **Criterios de aceptación:**
 1. (A1) El TTL de la caché se lee de la variable de entorno `CACHE_TTL_MS`, con valor por defecto 300 000 ms, y no aparece como literal en ninguna clase Java.
 2. (A2) Bajo carga sobre `GET /api/mascotas`, existe al menos una clave `mascotas::*` en Redis y el comando TTL sobre ella devuelve un valor positivo y coherente con el configurado.
 3. (A3) La clave de caché incluye el correo del usuario autenticado y los parámetros de paginación, de modo que dos dueños distintos que piden la misma página no comparten entrada.
 4. (A4) Toda operación de escritura sobre `mascotas` invalida la caché completa del espacio `mascotas`.
 5. (B1) Existe un archivo de evidencia con la salida de `redis-cli INFO stats` (o del endpoint `/actuador/metrics` equivalente) tomada **antes** y **después** de una corrida de carga declarada sobre `GET /api/mascotas`.
 6. (B2) A partir de esos dos puntos se calcula el *hit ratio* de la corrida y el valor obtenido queda registrado junto con el número de peticiones, la duración y la fecha de la corrida.
 7. (B3) El *hit ratio* medido en régimen de caché caliente es $\geq 80\%$.
 8. (B4) La medición es reproducible: el guion o los comandos exactos que la generan están versionados en el repositorio.
- **Método de verificación:** obligación A — inspección de `Backend/src/main/resources/application.yml` (`spring.cache.redis.time-to-live: ${CACHE_TTL_MS:300000}`, `cache-null-values: false`) y de `.env.example` (`CACHE_TTL_MS=300000`) (A1); evidencia cruda archivada en `docs/mediciones/redis/: ttl-cache-mascotas.txt` (TTL real observado de 195 s sobre la clave de caché), `keys-mascotas.txt` (`KEYS mascotas::*`), `dbsize-durante-carga.txt` (DBSIZE bajo carga) y `config-maxmemory.txt` (A2); prueba automatizada `MascotaControllerTest.dosDuenosConMismaPaginaNoComparenResultadosDeCache` e inspección de la anotación `@Cacheable` de `MascotaService.listar`, cuya clave se compone del correo del usuario autenticado y de los parámetros de paginación (A3); inspección de las tres anotaciones `@CacheEvict(value = "mascotas", allEntries = true)` sobre `crear`, `actualizar` y `eliminar` de `MascotaService` (A4). Obligación B — **verificación prevista, no ejecutada:** captura de `redis-cli INFO stats | grep keyspace_` antes y después de una corrida k6 equivalente a las de REQ-NF-001, archivada en `docs/mediciones/redis/` con el cálculo del cociente; alternativamente, las métricas de caché de Micrometer expuestas por `/actuador/metrics` (`cache.gets{result=hit}` y `cache.gets{result=miss}`), vía ya habilitada en `management.endpoints.web.exposure.include`.
- **Trazabilidad:** `com.biopet.service.MascotaService` (anotaciones `@Cacheable/@CacheEvict`), `Backend/src/main/resources/application.yml` (`spring.cache.type: redis`, `spring.cache.redis.time-to-live`, `management.endpoints.web.exposure.include`), `.env.example` (`CACHE_TTL_MS`), decisión `docs/adr/ADR-003-jwt-redis.md`, evidencias `docs/mediciones/redis/` (TTL, DBSIZE, KEYS, maxmemory — que documentan la **configuración y la existencia** de la clave, no su tasa de aciertos), guiones de carga en k6/ y reporte `docs/mediciones/perf/REPORT.md`, prueba `Backend/src/test/java/com/biopet/MascotaControllerTest.java`; requisito de rendimiento dependiente REQ-NF-001; comportamiento ante fallo de Redis, REQ-NF-014.
- **Estado:** implementado
- **Observaciones:** el requisito **no está cumplido**. La obligación A está realizada y evidenciada en sus cuatro criterios. La obligación B **no tiene ninguna medición**: no existe en el repositorio ningún registro de `keyspace_hits / keyspace_misses`, por lo que B1 a B4 carecen de artefacto. Una versión anterior de la matriz de trazabilidad afirmaba un “hit ratio cercano al 100 % documentado en `docs/mediciones/perf/REPORT.md`”; esa afirmación **no es cierta** —ese reporte no contiene tal medición— y fue retirada en la revisión del 2026-09-07. No se declara aquí ninguna cifra de *hit ratio*.

REQ-NF-013 — Estrategia híbrida de acceso a datos (ORM + procedimientos almacenados)

- **Tipo:** Mantenibilidad / Seguridad
- **Prioridad:** Must
- **Enunciado:** Toda operación de base de datos que no sea un CRUD elemental sobre una única tabla (según la definición del bloque A.2.1 de la Guía) deberá implementarse como función o procedimiento almacenado versionado en `db/procs/` e invocado desde un repositorio Spring Data mediante el mecanismo formal `@Procedure`, quedando prohibida la concatenación de entrada de usuario en cualquier fragmento JPQL, HQL o SQL nativo.
- **Rationale:** requisito exigido explícitamente por el bloque A.2 de la Guía. Una versión anterior de este documento lo marcaba “cumplida por alcance actual” porque entonces no existía ninguna operación no elemental; desde v0.9.0-rc existen seis rutinas reales.
- **Criterios de aceptación:**
 1. Las seis rutinas de `db/procs/` son objetos PostgreSQL PROCEDURE versionados en el repositorio y aplicados por migraciones Flyway.
 2. Las seis se invocan desde Java exclusivamente con `@Procedure` (no con `@Query(nativeQuery = true)` ni JDBC directo desde los servicios).
 3. El análisis `scripts/audit-sql-dynamic.sh` sobre los archivos de `db/procs/` devuelve 0 hallazgos de SQL dinámico construido por concatenación, con código de salida 0.
 4. Cada rutina tiene al menos una prueba de integración que la ejercita contra un PostgreSQL real.
 5. El catálogo documental de rutinas está completo: cada fila es trazable a un archivo `.sql` real del repositorio.
- **Método de verificación:** inspección de `db/procs/` y de las migraciones `Backend/src/main/resources/db/migration/V5__procedimientos_biopet.sql` y `V6__formalizar_procedimientos_jpa.sql` (criterio 1); inspección de `com.biopet.repository.ProcedimientoBiopetRepository`, cuyos seis métodos están anotados con `@Procedure` (criterio 2); ejecución archivada de `scripts/audit-sql-dynamic.sh` con resultado `audit-sql-dynamic: 0 hallazgos en 7 archivo(s)`. PASA (código de salida 0), evidencia en `docs/mediciones/sec/audit-sql-dynamic-v1.0.0.txt` con su suma `audit-sql-dynamic-v1.0.0.sha256` (criterio 3); pruebas de integración `ProcedimientosBiopetIntegrationTest` (13 pruebas sobre las seis rutinas) y `ResumenEspeciesIntegrationTest`, ambas con Testcontainers sobre PostgreSQL real (criterio 4); `docs/basedatos/CATALOGO-SP.md` (criterio 5).
- **Trazabilidad:** rutinas `db/procs/fn_resumen_mascotas_por_especie.sql`, `fn_historial_clinico_mascota.sql`, `fn_reporte_dashboard.sql`, `fn_siguiente_numero_ficha.sql`, `sp_actualizar_estado_citas_masivas.sql`, `sp_registrar_consulta_validada.sql` y los permisos `zz_grants_biopet_app.sql`; invocación formal en `com.biopet.repository.ProcedimientoBiopetRepository` con los `@NamedStoredProcedureQuery` declarados en `com.biopet.entity.Mascota`; proyecciones `ResumenEspecie`, `HistorialClinico`, `ReporteDashboard`; migraciones `V5__procedimientos_biopet.sql` y `V6__formalizar_procedimientos_jpa.sql`; pruebas `Backend/src/test/java/com/biopet/repository/ProcedimientosBiopetIntegrationTest.java`, `ResumenEspeciesIntegrationTest.java`, `BiopetAppRolMinimoPrivilegiosIntegrationTest.java` y `Backend/src/test/java/com/biopet/SqlInjectionSecurityTest.java`; guion `scripts/audit-sql-dynamic.sh` (objetivo `sql-audit` del Makefile y `job sql-audit` del CI); catálogo `docs/basedatos/CATALOGO-SP.md`; decisión `docs/adr/ADR-007-acceso-datos.md`; requisitos que consumen estas rutinas: REQ-F-021, REQ-F-013, REQ-F-015, REQ-F-018, REQ-F-019.
- **Estado:** verificado
- **Observaciones:** la reproducción de `scripts/audit-sql-dynamic.sh` se generó después del tag v1.0.0, sobre el commit exacto de ese tag (`0d5cd525ce648cca7219da204e16fa622e671a87`), en un worktree aislado que no modificó el árbol de trabajo principal: el código auditado es el del tag; el log es evidencia posterior, no un artefacto que existiera dentro de ese commit. La rutina `fn_siguiente_numero_ficha` existe, está probada (`siguienteNumeroFicha_formatoYSecuencia`, `siguienteNumeroFicha_prefijoVacioUsaDefault`) y cumple la categoría “(6) Código secuencial” de la rúbrica, pero **ningún servicio de la aplicación la invoca todavía**: su consumidor natural es REQ-F-018 (numeración correlativa de comprobantes), que está pendiente. Se declara aquí para no presentar como funcionalidad de negocio lo que hoy es sólo un objeto de base de datos disponible.

REQ-NF-014 — Comportamiento del sistema ante indisponibilidad de Redis

- **Tipo:** Seguridad / Disponibilidad
- **Prioridad:** Must
- **Enunciado:** Cuando el servicio Redis no esté disponible, el sistema deberá aplicar una política explícita y documentada de degradación: las verificaciones de revocación de token deberán **fallar de forma cerrada** (*fail-closed*), rechazando la solicitud con **503 Service Unavailable** y cuerpo **ProblemDetail** en lugar de conceder acceso sin comprobar la lista negra; y las operaciones cuya única dependencia de Redis sea la caché de lectura deberán **degradarse a consulta directa a la base de datos** (*fail-open* limitado a la caché), sin exponer trazas internas al cliente.
- **Rationale:** requisito nuevo incorporado en esta revisión. La retroalimentación del docente-director señaló que el SRS no especificaba qué debe ocurrir si Redis cae, pese a que Redis sostiene tres mecanismos distintos: la lista negra de tokens (REQ-F-005), la caché de listados (REQ-NF-012) y la caché de la API externa (REQ-F-025). La revisión del código confirma que **hoy no existe ninguna política**: ni *fail-open* ni *fail-closed* deliberados, sino un fallo no controlado (ver Observaciones). Una caída de Redis no debe poder convertirse en una omisión silenciosa del control de revocación de tokens.
- **Criterios de aceptación:**
 1. Dado Redis no disponible y una solicitud a un recurso protegido con un access token, cuando `TokenBlacklistService.isRevoked` no puede consultarse, entonces la respuesta es **503 Service Unavailable** con `Content-Type: application/problem+json`, y **no** se establece autenticación en el contexto de seguridad.
 2. Dado Redis no disponible, la respuesta del criterio 1 no contiene traza de pila, nombre de clase ni mensaje de la excepción de conexión.
 3. Dado Redis no disponible y una solicitud a `POST /api/auth/logout`, entonces la respuesta es **503** con **ProblemDetail**, sin error 500 sin controlar.
 4. Dado Redis no disponible y una solicitud a `GET /api/mascotas` de un usuario ya autenticado por otra vía, entonces la operación se resuelve consultando directamente PostgreSQL y responde **200 OK**: el fallo de la caché no impide la lectura.
 5. Cada caída y cada recuperación de la conexión con Redis queda registrada en el log del sistema con marca de tiempo, siguiendo el formato de REQ-NF-009.
 6. Existe una prueba automatizada que fuerza el fallo de conexión con Redis y comprueba los criterios 1, 3 y 4.
- **Método de verificación:** previsto — prueba de integración que arranque la aplicación con un `String RedisTemplate` que lance `RedisConnectionFailureException` (o con un contenedor Redis detenido) y verifique los códigos de respuesta de los criterios 1, 3 y 4; más un `CacheErrorHandler` registrado en la configuración de caché para el criterio 4 y un manejador dedicado en `GlobalExceptionHandler` (o en `JwtAuthenticationFilter`) para los criterios 1 a 3. **No ejecutable hoy: no hay implementación.**
- **Trazabilidad:** `com.biopet.security.TokenBlacklistService` (métodos `revoke` e `isRevoked`, ambos invocan `StringRedisTemplate` sin captura de error), `com.biopet.security.JwtAuthenticationFilter` (bloque `catch (JwtException | IllegalArgumentException ex)`), `com.biopet.integration.ExternalApiService.guardarEnCache` (única ruta con manejo de error de Redis ya implementado, probada por `ExternalApiServiceTest.fallaAlEscribirEnRedisNoRompeLaRespuesta`), `com.biopet.service.MascotaService` y `ConsultaService` (anotaciones `@Cacheable/@CacheEvict`), configuración `spring.data.redis.*` y `spring.cache.type: redis` en `application.yml`, servicio `redis` de `docker-compose.yml`, decisión `docs/adr/ADR-003-jwt-redis.md`; requisitos afectados REQ-F-005, REQ-F-025, REQ-NF-012.
- **Estado:** pendiente
- **Observaciones:** comportamiento **real** verificado en el código actual, sin suponer nada: `TokenBlacklistService.isRevoked` invoca `redisTemplate.hasKey(...)` sin bloque `try`. Con Redis caído esa llamada lanza `RedisConnectionFailureException`, que es una `RuntimeException` de la jerarquía `Data AccessException` de Spring y **no** es capturada por el `catch (JwtException | IllegalArgumentException ex)` de `JwtAuthenticationFilter.doFilterInternal`. La excepción escapa del filtro antes de llegar al `DispatcherServlet`, por lo que tampoco la atiende `GlobalExceptionHandler` (un `@RestControllerAdvice` no cubre la cadena de filtros): el resultado es un error **500** del contenedor, sin `ProblemDetail`. Lo mismo ocurre en `TokenBlacklistService.revoke` durante el `logout`. Una búsqueda

de `RedisConnectionFailure`, `RedisSystemException`, `DataAccessException` o `CacheErrorHandler` sobre `Backend/src/main/java/` no devuelve ninguna coincidencia: **no hay manejo alguno**. Por tanto no se afirma aquí que el sistema sea *fail-open* ni *fail-closed*: hoy es un fallo no controlado, y este requisito especifica la política que **deberá** implementarse. El estado es **pendiente** y no **implementado** porque no existe ningún código que intente aplicar esta política. La única excepción parcial ya implementada es la escritura en caché de `ExternalApiService.guardarEnCache`, que sí captura el fallo y no rompe la respuesta; se cita en Trazabilidad como patrón de referencia para la implementación futura. **Corregir esto requiere modificar código de backend y queda explícitamente fuera del alcance de esta revisión documental.**

REQ-NF-015 — Aislamiento de datos por propietario (requisito transversal)

- **Tipo:** Seguridad
- **Prioridad:** Must
- **Enunciado:** Para todo recurso del sistema que tenga un propietario —mascotas, citas, consultas médicas, vacunas y cualquier recurso de propietario que se incorpore en el futuro—, el sistema deberá garantizar que un usuario autenticado con rol `ROLE_DUEÑO` acceda exclusivamente a los recursos asociados a su propia cuenta, tanto en las operaciones de listado como en las de consulta por identificador, devolviendo `403 Forbidden` con cuerpo `ProblemDetail` ante cualquier intento de acceso a un recurso de otro propietario y excluyendo del resultado, sin excepción, los elementos ajenos.
- **Rationale:** requisito transversal nuevo incorporado en esta revisión. La regla de aislamiento existía dispersa en los requisitos individuales (REQ-F-009, REQ-F-010, REQ-F-015, REQ-F-024) y en la implementación de cada servicio, pero **no estaba especificada una sola vez de forma transversal**, lo que permitió que un módulo —Consultas— se entregara sin ella sin que ningún requisito lo detectara. Se formaliza aquí para que cualquier módulo nuevo herede la obligación explícitamente y para que el incumplimiento sea trazable a un requisito propio.
- **Criterios de aceptación:**
 1. **Listados.** Para cada uno de `GET /api/mascotas`, `GET /api/citas`, `GET /api/consultas` y `GET /api/vacunas`: dado un usuario `ROLE_DUEÑO A` y existiendo al menos un recurso perteneciente a un dueño `B` distinto, cuando `A` invoca el listado con cualquier combinación de parámetros de paginación, entonces ningún elemento de ninguna página devuelta pertenece a `B`.
 2. **Consulta por identificador.** Para cada uno de `GET /api/mascotas/{id}`, `GET /api/citas/{id}`, `GET /api/consultas/{id}` y `GET /api/vacunas/{id}`: dado el dueño `A` y un recurso de `B`, cuando `A` lo solicita por su identificador, entonces la respuesta es `403` con `ProblemDetail` de tipo `urn:biopet:error:forbidden`, no `200` ni `404`.
 3. **Listado filtrado.** Dado el dueño `A` y una mascota de `B`, cuando `A` invoca `GET /api/vacunas/mascota/{mascotaId}` con el identificador de la mascota de `B`, entonces la respuesta es `403`.
 4. **Agregados.** Dado el dueño `A`, cuando invoca `GET /api/mascotas/resumen-especies` indicando el `duenioId` de `B`, entonces el resultado se calcula sobre `A`: el parámetro ajeno se ignora y nunca se devuelven totales de `B`.
 5. **Caché.** Dos dueños distintos que solicitan la misma página con los mismos parámetros no comparten entrada de caché ni resultados.
 6. **Cobertura.** Cada recurso de propietario del sistema tiene al menos una prueba automatizada de acceso cruzado (dueño `A` intentando acceder a un recurso de `B`) para el listado y para la consulta por identificador.
- **Método de verificación:** pruebas automatizadas de acceso cruzado: `MascotaControllerTest.duenoSoloVeSusPropiasMascotasEnListado`, `duenoConsultaMascotaDeOtroDuenioDevuelve403`, `dosDuenosConMismaPaginaNoComparenResultadosDeCache`; `CitaControllerTest.duenoSoloVeSusPropiasCitasEnListado`, `duenoNoPuedeConsultarCitaDeMascotaAjena`; `VacunaControllerTest.duenoSoloVeVacunasDeSusPropiasMascotas`, `duenoConsultaVacunaDeMascotaAjenaDevuelve403`; `ConsultaControllerTest.duenoDeOtraMascotaNoPuedeVerConsultaAjena`; más la evidencia HTTP

real de acceso cruzado entre dos dueños sobre TLS en docs/mediciones/sec/A01-access-control.md y docs/mediciones/sec/raw/A01-access-control.txt. Criterio 4: inspección de MascotaService.resumenPorEspecie. Criterio 1 para Consultas y criterio 3: **sin artefacto** — ver Observaciones.

- **Trazabilidad:** reglas de propiedad implementadas en MascotaService.verificarPropiedad, CitaService.verificarAccesoLectura, VacunaService.verificarAcceso y ConsultaService.verificarAcceso; métodos de repositorio con filtro por propietario MascotaRepository.findAllByDuenioIdAndActivoTrue, CitaRepository.findAllByMascota_Duenio_IdAndActivoTrue, VacunaRepository.findAllByMascota_Duenio_IdAndActivoTrue (**no existe el equivalente en ConsultaRepository**); matriz de permisos, sección 2.8; evidencia docs/mediciones/sec/A01-access-control.md; control OWASP A01; requisito funcional relacionado REQ-F-006.
- **Estado:** implementado
- **Observaciones:** el requisito **NO** está cumplido. El criterio 1 falla para GET /api/consultas, y el criterio 6 falla para ese mismo endpoint. ConsultaService.listar() devuelve consultaRepository.findAllByActivoTrue(pageable) para todos los roles, incluido ROLE_DUENO, y ConsultaRepository no declara ningún método de filtrado por propietario. En consecuencia, un dueño autenticado puede ver en ese listado las consultas médicas de mascotas ajenas: el aislamiento transversal que exige el enunciado **no se cumple hoy**.

Por qué implementado y no pendiente. Según la sección 1.3.1, *pendiente* significa que *no existe implementación específica de lo que el requisito exige*, y eso sería falso aquí: tres de los cuatro recursos de propietario —mascotas, citas y vacunas— implementan la regla con métodos de repositorio filtrados por dueño y con pruebas de acceso cruzado que pasan (duenoSoloVeSusPropiasMascotasEnListado, duenoSoloVeSusPropiasCitasEnListado, duenoSoloVeVacunasDeSusPropiasMascotas), además de la captura HTTP real de IDOR entre dos dueños. Declararlo *pendiente* ocultaría esa implementación real y describiría mal el defecto, que es **una omisión en un módulo concreto**, no la ausencia del mecanismo. Ambos estados significan igualmente “requisito no cumplido” (sección 1.3.1) y este requisito se contabiliza como **no cumplido** en las secciones 3.4.1 y 3.4.2; **implementado** es el que describe con exactitud la situación.

Los criterios 2, 4 y 5 sí están verificados para los cuatro recursos; el criterio 3 se sostiene por inspección de VacunaService.listarPorMascota (ver REQ-F-024), sin prueba dedicada. **La corrección del criterio 1 requiere modificar código de backend y queda explícitamente fuera del alcance de esta revisión documental**; se reporta como acción de seguimiento en la sección 7.4.

REQ-NF-016 — Política de contraseñas

- **Tipo:** Seguridad
- **Prioridad:** Must
- **Enunciado:** El sistema deberá exigir que toda contraseña establecida por un usuario —en el registro público, en la creación administrativa de una cuenta y en su actualización— tenga una longitud mínima de 8 y máxima de 80 caracteres, rechazando con 422 **Unprocessable Entity** las que no cumplan, y no deberá devolver nunca la contraseña ni su hash en ninguna respuesta de la API.
- **Rationale:** requisito nuevo incorporado en esta revisión. REQ-NF-008 cubría el **almacenamiento** cifrado de la contraseña, pero ninguna especificación recogía la **política de admisión** que el código ya aplica mediante @Size(min = 8, max = 80). Se documenta la política real, sin atribuirle reglas de complejidad, expiración o comprobación contra listas de contraseñas filtradas que el sistema no implementa.
- **Criterios de aceptación:**
 1. Dado un registro público con contraseña de menos de 8 caracteres, cuando se envía, entonces la respuesta es 422 con ProblemDetail de validación y el objeto errors incluye la entrada password.
 2. Dado el mismo caso en la creación administrativa de usuarios (POST /api/usuarios), entonces la respuesta es 422 con la misma estructura.
 3. Dada una creación administrativa **sin** contraseña, entonces la respuesta es 400 con ProblemDetail: la contraseña es obligatoria al crear.

4. Dada una actualización administrativa con contraseña vacía o ausente, entonces la contraseña actual se conserva sin cambios y la operación responde 200 OK.
 5. Ninguna respuesta de la API incluye la contraseña ni el hash.
 6. La contraseña admitida se almacena cifrada según REQ-NF-008.
- **Método de verificación:** pruebas automatizadas `AuthControllerTest.registroConCamposInvalidos`, que envía `"password":"corta"` y `asserta 422 con $.errors.password` como arreglo (criterio 1); `UsuarioControllerTest.crearUsuarioConCamposInvalidosDevuelve422` (criterio 2); `crearUsuarioSinPasswordDevuelve400` (criterio 3); `actualizarUsuarioDevuelve200`, que omite el campo `password` y obtiene 200 OK, más inspección de `UsuarioService.actualizar`, cuya guarda `if (request.password() != null && !request.password().isBlank())` deja intacto `passwordHash` cuando no se envía contraseña (criterio 4). Criterio 5: inspección de `UsuarioResponse`, `AuthResponse` y `AuthSessionResponse`, ninguno de los cuales declara campo de contraseña. Criterio 6: ver REQ-NF-008. **El límite superior de 80 caracteres del criterio 2 no tiene ninguna prueba automatizada** — ver Observaciones.
 - **Trazabilidad:** restricciones declarativas `@NotBlank @Size(min = 8, max = 80)` en `com.biopet.dto.RegistroRequest` y `@Size(min = 8, max = 80)` en `com.biopet.dto.UsuarioRequest` (sin `@NotBlank`, deliberadamente, para permitir la actualización sin cambio de contraseña); `UsuarioService.crear` (validación explícita de contraseña obligatoria al crear); `GlobalExceptionHandler.validacion` → `ProblemType.VALIDATION`; pruebas `AuthControllerTest` y `UsuarioControllerTest`; requisito de almacenamiento REQ-NF-008; requisito dependiente REQ-F-026.
 - **Estado:** implementado
 - **Observaciones:** el requisito **no está cumplido** por falta de evidencia, no por defecto: la política está realmente implementada, pero una de las reglas que exige el enunciado no está probada. **Cobertura real, regla por regla:** longitud mínima de 8 en el registro público → `AuthControllerTest.registroConCamposInvalidos` (envía `"corta"` y `asserta 422 con $.errors.password`); longitud mínima de 8 en la creación administrativa → `UsuarioControllerTest.crearUsuarioConCamposInvalidosDevuelve422` (mismo valor y misma aserción); contraseña obligatoria al crear → `crearUsuarioSinPasswordDevuelve400`; conservación de la contraseña al actualizar → `actualizarUsuarioDevuelve200` más inspección de `UsuarioService.actualizar`; ausencia de la contraseña en las respuestas → inspección de los tres records de salida; cifrado → REQ-NF-008. **El límite superior de 80 caracteres no está cubierto por ninguna prueba:** sólo consta en la anotación `@Size(min = 8, max = 80)` de `RegistroRequest` y `UsuarioRequest`. No existe ningún caso que envíe una contraseña de más de 80 caracteres y compruebe el 422. Mientras no exista esa prueba, el requisito no puede declararse **verificado**. Por lo demás, la política implementada es **solo de longitud**: el sistema **no** exige complejidad (mayúsculas, dígitos o símbolos), **no** aplica caducidad, **no** impide la reutilización de contraseñas anteriores y **no** comprueba la contraseña contra listas de credenciales filtradas. Se documenta lo que existe, sin declarar conformidad con ninguna guía externa de contraseñas que el proyecto no haya aplicado.

REQ-NF-017 — Respaldo y recuperación de la base de datos

- **Tipo:** Operación
- **Prioridad:** Should
- **Enunciado:** El proyecto deberá mantener documentado un procedimiento de respaldo y restauración de la base de datos PostgreSQL que especifique frecuencia, formato, destino, retención y verificación de integridad, y deberá haber demostrado al menos una restauración real sobre un volcado generado con ese mismo procedimiento, comprobando que los datos y los procedimientos almacenados sobreviven a la restauración.
- **Rationale:** requisito nuevo incorporado en esta revisión. El procedimiento y su prueba de restauración ya existían en el repositorio (`docs/despliegue/BACKUP.md`) pero no estaban recogidos como requisito, de modo que ni el SRS ni la matriz los declaraban. Se documenta lo que existe en **este** repositorio, sin tomar prácticas de ningún otro proyecto.
- **Criterios de aceptación:**
 1. El procedimiento documentado especifica: qué se respalda, frecuencia, formato del volcado, destino

- separado del host de producción, retención y método de verificación por suma de comprobación.
2. Existe evidencia fechada de una restauración real: los comandos exactos ejecutados y el resultado obtenido, sobre un volcado generado con el procedimiento documentado.
 3. Tras la restauración, la comprobación de integridad confirma el número de filas esperado en las tablas del sistema.
 4. Tras la restauración, los procedimientos almacenados siguen siendo invocables: al menos una rutina se ejecuta correctamente sobre la base restaurada.
 5. Se conserva en el repositorio un volcado de ejemplo del mismo formato, para revisión.
- **Método de verificación:** inspección de docs/despliegue/BACKUP.md, secciones 0 y 1 (criterio 1) y sección 4 (criterios 2 a 4): restauración ejecutada el **2026-08-17** sobre PostgreSQL 16 en el contenedor biopet-postgres, con `pg_dump --clean --if-exists --no-owner` (64 762 bytes, 6 tablas), restauración en una base de prueba `biopet_restore_test`, verificación de conteos (`usuarios`: 1 fila; `mascotas`, `citas`, `consultas`, `vacunas`: 0 filas, consistente con la base de origen) e invocación posterior de `fn_siguiete_numero_ficha('RST')` devolviendo RST-000001. Criterio 5: archivo docs/despliegue/ejemplo-backup-20260817.sql presente en el repositorio.
 - **Trazabilidad:** docs/despliegue/BACKUP.md (procedimiento y evidencia), docs/despliegue/ejemplo-backup-20260817.sql (volcado de ejemplo), docs/despliegue/RUNBOOK.md y DEPLOYMENT.md (operación del despliegue), `render.yaml` (servicio de base de datos gestionado), rutinas de `db/procs/` que sobreviven a la restauración (REQ-NF-013), migraciones Flyway en `Backend/src/main/resources/db/migration/`.
 - **Estado:** implementado
 - **Observaciones:** el requisito **no está cumplido**: dos de sus cinco criterios se apoyan en un relato, no en un artefacto reproducible. **Toda la evidencia citada pertenece a este repositorio (ENTREGA-FINAL-BIOPET); no se toma nada de ningún otro proyecto.** Comprobación independiente realizada en esta revisión sobre el artefacto docs/despliegue/ejemplo-backup-20260817.sql (64 762 bytes, codificado en UTF-16): contiene exactamente las **6 tablas** del esquema de este proyecto (`usuarios`, `mascotas`, `citas`, `consultas`, `vacunas`, `flyway_schema_history`), **6 secuencias** —incluida `seq_ficha_biopet`—, **4 triggers** y **7 rutinas** (`fn_resumen_mascotas_por_especie`, `fn_historial_clinico_mascota`, `fn_reporte_dashboard`, `fn_siguiete_numero_ficha`, `sp_actualizar_estado_citas_masivas`, `sp_registrar_consulta_validada` y la función de trigger `set_actualizado_en`), y los datos son `usuarios`: 1 fila y `mascotas`, `citas`, `consultas`, `vacunas`: 0 filas. Esto confirma los criterios 3 y 5 y corrobora el contenido declarado en BACKUP.md. **Lo que no está respaldado:** los criterios 2 y 4 —que la restauración se ejecutó realmente y que una rutina se invocó con éxito sobre la base restaurada— constan únicamente como narración en BACKUP.md sección 4; **no existe ningún log crudo archivado** de esa ejecución, a diferencia de otras evidencias del proyecto (`docs/mediciones/sec/raw/`, las corridas k6 o las de Lighthouse). Mientras no se archive esa salida, el requisito no puede declararse verificado. Además, el respaldo **no está automatizado dentro del repositorio**: no existe ningún guion en `scripts/` ni tarea programada que lo ejecute. BACKUP.md describe un procedimiento manual del operador y cierra con una lista de comprobación (backup diario programado, registro de sumas, retención de 30 días, restauración de prueba mensual) que es responsabilidad operativa y **no** está marcada como cumplida de forma continua.

REQ-NF-018 — Accesibilidad de la interfaz

- **Tipo:** Usabilidad
- **Prioridad:** Should
- **Enunciado:** Las pantallas auditadas de la aplicación —`/login` y `/mascotas`— deberán obtener una puntuación de la categoría *Accessibility* de Lighthouse igual o superior a 90 sobre 100, tanto en perfil móvil como en perfil de escritorio, y ese umbral deberá estar declarado como condición de fallo en la configuración de medición. Este requisito **no** exige ni declara conformidad con WCAG en ningún nivel.
- **Rationale:** requisito nuevo incorporado en esta revisión. La accesibilidad se medía desde la Tercera Entrega como parte de las corridas Lighthouse (y su resultado se citaba dentro de REQ-NF-005), pero no existía un requisito propio con un umbral declarado, de modo que el valor medido no estaba contrastado contra ninguna exigencia.

- **Criterios de aceptación:**

1. En las corridas Lighthouse sobre la ruta `/login`, la puntuación de Accessibility es ≥ 90 en perfil móvil y en perfil de escritorio.
2. En las corridas Lighthouse sobre la ruta `/mascotas` (vista autenticada), la puntuación de Accessibility es ≥ 90 en ambos perfiles.
3. El umbral de 0,90 está declarado a nivel **error** en las dos configuraciones de medición (perfil móvil y perfil de escritorio), de modo que una regresión por debajo de 90 haga fallar la comprobación.
4. Los archivos crudos de las corridas quedan archivados en el repositorio junto con un registro de procedencia que indique fecha, commit auditado, versiones de herramienta, URLs y perfiles.

- **Método de verificación:** medición Lighthouse — 12 archivos crudos archivados en `docs/mediciones/lighthouse/lhci-20260818-0538-*.json` (perfiles móvil y desktop $\times$ rutas `/login` y `/mascotas` $\times$ 3 corridas). Lectura directa del campo `categories.accessibility.score` de los 12 archivos: **91 en los doce casos**, sin ninguna dispersión, por encima del umbral de 90 (criterios 1 y 2). Criterio 3: inspección de `lighthousec.js` (línea 47) y `lighthousec.desktop.js` (línea 56), ambas con `'categories:accessibility': ['error', { minScore: 0.9 }]`. Criterio 4: archivo de procedencia `docs/mediciones/lighthouse/lhci-20260818-0538.meta.txt`, que registra `fecha_iso8601: 2026-08-18T05:38:06Z`, `commit_hash_corto: e94bb72`, `node_version: v24.18.0`, `lighthouse_cli_version: 0.14.0`, las dos URLs auditadas y los dos perfiles con sus respectivas configuraciones.

- **Trazabilidad:** configuración `lighthousec.js` y `lighthousec.desktop.js` (umbrales), guion `scripts/run-lighthouse.sh`, evidencias `docs/mediciones/lighthouse/` con `README.md` (procedencia y anonimización declarada) y `lhci-20260818-0538.meta.txt`; pantallas auditadas `frontend/src/app/features/login.component.ts` y `mascotas.component.ts`; requisito hermano de usabilidad REQ-NF-005.

- **Estado:** verificado

- **Observaciones:** el alcance de este requisito está deliberadamente limitado a lo que la evidencia existente demuestra, y no más: **(a) Sin declaración de norma. No se declara conformidad con WCAG 2.1 en ningún nivel (A, AA o AAA)**, ni con ninguna otra norma de accesibilidad. No se ha realizado auditoría manual, ni pruebas con lector de pantalla, ni revisión completa de navegación por teclado; Lighthouse cubre sólo un subconjunto automatizable de los criterios WCAG. La métrica obtenida (91/100) es la puntuación de esa herramienta, no un nivel de conformidad. **(b) Alcance de pantallas.** Las rutas auditadas son exactamente `/login` y `/mascotas`. La pantalla de vacunas (`frontend/src/app/features/vacunas.component.ts`) **no** está incluida en las corridas archivadas, por lo que el requisito no afirma nada sobre ella. **(c) Entorno de medición.** Las 12 corridas se ejecutaron contra `http://localhost:4200` sobre el commit `e94bb72`, no contra el despliegue de producción. **(d) Sumas de comprobación.** Los archivos `docs/mediciones/lighthouse/SHA256SUMS.txt` y `SHA256SUMS-ORIGINAL.txt` corresponden a la corrida **original del 2026-08-01** (carpeta `raw/`) y **no** cubren los 12 archivos `lhci-20260818-*` citados aquí; por eso el criterio 4 se apoya en el registro de procedencia `lhci-20260818-0538.meta.txt` y no en esas sumas. Archivar las sumas de la re-corrida del 2026-08-18 queda como mejora de trazabilidad.

3.3. Criterios INVEST aplicados a las historias de usuario

Las 24 historias de usuario de `docs/requisitos/historias/HistoriasUsuario.md` se evalúan aquí explícitamente contra los seis criterios INVEST de Cohn (Independent, Negotiable, Valuable, Estimable, Small, Testable), con justificación breve y verificable contra campos que ya existen en cada historia (no se introduce información nueva para esta evaluación).

Nota de vocabulario. La taxonomía de tres estados de la sección 1.3.1 se aplica a los **requisitos** (REQ-F/REQ-NF) de las secciones 3.1 y 3.2. Las historias de usuario conservan su propio campo de estado en `HistoriasUsuario.md`; las dos escalas no se mezclan ni se contradicen.

Negotiable, Valuable y Testable se cumplen en las 24 historias, sin excepción, verificado estructuralmente así:

- **Negotiable:** las 24 historias siguen el formato Connextra (“Como... quiero... para...”), que expresa intención y valor, no una solución de diseño fija; el “cómo” (campos exactos, validaciones) queda abierto

a negociación en el caso de uso y en la implementación.

- **Valuable:** cada historia declara un rol beneficiario explícito (Como <rol>) y está trazada a un requisito funcional con *rationale* propio en la sección 3.1, nunca a una tarea puramente técnica sin beneficiario.
- **Testable:** las 24 historias incluyen un bloque **gherkin** con al menos un escenario **Given/When/Then** verificable; las que aún no están implementadas lo declaran explícitamente como “comportamiento esperado, no implementado”, sin fingir que ya existe evidencia de ejecución.

Independent, Estimable y Small varían por historia (justificación por fila, tomada del campo *Dependencias* y del estado real de cada historia):

HU	Independent	Estimable	Small
HU-001	Sí	Sí — implementada, esfuerzo ya observado	Nota — 2 REQ-F relacionados
HU-002	Sí	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-003	Parcial — depende de HU-002 (requiere haber iniciado sesión previamente)	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-004	Parcial — depende de HU-002	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-005	Parcial — depende de HU-002	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-006	Parcial — depende de HU-002	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-007	Parcial — depende de HU-005 (control de acceso por rol)	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-008	Parcial — depende de HU-005	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-009	Parcial — depende de HU-005, HU-008	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-010	Parcial — depende de HU-005, HU-009	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-011	Parcial — depende de HU-005, HU-009	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-012	Parcial — depende de HU-007 (requiere que la mascota exista)	Parcial — hay código y pruebas reales para la obligación A de REQ-F-013; el esfuerzo de la obligación B sigue siendo estimación	Nota — 1 REQ-F con 2 obligaciones (REQ-F-013)
HU-013	Parcial — depende de HU-012	Parcial — sin artefacto entregado que medir	Sí — 1 REQ-F
HU-014	Parcial — depende de HU-007	Parcial — la obligación A de REQ-F-015 está cubierta y probada; el esfuerzo del calendario (obligación B) sigue siendo estimación	Nota — 1 REQ-F con 2 obligaciones (REQ-F-015)
HU-015	Parcial — depende de HU-007	Parcial — sin artefacto entregado que medir	Sí — 1 REQ-F

HU	Independent	Estimable	Small
HU-016	Parcial — depende de HU-012 (requiere historial clínico)	Parcial — sin artefacto entregado que medir	Sí — 1 REQ-F
HU-017	Parcial — depende de HU-007	Parcial — sin artefacto entregado que medir	Sí — 1 REQ-F
HU-018	Sí	Parcial — sin artefacto entregado que medir	Sí — 1 REQ-F
HU-019	Parcial — depende de HU-002 (requiere usuario autenticado)	Parcial — sólo la parte de autenticación tiene artefacto medible	Sí — 1 REQ-F
HU-020	Parcial — depende de HU-005 y HU-007	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-021	Parcial — depende de HU-001 (registro de usuario)	Parcial — sin artefacto entregado que medir	Sí — 1 REQ-F
HU-022	Parcial — depende de HU-005 (control de acceso por rol)	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-023	Parcial — depende de HU-007 (requiere que la mascota exista)	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-024	Sí	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F

Lectura honesta de “Independent”. La mayoría de historias depende de al menos otra (típicamente HU-002 “sesión iniciada” o HU-005/HU-007 “rol autorizado”/“mascota existente”), lo cual es normal y esperado en un sistema con autenticación y CRUD relacional: ninguna historia de gestión de un recurso es realmente independiente de “existe sesión” y “el recurso padre existe”. Se documenta la dependencia real en vez de declarar “Independent: Sí” de forma genérica sin sustento, que habría sido la alternativa más fácil pero menos honesta.

Lectura honesta de “Estimable”. Las historias ya implementadas son estimables con certeza retrospectiva: el código y las pruebas existen, su esfuerzo ya fue observado. Las parciales (HU-012, HU-014, HU-019) son estimables sólo para el subconjunto ya entregado — el esfuerzo del resto (historial consolidado expuesto por API, calendario interactivo, auditoría genérica de CRUD) sigue siendo una estimación no verificada. Las pendientes no tienen ningún artefacto que permita estimar con base empírica; su tamaño es una suposición heredada de la Entrega 1A, no una medición.

Nota sobre “Small”. Tres historias agrupan más de un requisito. HU-001 agrupa REQ-F-001 y REQ-F-002 porque ambos comparten el mismo flujo de entrada (POST /api/auth/registro) y serían artificialmente pequeños por separado. HU-012 y HU-014 corresponden cada una a un único requisito (REQ-F-013 y REQ-F-015) que agrupa dos obligaciones: ni la historia ni el requisito se dividen en identificadores nuevos —el valor para el usuario sigue siendo uno solo—; lo que se separa explícitamente dentro del bloque son sus obligaciones A y B, que es lo que permite describir con exactitud qué parte está hecha y cuál no.

Nota sobre REQ-F-026. El requisito de recuperación de contraseña incorporado en esta revisión **no tiene historia de usuario ni caso de uso todavía**; se deberán crear la historia y el caso de uso correspondientes al planificar su implementación. No se inventa aquí una historia que no exista en `HistoriasUsuario.md`, para no introducir una referencia colgante.

3.4. Tabla de control de completitud de requisitos

Auditoría mecánica de los campos obligatorios de la plantilla (sección 1.3.2) sobre los **44 requisitos** de las secciones 3.1 y 3.2 — 26 funcionales y 18 no funcionales, uno por identificador. Cada celda indica si el campo

está presente y con contenido sustantivo (**OK**) o ausente (**FALTA**). El campo *Observaciones* no se audita aquí porque es opcional por diseño.

Abreviaturas de columna: **Tip** = Tipo · **Pri** = Prioridad · **Enu** = Enunciado · **Rat** = Rationale · **Cri** = Criterios de aceptación · **Ver** = Método de verificación · **Tra** = Trazabilidad · **Est** = Estado (valor declarado).

ID	Tip	Pri	Enu	Rat	Cri	Ver	Tra	Est
REQ-F-001	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-002	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-003	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-004	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-005	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-006	OK	OK	OK	OK	OK	OK	OK	implementado
REQ-F-007	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-008	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-009	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-010	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-011	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-012	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-013	OK	OK	OK	OK	OK	OK	OK	implementado
REQ-F-014	OK	OK	OK	OK	OK	OK	OK	pendiente
REQ-F-015	OK	OK	OK	OK	OK	OK	OK	implementado
REQ-F-016	OK	OK	OK	OK	OK	OK	OK	pendiente
REQ-F-017	OK	OK	OK	OK	OK	OK	OK	pendiente
REQ-F-018	OK	OK	OK	OK	OK	OK	OK	pendiente
REQ-F-019	OK	OK	OK	OK	OK	OK	OK	pendiente
REQ-F-020	OK	OK	OK	OK	OK	OK	OK	implementado
REQ-F-021	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-022	OK	OK	OK	OK	OK	OK	OK	pendiente
REQ-F-023	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-024	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-025	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-F-026	OK	OK	OK	OK	OK	OK	OK	pendiente
REQ-NF-001	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-NF-002	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-NF-003	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-NF-004	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-NF-005	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-NF-006	OK	OK	OK	OK	OK	OK	OK	pendiente
REQ-NF-007	OK	OK	OK	OK	OK	OK	OK	implementado
REQ-NF-008	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-NF-009	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-NF-010	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-NF-011	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-NF-012	OK	OK	OK	OK	OK	OK	OK	implementado
REQ-NF-013	OK	OK	OK	OK	OK	OK	OK	verificado
REQ-NF-014	OK	OK	OK	OK	OK	OK	OK	pendiente
REQ-NF-015	OK	OK	OK	OK	OK	OK	OK	implementado
REQ-NF-016	OK	OK	OK	OK	OK	OK	OK	implementado
REQ-NF-017	OK	OK	OK	OK	OK	OK	OK	implementado
REQ-NF-018	OK	OK	OK	OK	OK	OK	OK	verificado

Resultado de la auditoría: 0 celdas FALTA sobre 44 requisitos × 8 campos obligatorios (352

celdas).

3.4.1. Resumen por estado

Estado	REQ-F	REQ-NF	Total
verificado (cumplidos)	15	11	26
implementado (no cumplidos)	4	5	9
pendiente (no cumplidos)	7	2	9
Total de requisitos	26	18	44

El número oficial de requisitos del SRS es 44: REQ-F-001 a REQ-F-026 (26 funcionales) y REQ-NF-001 a REQ-NF-018 (18 no funcionales). Un requisito, un identificador, una fila en esta tabla y una fila en docs/trazabilidad/matriz.csv: los conjuntos de identificadores de ambos documentos son **idénticos**, sin excedentes ni faltantes en ninguna dirección. Tres de esos 44 (REQ-F-013, REQ-F-015 y REQ-NF-012) agrupan dos obligaciones cada uno, separadas dentro de su bloque (sección 1.3.2) sin generar identificadores adicionales.

De los 44, **26 están cumplidos** (verificado) y **18 no lo están** (9 implementado y 9 pendiente), conforme a la lectura obligada de la sección 1.3.1.

3.4.2. Cobertura de los requisitos de prioridad Must Hay **25 requisitos Must** (11 funcionales y 14 no funcionales). De ellos, **20 están verificado** y **5 no**. No se declara una cobertura de 25/25:

Requisito Must	Estado	Por qué no está verificado
REQ-F-006	implementado	<code>ConsultaService.listar()</code> no filtra por propietario para <code>ROLE_DUENO</code> ; el criterio 3 de aislamiento falla.
REQ-NF-007	implementado	El endpoint de salud está verificado, pero no existe registro fechado de sondeos que permita calcular un porcentaje de disponibilidad (criterio 4).
REQ-NF-014	pendiente	No existe ninguna política implementada ante indisponibilidad de Redis: hoy el fallo no está controlado.
REQ-NF-015	implementado	El aislamiento falla en el listado de consultas médicas (misma causa que REQ-F-006).
REQ-NF-016	implementado	El límite superior de 80 caracteres de la política de contraseñas no tiene ninguna prueba automatizada.

Dos de los cinco comparten una única causa raíz —la falta de filtrado por propietario en `ConsultaService.listar()`— y un tercero es la ausencia de política ante fallo de Redis; ambas están registradas en la sección 7.4 como acciones que **requieren modificar código de backend** y quedan fuera del alcance de esta revisión documental. Los otros dos (REQ-NF-007 y REQ-NF-016) se cierran archivando evidencia, sin tocar código.

4. Trazabilidad (resumen)

La matriz completa vive en `docs/trazabilidad/matriz.csv` (bloque A.3.3 de la Guía), con una fila por requisito y las columnas `id_requisito`, `tipo`, `prioridad_moscow`, `historia_usuario`, `caso_de_uso`, `modulo_codigo`, `endpoint_api`, `prueba_automatizada`, `tipo_acceso`, `evidencia_empirica` y `estado`. Este SRS es la fuente de verdad para las columnas `id_requisito`, `tipo`, `prioridad_moscow` y `estado`; la matriz no debe declarar un requisito que no exista aquí, ni omitir ninguno de los declarados en la sección 3.

La consistencia entre ambos documentos la comprueba automáticamente `scripts/validate-traceability.sh` (job `traceability` del CI), que verifica que todo REQ-F-/REQ-NF- del SRS tenga fila en la matriz y viceversa, que ninguna fila quede sin historia, caso de uso ni prueba, que toda referencia HU-/CU- exista realmente en `HistoriasUsuario.md/CasosDeUso.md`, y que no haya identificadores duplicados.

Nota sobre los requisitos con dos obligaciones. REQ-F-013, REQ-F-015 y REQ-NF-012 agrupan cada uno dos obligaciones, separadas dentro de su bloque como *Obligación A* y *Obligación B* (sección 1.3.2). **No generan identificadores adicionales.** Cada uno tiene un único identificador, un único bloque en la sección 3, una única fila en la tabla de control 3.4 y una única fila en `docs/trazabilidad/matriz.csv`, con **un solo** valor de Estado, que describe el requisito completo: basta con que una de sus dos obligaciones no esté cumplida para que el requisito no pueda declararse **verificado**. El campo `evidencia_empirica` de la fila de la matriz detalla la situación de cada obligación por separado. Así la trazabilidad histórica hacia las historias, los casos de uso y los identificadores RF-NN de la Entrega 1A se mantiene intacta.

Correspondencia exacta de identificadores. El conjunto de identificadores de la sección 3 de este SRS y el conjunto de la columna `id_requisito` de `docs/trazabilidad/matriz.csv` son **el mismo conjunto de 44 elementos**: la diferencia SRS – matriz es vacía y la diferencia matriz – SRS es vacía. Esta igualdad la comprueba `scripts/validate-traceability.sh` en el CI y, campo a campo, `docs/requisitos/tools/auditoria-srs.py`.

Correspondencia REQ-F ↔ HU ↔ CU:

REQ-F	HU	CU	REQ-F	HU	CU
001	HU-001	CU-01	014	HU-013	CU-13
002	HU-001	CU-01	015	HU-014	CU-14
003	HU-002	CU-02	016	HU-015	CU-15
004	HU-003	CU-03	017	HU-016	CU-16
005	HU-004	CU-04	018	HU-017	CU-17
006	HU-005	CU-05	019	HU-018	CU-18
007	HU-006	CU-06	020	HU-019	CU-19
008	HU-007	CU-07	021	HU-020	CU-20
009	HU-008	CU-08	022	HU-021	CU-21
010	HU-009	CU-09	023	HU-022	CU-22
011	HU-010	CU-10	024	HU-023	CU-23
012	HU-011	CU-11	025	HU-024	CU-24
013	HU-012	CU-12	026	(sin HU/CU aún)	(sin HU/CU aún)

REQ-F-026 es el único requisito funcional sin historia ni caso de uso: se incorporó en esta revisión y su historia y caso de uso deberán crearse al planificar su implementación. No se declara ninguna referencia a historias o casos de uso inexistentes.

Trazabilidad de los requisitos no funcionales. Ningún REQ-NF declara historia de usuario, porque ninguno la tiene: su trazabilidad apunta a decisiones de arquitectura, configuración, clases, pruebas, guiones, mediciones, evidencias y normas, tal como se detalla en el campo *Trazabilidad* de cada bloque de la sección 3.2. Este es un cambio deliberado respecto de la práctica de inventar historias para rellenar la columna.

Trazabilidad de los procedimientos almacenados. Las seis rutinas de `db/procs/` se declaran en la trazabilidad del requisito al que sirven, no como requisitos propios:

Rutina	Requisito al que sirve	Situación
<code>fn_resumen_mascotas_por_especie</code>	REQ-F-021	Consumida por <code>MascotaService.resumenPorEspecie</code> y expuesta en <code>GET /api/mascotas/resumen-especies</code> .
<code>fn_historial_clinico_mascota</code>	REQ-F-013 (obligación B)	Invocable desde el repositorio y probada; sin endpoint que la exponga.
<code>fn_reporte_dashboard</code>	REQ-F-019	Invocable desde el repositorio y probada; sin endpoint ni exportación.
<code>sp_registrar_consulta_validada</code>	REQ-F-013 (obligación A)	Validación cruzada de mascota y veterinario; probada, no invocada desde <code>ConsultaService</code> .
<code>sp_actualizar_estado_citas_masivas</code>	REQ-F-015 (obligación A)	Cierre masivo de citas por veterinario y fecha límite; probada, no expuesta como endpoint.
<code>fn_siguiete_numero_ficha</code>	REQ-NF-013 (y, a futuro, REQ-F-018)	Generador de códigos correlativos; probado, sin consumidor en la aplicación todavía.

Ninguna de las seis genera un requisito funcional nuevo: cuatro implementan la capa de datos de requisitos que ya existen, una es consumida en producción y la sexta es un objeto disponible sin consumidor, declarado como tal en las Observaciones de REQ-NF-013.

4.1. Trazabilidad histórica: identificadores originales → identificadores actuales

La retroalimentación oficial del SGA de la Entrega 1A señaló que “los RF-WEB se remapean a RF-16/RF-17 sin matriz de trazabilidad explícita en esta entrega” (ver `docs/observaciones/OBSERVACIONES.md`, OBS-03). La siguiente tabla consolida el vínculo entre los identificadores originales (RF-NN/RF-WEB-NN, Entrega 1A) y los actuales (REQ-F-NNN), sin omitir ningún requisito funcional y sin inventar ningún origen.

Identificador anterior (Entrega 1A)	Identificador actual	Descripción	CU	HU	Estado
RF-01	REQ-F-001	Registro de usuario dueño de mascota	CU-01	HU-001	verificado
RF-01, RF-02	REQ-F-008	Creación de mascota asociada a un dueño existente	CU-07	HU-007	verificado
RF-02	REQ-F-009	Listado paginado de mascotas activas por propietario/rol	CU-08	HU-008	verificado
RF-02	REQ-F-011	Actualización de mascota con verificación de propiedad	CU-10	HU-010	verificado

Identificador anterior (Entrega 1A)	Identificador actual	Descripción	CU	HU	Estado
RF-03, RF-04	REQ-F-013	Registro de atención médica (obligación A) y consulta del historial clínico consolidado (obligación B)	CU-12	HU-012	implementado
RF-05	REQ-F-014	Registro de medicamentos prescritos	CU-13	HU-013	pendiente
RF-06	REQ-F-015	Gestión de citas por API (obligación A) y calendario interactivo (obligación B)	CU-14	HU-014	implementado
RF-07	REQ-F-022	Notificaciones por correo electrónico (cierre de OBS-02)	CU-21	HU-021	pendiente
RF-08, RF-09	REQ-F-016	API de recepción de telemetría de dispositivos IoT	CU-15	HU-015	pendiente
RF-10	REQ-F-017	Recomendaciones clínicas informativas (redacción cerrada en OBS-04)	CU-16	HU-016	pendiente
RF-11, RF-12	REQ-F-018	Comprobantes de pago digitales	CU-17	HU-017	pendiente
RF-13, RF-WEB-02	REQ-F-006	Control de acceso por rol y por propietario	CU-05	HU-005	implementado
RF-14	REQ-F-019	Reportes estadísticos exportables	CU-18	HU-018	pendiente
RF-15	REQ-F-020	Auditoría de operaciones del sistema	CU-19	HU-019	implementado
RF-16, RF-WEB-01	REQ-F-003	Autenticación mediante usuario y contraseña	CU-02	HU-002	verificado
RF-17, RF-WEB-04	REQ-F-005	Cierre de sesión con revocación de token	CU-04	HU-004	verificado

Identificador anterior (Entrega 1A)	Identificador actual	Descripción	CU	HU	Estado
RNF-03, RNF-WEB-03	REQ-F-004	Renovación de sesión (refresh)	CU-03	HU-003	verificado

Requisitos funcionales sin origen en la Entrega 1A. Agregados durante la Tercera Entrega para documentar funcionalidad ya presente en el código: REQ-F-002 (rechazo de correo duplicado), REQ-F-007 (consulta del perfil propio), REQ-F-010 (consulta de mascota por id), REQ-F-012 (baja lógica de mascota) y REQ-F-021 (resumen de mascotas por especie, exigido por el bloque A.2.2 de la Guía). Agregados durante la Unidad IV: REQ-F-023 (gestión administrativa de usuarios), REQ-F-024 (gestión de vacunas) y REQ-F-025 (consulta externa de especies con caché). Agregado en la revisión 2026-09-11: REQ-F-026 (recuperación de contraseña). Ninguno sustituye ni duplica un identificador RF-NN original.

Requisitos no funcionales sin origen en la Entrega 1A. REQ-NF-004 (claims RFC 7519), REQ-NF-009 (OWASP A09), REQ-NF-010 (OWASP A07) y REQ-NF-013 (estrategia híbrida de acceso a datos), exigidos por los bloques A.1, A.2 y C.2 de la Guía. Agregados en la revisión 2026-09-11: REQ-NF-014 (indisponibilidad de Redis), REQ-NF-015 (aislamiento de datos por propietario), REQ-NF-016 (política de contraseñas), REQ-NF-017 (respaldo y recuperación) y REQ-NF-018 (accesibilidad).

5. Modelo de datos (referencia)

El diccionario de datos completo de las entidades implementadas (**usuarios, mascotas, citas, consultas, vacunas**) está en `docs/diccionario_datos.md`, sincronizado con las migraciones Flyway de `Backend/src/main/resources/db/migration/` y con `database/migrations/V1__schema_inicial.sql`. El modelo entidad-relación conceptual completo (incluyendo las entidades pendientes: `Dispositivo_IoT`, `Chat_Triage`, `Producto_Servicio`, `Factura`, `Detalle_Factura`, `Proveedor`, `Mercaderia`, `Detalle_Ingreso`) proviene de la Entrega 1A (`PFC_Entrega1A_BMT.pdf`, sección 6) y se conserva como visión de producto pendiente de materialización, sin duplicar aquí el DDL completo. Los estados y transiciones de las entidades implementadas están en la sección 2.9.

Diagrama entidad-relación (DER) — dos artefactos distintos, no intercambiables (cierre de OBS-05):

- `docs/diagrams/der-biopet/der-biopet.png` — renderizado generado a partir de la fuente `Graphviz der-biopet.dot`. Es un diagrama **dibujado**, no una exportación de una herramienta de modelado de base de datos.
- `docs/observaciones/evidencias/DER-BIOPET-pgAdmin-ERD-Tool.png` — exportación **real** generada desde **pgAdmin 4**, herramienta **ERD Tool**, solicitada explícitamente por la retroalimentación oficial del SGA de la Entrega 1B: “Exportar el DER desde pgAdmin 4 (ERD Tool) como PNG de alta resolución para el informe final” (ver `docs/observaciones/OBSERVACIONES.md`, OBS-05). PNG válido (firma de archivo verificada), 768×883 px, generado directamente sobre el esquema real de PostgreSQL, no sobre la fuente `.dot`.

6. Interfaces de usuario (referencia)

Los wireframes conceptuales (login unificado, dashboard por rol, historial clínico, gestión de citas) están documentados en la Entrega 1A (`PFC_Entrega1A_BMT.pdf`, sección 7). La interfaz realmente implementada en v1.0.0 está en `frontend/src/app/features/`:

Componente	Cubre	Requisitos
<code>login.component.ts</code>	Autenticación	REQ-F-003, REQ-F-004, REQ-F-005

Componente	Cubre	Requisitos
<code>mascotas.component.ts</code>	CRUD de mascotas con paginación, formularios, confirmación de borrado, resumen por especie y atributos de accesibilidad	REQ-F-008 a REQ-F-012, REQ-F-021
<code>vacunas.component.ts</code>	CRUD de vacunas	REQ-F-024

Módulos con API real y sin pantalla propia. Citas (REQ-F-015, obligación A), consultas médicas (REQ-F-013, obligación A), gestión administrativa de usuarios (REQ-F-023) y consulta externa de especies (REQ-F-025) tienen backend real y verificado, pero **no** tienen componente en `frontend/src/app/features/`. El calendario interactivo de citas es la obligación B de REQ-F-015, no implementada. Ninguna de estas pantallas cuenta con wireframe formal actualizado; se deja como observación abierta en la sección 7.

Las interfaces del sistema con su entorno (software, hardware y comunicaciones) están en la sección 2.7.

7. Observaciones e información pendiente

Esta sección se revisó contra el estado real del repositorio en el commit de revisión 8130ee00b0083808fc60567018589e38302b375d. Se mantiene el mismo principio que en versiones anteriores: no se marca nada como resuelto sin evidencia verificable, y no se inventa contenido para cerrar un pendiente.

7.1. Cerrado en revisiones anteriores

- **ADR de cambio de pila tecnológica** — CERRADO. `docs/adr/ADR-002-pila-tecnologica.md` existe, está en estado “Aceptado” y documenta explícitamente la migración de ASP.NET Core 8 (ADR-001, Entrega 1A) a Java 21 / Spring Boot 3.2, con contexto, evidencia verificable en `Backend/pom.xml` y alternativas consideradas.
- **Evidencia empírica de rendimiento (REQ-NF-001)** — CERRADO. 10 corridas k6 reales (5 en caliente, 5 en frío) del 2026-09-03 en `docs/mediciones/perf/`, con p95 entre 6,04 ms y 18,56 ms, muy por debajo del umbral de 200/500 ms.
- **Evidencia OWASP A07/A09 (REQ-NF-009, REQ-NF-010)** — CERRADO. `docs/mediciones/sec/A07-authentication.md` y `A09-logging.md` contienen capturas HTTP (curl) y de log reales, no sólo pruebas JUnit.
- **Lighthouse** — **perfil de escritorio y re-ejecución tras el fix de SEO** — CERRADO. Re-corrida real del 2026-08-18 (`docs/mediciones/lighthouse/lhci-20260818-0538-*.json`, 12 archivos). SEO pasó de 82/100 a 100/100 en las 12 corridas; Performance ≥ 90 en todos los casos; Accessibility 91 en todos (ver REQ-NF-005 y REQ-NF-018).
- **PRISMA (trabajos relacionados)** — CERRADO con 10 estudios y 39 referencias BibTeX integradas en `docs/informe/referencias.bib`.
- **INVEST** — CERRADO. Sección 3.3, con los seis criterios aplicados explícitamente a las 24 historias de usuario.
- **Bitácora de observaciones** (`docs/observaciones/OBSERVACIONES.md`): cubre OBS-01 a OBS-15 con fuente primaria (capturas SGA) y verificación cruzada contra `git log`.
- **REQ-NF-013 (estrategia híbrida de acceso a datos)** — CERRADO. `scripts/audit-sql-dynamic.sh` se reprodujo sobre el commit exacto del tag histórico v1.0.0 (0d5cd525ce648cca7219da204e16fa622e671a87) con resultado “0 hallazgos en 7 archivos. PASA”; evidencia archivada en `docs/mediciones/sec/audit-sql-dynamic-v1.0.0.txt`.

7.2. Cerrado en la revisión 2026-09-11 (esta revisión)

- **Criterios de aceptación en todos los requisitos** — CERRADO. Los 47 bloques de las secciones 3.1 y 3.2 declaran criterios de aceptación observables en forma *Dado/cuando/entonces*, que permiten responder CUMPLE / NO CUMPLE. Antes sólo dos requisitos los tenían.

- **Bloques individuales para REQ-F-013 a REQ-F-020 — CERRADO.** Los ocho requisitos que aparecían únicamente como filas de una tabla resumen tienen ahora bloque completo con la plantilla única.
- **Taxonomía de estados — CERRADO.** La sección 1.3.1 define exactamente tres estados y todo el documento los usa sin excepción. Los valores `parcial`, `verificado parcialmente`, `implementado pero no verificado`, `pendiente de evidencia archivada` y `verificado en configuración TLS de desarrollo` han desaparecido del campo Estado; sus matices se trasladaron al campo Observaciones del requisito correspondiente.
- **Requisitos compuestos — CERRADO.** REQ-F-013, REQ-F-015 y REQ-NF-012 se descomponen en subrequisitos con estado propio, conservando el identificador base y la trazabilidad histórica.
- **Sección 2.6 (servicios externos) — CERRADO.** Se corrige la contradicción: la integración con API Ninjas *sí* existe y se declara explícitamente, junto con las tres integraciones que no existen.
- **REQ-NF-007 (disponibilidad) — CERRADO** como redacción. El enunciado no verificable se reformula en criterios medibles y el compromiso académico se traslada a la sección 2.5 como restricción operacional.
- **Interfaces externas, matriz de permisos y estados del dominio — CERRADO.** Secciones 2.7, 2.8 y 2.9, construidas exclusivamente desde el código de este repositorio.
- **Control documental — CERRADO.** Fecha de emisión explícita, historial de revisiones respaldado por commits reales (sección 1.6) y eliminación del índice manual duplicado.
- **Correspondencia exacta SRS ↔ matriz — CERRADO.** Una versión intermedia de esta revisión había descompuesto REQ-F-013, REQ-F-015 y REQ-NF-012 en subrequisitos con sufijo de letra, lo que dejaba al SRS con identificadores que la matriz no contenía. Se revirtió: los tres conservan un identificador único y separan sus obligaciones dentro del bloque, de modo que ambos documentos declaran exactamente los mismos 44 identificadores.
- **Referencia a prueba inexistente en REQ-F-001 — CERRADO.** Se retira la cita al test “Auth ControllerTest (registro exitoso)”, que no existe, y se sustituye por la evidencia HTTP real que sí existe.

7.3. Limitaciones declaradas que siguen abiertas

Ninguna de estas se cierra con datos inventados.

1. **Aislamiento de datos incompleto en Consultas (REQ-F-006, REQ-F-013, REQ-NF-015).** `ConsultaService.listar()` no filtra por propietario para `ROLE_DUEÑO`: ambas ramas del `if` devuelven `consultaRepository.findAllByActivoTrue(pageable)`, y `ConsultaRepository` no declara el método de filtrado que sí tienen `CitaRepository` y `VacunaRepository`. Un dueño autenticado puede ver consultas médicas de mascotas ajenas a través de `GET /api/consultas`. **Requiere cambio de código de backend**, fuera del alcance de esta revisión documental.
2. **Sin política definida ante indisponibilidad de Redis (REQ-NF-014).** Con Redis caído, `TokenBlacklistService.isRevoked` lanza `RedisConnectionFailureException`, que no captura ni `JwtAuthenticationFilter` ni `GlobalExceptionHandler`, produciendo un error 500 sin `ProblemDetail`. No hay `CacheErrorHandler` registrado. El requisito especifica la política que deberá implementarse. **Requiere cambio de código de backend.**
3. **Cobertura de pruebas del módulo de Consultas (REQ-F-013).** `GET /api/consultas` (listado) y `PUT /api/consultas/{id}` no tienen prueba automatizada dedicada: `ConsultaControllerTest` tiene 6 pruebas frente a las 25 de `CitaControllerTest` para el módulo equivalente.
4. **Hit ratio de caché Redis (REQ-NF-012, obligación B).** Sigue sin una medición de `keyspace_hits/keyspace_misses`; sólo hay evidencia de TTL y existencia de clave bajo carga.
5. **Compatibilidad de navegadores (REQ-NF-006).** Sin evidencia archivada de ejecución de los flujos críticos en Chrome, Firefox y Edge, y sin configuración específica de compatibilidad en el repositorio (no hay `browserslist` propio). Ninguno de sus criterios tiene artefacto, por lo que el requisito queda pendiente.
6. **Disponibilidad sostenida (REQ-NF-007).** No existe monitoreo continuo ni registro fechado de sondeos a `/actuator/health` durante una ventana de evaluación; no se declara ninguna cifra de disponibilidad histórica.

7. **Auditoría de operaciones de negocio (REQ-F-020).** Sólo se auditan los eventos de autenticación; no hay registro de las operaciones CRUD sobre mascotas, citas, consultas, vacunas ni usuarios.
8. **Historial clínico consolidado sin endpoint (REQ-F-013, obligación B).** La rutina `fn_historial_clinico_mascota` y su repositorio existen y están probados, pero ningún controlador los expone; además la rutina devuelve un resumen consolidado, no la lista cronológica completa de eventos.
9. **Reportes exportables sin endpoint (REQ-F-019).** `fn_reporte_dashboard` existe y está probada; la exportación a PDF y Excel y el endpoint no existen.
10. **fn_siguiente_numero_ficha sin consumidor (REQ-NF-013).** El generador de códigos correlativos está implementado y probado, pero ningún servicio de la aplicación lo invoca; su consumidor natural es REQ-F-018.
11. **Cobertura de GET /api/vacunas/mascota/{mascotaId} (REQ-F-024).** Sin prueba automatizada dedicada; la corrida Newman archivada (`docs/mediciones/postman/newman-report.json`, 2026-07-31) es anterior a la colección de vacunas y no puede citarse como evidencia de su ejecución.
12. **Recuperación de contraseña ausente (REQ-F-026).** No existe implementación alguna; hoy la única vía de restablecimiento es que un `ROLE_ADMIN` sobrescriba la contraseña. Se incorpora como requisito pendiente, con su historia de usuario y su caso de uso por crear.
13. **Accesibilidad (REQ-NF-018).** Se declara únicamente el umbral de la métrica automatizada de Lighthouse (≥ 90 , medido 91 en las 12 corridas). **No se declara conformidad con WCAG 2.1 en ningún nivel:** no hay auditoría manual, pruebas con lector de pantalla ni revisión completa de navegación por teclado. La pantalla de vacunas no está entre las rutas auditadas, la medición se hizo contra `http://localhost:4200` y las sumas SHA-256 archivadas corresponden a la corrida del 2026-08-01, no a las 12 corridas del 2026-08-18 que se citan como evidencia.
14. **Restauración sin log archivado y respaldo no automatizado (REQ-NF-017).** El procedimiento está documentado y el volcado de ejemplo `docs/despliegue/ejemplo-backup-20260817.sql` es real y verificable (6 tablas, 6 secuencias, 4 triggers y 7 rutinas de este proyecto), pero la ejecución de la restauración y la invocación de una rutina sobre la base restaurada constan sólo como narración en `BACKUP.md`, sin log crudo archivado. Además no existe guion ni tarea programada en el repositorio que ejecute el respaldo diario: es un procedimiento manual del operador.
15. **Límite de intentos de login por instancia (REQ-NF-010).** El contador vive en memoria del proceso; en un despliegue multiinstancia el límite se aplicaría por instancia. El despliegue actual es de instancia única.
16. **Separación de capas sin control automatizado (REQ-NF-011).** No existe una regla de arquitectura automatizada (por ejemplo ArchUnit) que impida introducir una dependencia de controlador a repositorio.
17. **Wireframes desactualizados.** Los de la pantalla de Mascotas y de los módulos pendientes (historial clínico, citas, facturación) siguen siendo los de la Entrega 1A. El producto real y su documentación de arquitectura (C4, DER, este SRS) sí reflejan el sistema implementado.
18. **Pantallas sin interfaz propia.** Citas, historial clínico consolidado, gestión administrativa de usuarios y consulta externa de especies tienen backend real, pero ninguna tiene componente en `frontend/src/app/features/`.
19. **Sin SIEM centralizado (REQ-NF-009).** Los eventos de auditoría de autenticación quedan en el log de la aplicación.
20. **Límite superior de contraseña sin prueba (REQ-NF-016).** El máximo de 80 caracteres consta en la anotación `@Size(min = 8, max = 80)` de `RegistroRequest` y `UsuarioRequest`, pero ninguna prueba envía una contraseña más larga para comprobar el 422. Es la única regla de la política que carece de cobertura automatizada.

7.4. Acciones que requieren cambio de código (fuera del alcance de esta revisión)

Los puntos 1 y 2 de la lista anterior son los únicos que impiden que un requisito **Must** alcance el estado **verificado** y que **no** pueden resolverse documentalmente:

#	Cambio necesario	Requisitos que desbloquea
1	Añadir <code>findAllByMascota_Duenio_IdAndActivoTrue</code> a <code>ConsultaRepository</code> y usarlo en la rama <code>ROLE_DUEÑO</code> de <code>ConsultaService.listar()</code> , con prueba de acceso cruzado en <code>ConsultaControllerTest</code> .	REQ-F-006, REQ-F-013, REQ-NF-015
2	Capturar el fallo de conexión con Redis en <code>TokenBlacklistService/JwtAuthenticationFilter</code> y traducirlo a 503 con <code>ProblemDetail</code> (<i>fail-closed</i> para la revocación), y registrar un <code>CacheErrorHandler</code> que degrade la caché a consulta directa (<i>fail-open</i> limitado a la caché), con la prueba correspondiente.	REQ-NF-014

Ambas quedan registradas como acciones de seguimiento para el propietario de los módulos correspondientes. Este documento **no modifica código**.